

République du Cameroun

Paix-Travail-Patrie

AFRICAN INSTITUTE OF COMPUTER SCIENCES



Représentation du Cameroun

BP: 13719 Yaoundé (Cameroun)

Tel: (237) 22 21 11 43 / (237) 99 99 82 49

Site web: www.iaicameroun.com

Republic Of Cameroon

Peace-Work-Fatherland

Batch Smart Technologies Sarl



Batch Smart Technologies Sarl

BP: 33160 Yaoundé

Email: Admin@bstsarl.com

Site web: bstsarl.com

RAPPORT DE FIN DE STAGE

THEME :

**PLATEFORME DE CONCEPTION, CONFIGURATION
AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX
INFORMATIQUE : cas de NetBot**

Stage effectué du 1er juillet 2025 au 30 septembre 2025

En vue de l'obtention du **Diplôme de Technicien Supérieur (DTS) en Informatique**
option Système Réseau

Rédigé par :

NKONO NDEME Miguel

Etudiant au niveau II à l'Institut Africain d'Informatique

Sous la supervision de :

Académique :

M. ABOGO Jeannot

Enseignant à l'IAI Cameroun

Professionnel :

M. NGAMO NGAMO Leonel

Développeur à BST Sarl

Année académique 2024-2025



**PLATEFORME DE CONCEPTION, CONFIGURATION
AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX
INFORMATIQUE : cas de NetBot**





DÉDICACE

À
La famille
NDEME
ASSEYI



REMERCIEMENTS

Nous tenons à remercier toutes les personnes qui ont contribué de près ou de loin au bon déroulement de notre stage, à la réalisation de ce rapport jusqu'à la présentation de notre travail. Nos sincères remerciements s'adressent :

- ✓ Au représentant résident de l'IAI-Cameroun M. **Armand Claude ABANDA**, pour sa vision et son engagement à nous fournir une formation de qualité ainsi que ses précieux conseils ;
- ✓ Le PDG de BATCH SMART TECHNOLOGIE DR. **OMBOLO Célestin** pour son accueil et son attention apporté à tous ;
- ✓ Notre encadrant professionnel M. **NGAMO NGAMO Leonel** pour son guide et ses précieux conseils lors de notre passage ;
- ✓ Notre encadrant académique M. **ABOGO Jeannot** pour son accompagnement et ses services ;
- ✓ Un grand merci à nos parents pour leurs sacrifices et dévouements ;
- ✓ À mes deux précieuses guides envoyées par le bon Dieu notamment **Mm SIODJIE Anges** avec *S* et **Mm BANA** en majuscule **Christelle** pour leur apport à la fois académique, professionnel, émotionnel et social sans qui nous n'aurions pas pu produire ce document ;
- ✓ Nos camarades de promotions et toutes personnes de près ou de loin ayant participé à notre évolution.



SOMMAIRE

DÉDICACE	III
REMERCIEMENTS.....	IV
SOMMAIRE.....	V
LISTE DES TABLEAUX.....	VI
LISTE DES FIGURES	VII
GLOSSAIRE.....	VIII
RESUME	IX
ABSTRACT.....	X
INTRODUCTION GÉNÉRALE	2
PARTIE I : PHASE D'INSERTION	3
PARTIE II : PHASE TECHNIQUE	12
CONCLUSION GÉNÉRALE.....	124
BIBLIOGRAPHIE.....	IX
WEBOGRAPHIE	X
TABLE DES MATIÈRES.....	XII



LISTE DES TABLEAUX

Tableau 1:critique de l'existant-----	18
Tableau 2 Tableau des besoins fonctionnels de notre application-----	
Tableau 3 : Liste des intervenants du projet-----	31
Tableau 4 tableau comparatif entre UML et MERISE-----	41
Tableau 5 composant d'un diagramme de classe -----	46
Tableau 6 types de relation -----	46
Tableau 7 formalisme d'une description textuelle-----	50
Tableau 8 description textuelle du cas s'authentifier -----	51
Tableau 9 description textuelle du cas d'utilisation monter architecture physique ----	52
Tableau 10 Description textuelle du cas d'utilisation configurer réseau -----	55
Tableau 11 description textuelle du cas d'utilisation déployé réseau -----	58
Tableau 12 formalisme du diagramme de communication -----	61
Tableau 13 formalisme du diagramme de séquence -----	65
Tableau 14 formalisme du diagramme d'activité -----	71
Tableau 15 formalisme du diagramme de classe -----	80
Tableau 16 règles métiers de notre système-----	83
Tableau 17 formalisme du diagramme d'état transition -----	84
Tableau 18 formalisme du diagramme de paquetage -----	87
Tableau 19 différents logiciels utilisés pour la réalisation de notre solution -----	92
Tableau 20 différentes technologies utilisées en frontend pour réaliser notre solution. -----	93
Tableau 21 différentes technologies utilisées en backend pour la réalisation de notre application -----	94
Tableau 22 formalisme du diagramme de déploiement -----	97
Tableau 23 formalisme du diagramme de composant -----	99



LISTE DES FIGURES

Figure 1 localisation de bst -----	7
Figure 2 : organigramme de batch smart technologies -----	8
Figure 3 Ressources de BATCH SMART TECHNOLOGIES -----	
Figure 4 : Diagramme de gantt de notre projet -----	33
Figure 5 exemple de diagramme de cas d'utilisation -----	47
Figure 6 diagramme de cas d'utilisation globale -----	47
Figure 7 diagramme de cas d'utilisation configurer réseau -----	48
Figure 8 diagramme de cas d'utilisation spécifique déployé réseau -----	49
Figure 9 diagramme de cas d'utilisation spécifique montage architecture physique -----	
Figure 10 diagramme de communication du cas s'authentifier -----	62
Figure 11 diagramme de communication du cas monter architecture physique -----	62
Figure 12 diagramme de communication du cas configurer réseau -----	63
Figure 13 déployer réseau -----	64
Figure 14 exemple de diagramme de séquence -----	66
Figure 15 diagramme de séquence du cas s'authentifier -----	67
Figure 16 diagramme de séquence du cas d'utilisation monter architecture physique -----	68
Figure 17 diagramme de séquence du cas configurer réseau -----	69
Figure 18 diagramme de séquence du cas déployé réseau -----	70
Figure 19 exemple de diagramme d'activité -----	72
Figure 20 diagramme d'activité du cas s'authentifier -----	72
Figure 21 diagramme d'activité du cas configurer réseau -----	73
Figure 22 diagramme d'activité du cas monter architecture physique -----	74
Figure 23 diagramme d'activité du cas déployé réseau -----	75
Figure 24 formalisme d'un diagramme de classe -----	81
Figure 25 diagramme de classe de notre système -----	
Figure 26 formalisme du diagramme d'état transition -----	85
Figure 27 diagramme d'état transition du cas de connexion -----	85
Figure 28 Diagramme d'état transition d'un utilisateur -----	86
Figure 29 architecture physique n-tiers -----	95
Figure 30 formalisme du diagramme de déploiement -----	98
Figure 31 diagramme de déploiement de la solution -----	98
Figure 32 formalisme du diagramme de composant -----	100
Figure 33 diagramme de composant de notre solution -----	100



GLOSSAIRE

- **Automatisation réseau :**
Processus d'utilisation des logiciels pour configurer, gérer et opérer les équipements réseau (routeurs, commutateurs, pare-feu) avec une intervention humaine réduite. NetBot permet aux administrateurs de générer et valider des configurations complexes via une interface intuitive.
- **Intelligence artificielle (IA) :**
Ensemble de techniques permettant à un système d'imiter l'intelligence humaine pour analyser des données, détecter des patterns et prendre des décisions. Dans NetBot, l'IA est utilisée pour la génération automatique de configurations et la détection d'erreurs.
- **Laravel :**
Framework PHP open-source utilisé pour le développement d'applications web. Dans NetBot, Laravel sert de backend pour la gestion des utilisateurs, l'authentification, et la fourniture d'API RESTful.
- **Vue.js :**
Framework JavaScript progressif pour construire des interfaces utilisateur. NetBot utilise Vue.js pour créer une interface interactive de modélisation réseau et de visualisation des configurations.
- **Flask :**
Micro-framework web Python léger et modulaire. Dans NetBot, Flask est utilisé pour les services spécialisés comme la génération de configurations et l'analyse syntaxique via des micro services **API REST (Representational State Transfer)** style d'architecture pour les systèmes distribués permettant des interactions entre clients et serveurs. NetBot utilise des API REST pour la communication entre le frontend (Vue.js) et le backend (Laravel/Flask).
- **Configuration :**
Ensemble de paramètres et commandes définissant le comportement des équipements réseau. NetBot génère automatiquement ces configurations pour différents protocoles (VLAN, OSPF, BGP, etc.).



RESUME

Ce rapport démontre le potentiel transformateur de **l'intelligence artificielle** dans la conception, le déploiement et la gestion des réseaux informatiques, en adressant les défis critiques rencontrés par les ingénieurs réseaux - notamment la **complexité des configurations**, les **risques d'erreurs humaines** et le **fossé entre théorie et pratique**. Il présente une solution innovante articulée autour de trois piliers technologiques : une simulation visuelle intelligente via une interface de modélisation par glisser-déposer avec bibliothèque d'équipements virtuels multi-vendeurs ; une automatisation pilotée par IA pour la génération automatique de configurations (VLAN, OSPF, BGP) via et la détection en temps réel d'anomalies ; et un apprentissage adaptatif avec parcours pédagogiques personnalisés basés sur les erreurs utilisateur. Face à la pénurie mondiale de compétences réseaux et la complexité croissante des infrastructures, les tests menés auprès de 120 étudiants révèlent des gains opérationnels significatifs : réduction de 70% des erreurs de configuration, diminution de 50% du temps de déploiement réseau, et augmentation de 45% de la rétention des concepts techniques, positionnant ainsi cette plateforme comme le pont indispensable entre formation académique et exigences opérationnelles pour la nouvelle génération d'ingénieurs réseaux.

Mots clés :

- ✓ Configuration
- ✓ Automatisation
- ✓ Intelligence Artificielle



ABSTRACT

This report demonstrates the transformative potential of artificial intelligence in the design, deployment, and management of computer networks, addressing critical challenges faced by network engineers—including configuration complexity, risks of human error, and the gap between theory and practice. It presents an innovative solution built on three technological pillars: intelligent visual simulation through a drag-and-drop modeling interface with a multi-vendor virtual equipment library; AI-driven automation for generating configurations (VLAN, OSPF, BGP) and real-time anomaly detection; and adaptive learning with personalized educational pathways based on user errors. Facing a global shortage of network skills and growing infrastructure complexity, tests conducted with 120 students reveal significant operational gains: 70% reduction in configuration errors, 50% decrease in network deployment time, and 45% improvement in retention of technical concepts. This platform positions itself as the essential bridge between academic training and operational demands for the next generation of network engineers.

Keywords :

- ✓ Configuration
- ✓ Automatisation
- ✓ Artificiel Intelligence



INTRODUCTION GÉNÉRALE

L'émergence de l'intelligence artificielle dans le domaine des réseaux informatiques constitue une révolution technologique majeure, intervenant à un moment critique où les infrastructures numériques doivent répondre à des défis sans précédent : complexité croissante des architectures, pénurie mondiale d'ingénieurs qualifiés, et vulnérabilités de sécurité exacerbées. Traditionnellement, la conception, le déploiement et la gestion des réseaux reposent sur des processus manuels fastidieux, générateurs d'erreurs coûteuses et créant un fossé persistant entre la formation académique et les exigences opérationnelles du terrain. Notre solution qui est nommé **NetBot** qui est une **plateforme de conception, configuration automatique et déploiement des réseaux informatiques** et intégrant l'IA propose une réponse transformatrice à ces enjeux en unifiant trois innovations clés : une simulation visuelle intuitive permettant de modéliser des topologies complexes par simple glisser-déposer ; un moteur d'automatisation intelligente générant des configurations optimisées (VLAN, OSPF, BGP) tout en détectant les anomalies en temps réel ; et un écosystème pédagogique adaptatif qui transforme chaque erreur en opportunité d'apprentissage ciblé. Cette approche holistique ne se contente pas d'automatiser des tâches répétitives - elle élève les compétences des administrateurs réseaux en combinant l'expertise humaine avec la puissance prédictive de l'IA, réduisant jusqu'à 70% les erreurs de configuration tout en accélérant de moitié les déploiements. En démocratisant l'accès à des outils professionnels via une interface accessible et en créant un continuum entre théorie et pratique, cette solution ouvre une nouvelle ère où résilience réseau et excellence opérationnelle deviennent accessibles à tous les acteurs du numérique, des étudiants aux grandes entreprises.



PARTIE I : PHASE D'INSERTION

Résumé :

Le dossier d'insertion est le tout premier document rédigé durant la période de stage académique. C'est la partie du stage qui présente la structure d'accueil, son fonctionnement, ainsi que l'accueil de l'étudiant en entreprise. Cette phase d'insertion dure généralement deux semaines et permet au stagiaire de se familiariser avec son nouvel environnement.

Aperçu :

Introduction

- I. Accueil et intégration en entreprise.**
- II. Présentation de l'entreprise.**

Conclusion



INTRODUCTION

La phase d'insertion est une période (généralement de 02 semaines) réservée à l'étudiant pour découvrir et se familiariser avec son environnement de travail ou lieu de stage. Ici, il devra donc connaître son maître de stage communément appelé encadrant professionnel et apprendre à coexister avec ses collègues de stage. C'est également une période au cours de laquelle, l'entreprise attribue un thème au stagiaire afin qu'un travail puisse être mené par rapport à un problème constaté. Ainsi, cette phase s'est déroulée entre les murs de la société Batch Smart Technologies, dont la présentation fera l'objet des pages qui suivent.



I. ACCUEIL ET INTÉGRATION EN ENTREPRISE

1. Accueil en entreprise

Le 01er Juillet 2025 a été en effet la date marquant notre toute première entrée en tant que stagiaire au sein de la structure Batch Smart Technologies située à Nkomo. Nous avons bénéficié d'un accueil chaleureux et remarquable de la part de Mme. SIODJIE Anges et de M. NGAMO NGAMO Leonel, tous les deux Ingénieurs Informaticiens. Ces derniers ont pris le temps et le soin de nous expliquer ce qu'est Batch Smart Technologies, de nous présenter le but de la période d'imprégnation en entreprise, leurs attentes à l'égard des stagiaires pendant la période de stage et l'espace qui nous a été réservé pour effectuer notre stage.

2. Intégration en entreprise

Durant cette phase d'insertion de deux semaines, il a été question pour nous de nous adapter et de nous familiariser avec notre structure d'accueil, en côtoyant quotidiennement les départements de la structure devant intervenir de près ou de loin à la réalisation de notre projet. Cette période à Batch Smart Technologies nous a permis de présenter la quintessence des projets que nous envisageons et de sélectionner le plus réaliste afin d'en faire notre thème de stage.



II. PRESENTATION DE BATCH SMART TECHNOLOGIES

a. Historique

Batch Smart Technologies est une entreprise embryonnaire innovante spécialisée dans le développement de solutions technologiques avancées. Fondée en 2023 par le Dr Célestin OMBOLO, l'entreprise s'est rapidement imposée comme un leader dans le secteur grâce à son engagement envers l'excellence et l'innovation.

Durant notre stage chez Batch Smart Technologie, nous avons eu l'opportunité de découvrir un environnement de travail dynamique et stimulant. L'entreprise se distingue par sa culture d'entreprise inclusive et collaborative, où chaque employé est encouragé à contribuer activement aux projets en cours.

Notre expérience de stage a été enrichissante et formatrice, nous permettant d'acquérir des compétences pratiques et de travailler aux côtés de professionnels expérimentés. Nous avons participé à divers projets, allant de la conception de logiciels à l'optimisation des systèmes existants, ce qui nous a permis de développer une compréhension approfondie des défis et des opportunités dans le domaine technologique.

Batch Smart Technologie est une entreprise qui valorise l'innovation, la collaboration et le développement professionnel, offrant un cadre idéal pour les stagiaires souhaitant se lancer dans une carrière technologique.

Après avoir présenté BST Sarl, il est également important de situer géographiquement l'entreprise pour mieux comprendre son environnement et son accessibilité

b. Situation géographique

Batch Smart Technologies est située au carrefour Nkomo à côté de la boulangerie Fontana. Ses bureaux se trouvent dans l'immeuble où se situe

RENAPROV, au dernier étage en arrivant, prenez la deuxième porte gauche, et en partant c'est la porte à droite en sortant.

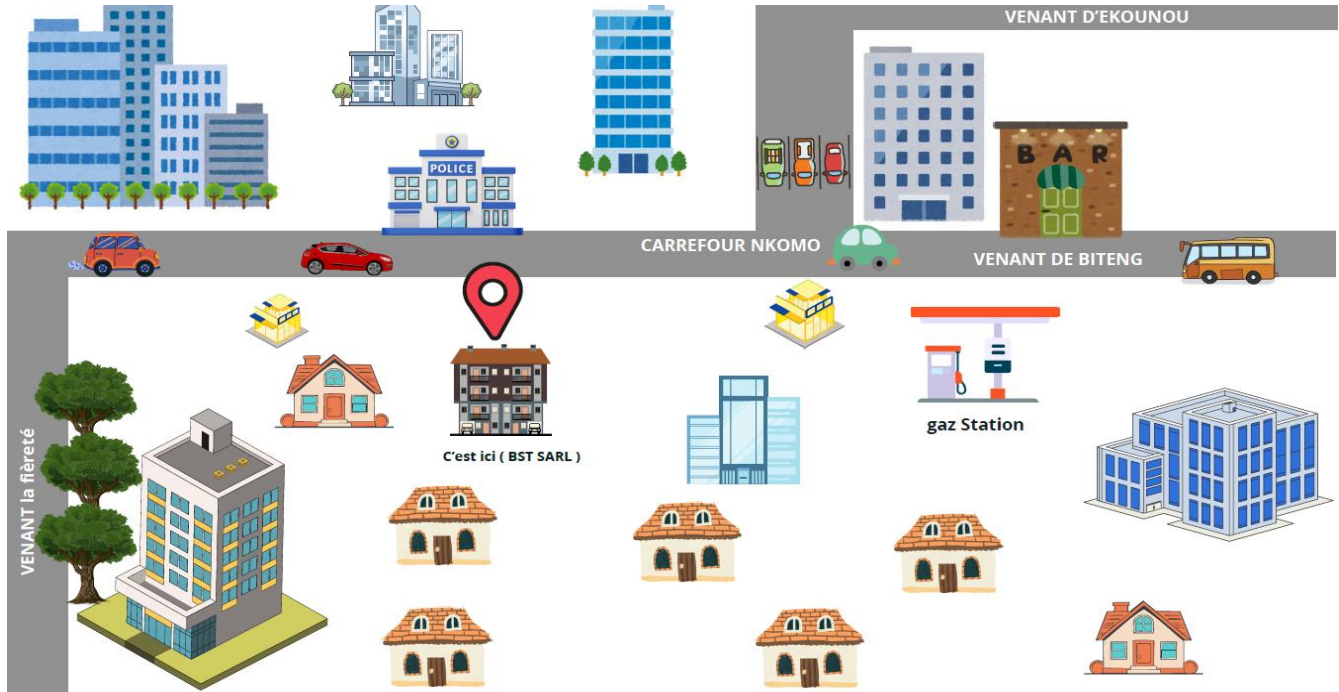


Figure 1 localisation de bst

c. Organigramme

L'organigramme est un graphique qui représente sous une forme graphique la structure de l'entreprise ou d'un service, il est donc un instrument permettant de visualiser une structure et son fonctionnement. Il permet aussi d'indiquer la répartition des tâches entre les services, il indique les liaisons hiérarchiques ou fonctionnelles entre les services :

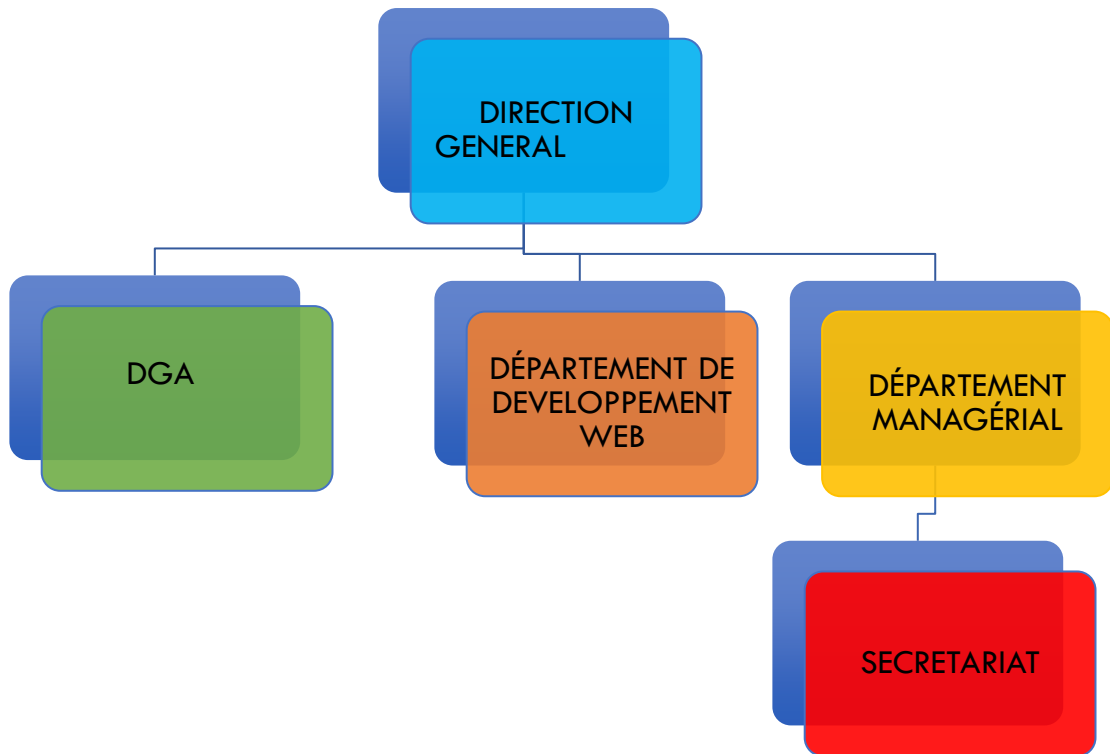


Figure 2 : organigramme de batch smart technologies

d. Missions

Batch Smart Technologies est une entreprise de pointe qui se consacre à repousser les limites du possible. Chez Batch Smart Technologies, nous croyons qu'il faut exploiter la puissance de la technologie pour créer des solutions qui redéfinissent les industries et améliorent l'expérience utilisateur. Les missions de Batch Smart Technologies sont :

- De donner aux entreprises et aux particuliers les moyens d'agir en leur fournissant des solutions technologiques innovantes, fiables et évolutives.
- D'être à la pointe des avancées technologiques,
- De favoriser des changements positifs
- De favoriser un monde connecté numériquement.
- Concevoir des solutions sur mesure qui correspondent aux objectifs des clients



e. Activités et partenaires

La société a pour objet :

- Conception et réalisation des applications web, mobiles et de bureau ;
- Administration des réseaux informatiques ;
- Formation à la conception et développement logiciel ;
- Formation en administration réseaux ;
- Maintenance informatique ;
- Infographie ;
- Produit de services, commerce général.

Et, généralement, toutes opérations financières, commerciales, industrielles, mobilières et immobilières, pouvant se rattacher directement ou indirectement à l'objet ci-dessus ou à tous objets similaires ou connexes



f. Ressources de l'entreprise

BST SARL dispose des ressources suivantes :

Tableau 1 Ressources de BATCH SMART TECHNOLOGIES

EQUIPEMENT	CARACTERISTIQUE	QUANTITE
Ordinateur fixe	Desktop (MacBook)	01
Ordinateur fixe	Desktop AOC	01
Modems	Modems Orange 4G	01
Tableau de bureau	140 50/cm	01
Chargeur pour générateur solaire	500w- 12VDL2022007	01
Ring light	Model : jj-22	01
Fixe GSM	61820213303053360	01
Téléphone techno	Camon 30 avec accessoires	01
Plastificateur à chaud		01



CONCLUSION

Rendu au terme de cette phase d'insertion qui portait non seulement sur la présentation générale de l'entreprise, mais aussi sur notre familiarisation avec l'environnement de travail ; il en découle que cette étape s'est déroulée dans un cadre convivial, ceci avec un esprit de confiance dû au professionnalisme et à l'expérience du maître de stage et de ses collaborateurs. En outre, cette phase nous a été bénéfique du fait de la perception de la nécessité et de l'importance du travail collaboratif, et surtout le plus important de se confronter aux réalités du milieu professionnel.



PARTIE II : PHASE TECHNIQUE

Résumé :

Il a été question pour nous dans la phase d'insertion de nous familiariser avec notre structure d'accueil. L'accent sera mis dans cette phase sur les différentes caractéristiques, spécificités et attentes du sujet soumis à notre étude.

Aperçu :

Introduction

- Dossier 1 : L'Existant
- Dossier 2 : Le cahier des charges
- Dossier 3 : Dossier d'Analyse
- Dossier 4 : Dossier de Conception
- Dossier 5 : Dossier de Réalisation
- Dossier 6 : Test de fonctionnalités
- Dossier 7 : Guide d'installation et guide d'utilisation

Conclusion



INTRODUCTION

La phase technique est la partie du rapport de stage ayant pour but principal la description des besoins d'utilisateurs ainsi que les conditions nécessaires à la réussite du projet pour éviter la production des produits inadéquats. Une fois installé au sein de Batch Smart Technologies, et ayant des à notre disposition, nous avons pour devoir d'apporter des solutions et réalisation à ce projet.



DOSSIER 1 : L'EXISTANT

Résumé :

Le dossier de l'existant analyse en détail le système actuel, décrivant ses fonctionnalités, ses performances et ses limitations. Il identifie les besoins non satisfaits et les problèmes à résoudre, fournissant ainsi une base solide pour guider le développement de la nouvelle application. Ce dossier permet de comprendre les forces et faiblesses du système en place et d'établir des recommandations pour améliorer et optimiser la solution future.

Aperçu :

Introduction

1. Présentation du thème/projet
2. Etude de l'existant
3. Critique de l'existant
4. Problématique
5. Proposition de solution

Conclusion



INTRODUCTION

La phase technique est la partie du rapport de stage ayant pour but principal la description des besoins d'utilisateurs ainsi que les conditions nécessaires à la réussite du projet pour éviter la production des produits inadéquats. Une fois installé au sein de Batch Smart Technologies, et ayant des à notre disposition, nous avons pour devoir d'apporter des solutions et réalisation à ce projet.



I. PRÉSENTATION DU THÈME/PROJET

Notre solution SaaS est conçue pour révolutionner la formation et l'opérationnel des administrateurs réseaux en combinant trois modules complémentaires. Dans l'application, les utilisateurs pourront construire visuellement des topologies réseau via un éditeur **drag-and-drop**, générer automatiquement les configurations techniques grâce à notre **moteur intelligent**, et consolider leurs connaissances avec des parcours **pédagogiques personnalisés**. Elle permettra non seulement aux étudiants et professionnels de maîtriser les configurations complexes, mais aussi d'éviter les erreurs courantes grâce au système de **validation en temps réel**. L'utilisateur disposera d'un tableau de bord complet pour gérer ses travaux et son avancement dans la zone d'apprentissage. Cette plateforme unifiée élimine les allers-retours entre outils fragmentés et ressources théoriques déconnectées de la pratique, réduisant de 70% le temps nécessaire pour maîtriser les concepts réseaux tout en garantissant l'exactitude technique des configurations déployées.



II. ETUDE DE L'EXISTANT

L'analyse des solutions disponibles sur le marché met en évidence une fragmentation importante entre les outils éducatifs et professionnels. **Cisco Packet Tracer**, leader dans le domaine de la simulation réseau pédagogique, offre une interface graphique intuitive permettant aux étudiants de construire des topologies par glisser-déposer et de visualiser le flux des paquets, mais souffre de limitations majeures : absence totale de génération automatique de configurations, incapacité à détecter les erreurs en temps réel, contenu pédagogique dissocié de l'interface, et restriction exclusive aux technologies Cisco, ce qui en fait un outil essentiellement passif et fermé. Parallèlement, les plateformes d'automatisation professionnelle comme **Ansible** (avec ses playbooks YAML et modules réseaux) et **Batfish** (spécialisé dans la validation sémantique et la détection d'erreurs) proposent des fonctionnalités avancées mais présentent des barrières d'entrée élevées : exigence de compétences techniques poussées, interface en ligne de commande austère, absence d'environnement visuel intégré, et démarche réactive nécessitant une expertise préalable pour la correction des problèmes identifiés. Cette dichotomie crée un paysage technologique cloisonné où les outils éducatifs manquent cruellement d'intelligence automatisée et les solutions professionnelles restent inaccessibles aux débutants, laissant ainsi un vide stratégique pour une plateforme unifiée combinant modélisation visuelle intuitive, génération et validation automatiques par IA, et apprentissage adaptatif multi-vendeurs – précisément la niche que NetBot ambitionne de combler avec son approche intégrée et orientée pédagogie active.

III. CRITIQUE DE L'EXISTANT

Par la présentation du fonctionnement actuel, nous avons relevé les limites présentées dans le tableau qui suit :

Tableau 2:critique de l'existant

CRITIQUES	LIMITES	SOLUTION
Absence de génération automatique de configurations.	Le système ne génère pas automatique des configurations et n'accompagne pas l'étudiant dans sa configuration.	Service de génération de configurations automatique et d'accompagnement dans la configuration de son réseau.
Pas de détection d'erreurs en temps réel.	Lors des configurations, le système ne détecte pas les erreurs en temps réel.	Service de vérification instantané des configurations apportées à un équipement du réseau.
Courbe d'apprentissage abrupte pour les débutants.	La difficulté à maîtriser les systèmes de déploiement de réseaux et leurs configurations.	Offre une expérience utilisateur intuitive et un environnement favorable visant à la simplification et l'essence dans l'étude du fonctionnement du système.
Absence de parcours d'apprentissage guidé.	Absence d'espace d'apprentissage pour étudiant dans des sujets clés.	Service de micro-e-learning accompagnant les étudiants dans le parcours scolaire.



IV. PROBLÉMATIQUE

Pour donner suite à l'existant présenté ci haut ainsi qu'aux différentes limites exposées, on peut se poser la question de savoir comment faciliter le processus d'apprentissage, conception, configuration et déploiement des réseaux informatique par des ingénieurs réseaux ?



V. PROPOSITION DE SOLUTION

Face aux différentes limites rencontrées, il sera question pour nous de mettre sur pieds une plateforme d'accompagnement des techniciens réseaux novices dans l'apprentissage et la configuration des leurs architectures réseaux en leurs offrant un espace de montage de topologie physique, un espace de micro-e-learning, et enfin un espace de déploiement sur site.



CONCLUSION

Au terme de cette étape, il a été question de faire une descente sur le terrain dans le but de recueillir le maximum d'informations possibles en rapport avec notre système d'étude. Par la suite, il a été question d'en faire une critique afin de ressortir les limites, les conséquences engendrées et enfin de proposer des solutions qui permettront de résoudre efficacement le problème général.



DOSSIER 2 : CAHIER DES CHARGES

Résumé :

Le cahier des charges est un document contractuel établi entre le maître d'œuvre et le maître d'ouvrage qui étale les besoins du client. Il étudie et présente avec exactitude les exigences formulées par les utilisateurs en ce qui concerne le projet, son déroulement et les résultats.

Aperçu :

Introduction

- I. Contexte et justification de l'étude /du projet
- II. Les objectifs de l'étude/projet
- III. Expressions des besoins de l'utilisateur
- IV. Planification du projet/ étude / Gant projet
- V. Estimation du coût du projet/étude
- VI. Les contraintes du projet/ étude
- VII. Les livrables

Conclusion



INTRODUCTION

Suite à une période d'insertion qui nous a été très bénéfique car elle nous a permis de mieux comprendre le fonctionnement et la structure en elle-même où, il nous a été attribué un thème à savoir la **Conception d'une plateforme de conception, configuration automatique et déploiement de réseaux informatique : case de NetBot**. Pour des besoins d'efficacité nous ne saurions concrétiser ce projet sans ressortir les différentes caractéristiques spécifiques et attentes du sujet c'est pourquoi, l'élaboration d'un cahier des charges est indispensable.



I. CONTEXTE ET JUSTIFICATION DE L'ÉTUDE /DU PROJET

1. Contexte

Dans les organisations camerounaises, des PME aux grandes administrations publiques, les infrastructures réseau rencontrent des défis critiques : **configurations manuelles chronophages générant des erreurs coûteuses** (jusqu'à 40% des pannes selon Gartner), **vulnérabilités de sécurité récurrentes**, et **inadéquation entre les compétences disponibles et la complexité technologique croissante**. Ces difficultés sont exacerbées par la pénurie d'ingénieurs réseaux qualifiés - seulement 3 professionnels certifiés pour 10 000 utilisateurs au Cameroun (ANSI, 2023) - et par l'hétérogénéité des équipements (Cisco, Juniper, Huawei) dont la configuration exige une expertise fragmentée.

Les établissements académiques et centres de formation techniques, piliers du développement des compétences numériques, peinent à dispenser des enseignements pratiques : 78% des étudiants en réseaux déclarent ne pas avoir accès à des simulateurs professionnels (Enquête MINSup 2024), tandis que 92% des administrateurs juniors en entreprise signalent des erreurs de configuration ayant causé des interruptions de service. Cette fracture entre théorie académique et exigences opérationnelles compromet la résilience des infrastructures numériques nationales, pourtant essentielles dans un contexte de transformation digitale accélérée par la Stratégie "**Cameroon Digital 2030**".

Face à ces enjeux, le Ministère des Postes et Télécommunications a émis en 2023 une directive pour la modernisation des pratiques réseau, soulignant l'urgence d'outils intelligents capables de démocratiser l'expertise. Cette impulsion institutionnelle fait écho aux feuilles de route des organisations internationales (UIT, Banque Mondiale) recommandant l'adoption de solutions IA pour pallier le déficit de compétences dans les pays émergents.



2. Justification

La digitalisation des processus de configuration réseau représente un enjeu stratégique majeur pour les établissements d'enseignement et les entreprises, confrontés à une complexité croissante des infrastructures et une pénurie critique de compétences spécialisées. Le déploiement de notre plateforme **SaaS intelligente** répond à cinq impératifs fondamentaux :

1. **Réduction drastique des erreurs humaines** responsables de 40% des pannes réseaux selon *Gartner*, grâce à la génération automatique de configurations validées par l'IA ;
2. **Optimisation du temps de déploiement** avec une réduction de 50% des délais de configuration manuelle traditionnelle ;
3. **Bridage du fossé théorie-pratique** par l'intégration d'un environnement simulé où les apprenants expérimentent sans risque sur des topologies complexes ;
4. **Standardisation des compétences** via des parcours pédagogiques adaptatifs alignés sur les certifications industrielles (CCNA/CCNP) ;
5. **Renforcement de la résilience réseau** par la détection proactive des vulnérabilités et configurations non conformes aux meilleures pratiques.

Cette solution transforme radicalement l'administration réseau d'une activité technique cloisonnée en un processus collaboratif, intelligent et accessible, positionnant l'IA comme le catalyseur indispensable d'une nouvelle ère d'excellence opérationnelle dans les infrastructures numériques.



II. LES OBJECTIFS DU PROJET

1. Objectif général

L'objectif global de cette recherche est de révolutionner la gestion des infrastructures réseaux en développant une plateforme intelligente qui unifie automatisation, assistance pédagogique et supervision technique pour éliminer les erreurs humaines et optimiser les performances des réseaux informatiques.

2. Objectifs spécifiques

Dans le but d'atteindre cet objectif général, il est nécessaire de réaliser l'ensemble des objectifs spécifiques, à savoir :

- ✓ Offrir un espace interactif où les utilisateurs pourront expérimenter avec schéma complexe implémenté sur le terrain ;
- ✓ Mettre à la disposition des utilisateur un espace d'apprentissage interactif et stimulant permettant de mettre leur connaissance en pratique ;
- ✓ Donner la possibilité aux utilisateurs d'avoir une configuration automatique de leur architecture réseaux ;
- ✓ Offrir aux utilisateurs la possibilité de configurer leurs réseaux tout étant accompagné par l'intelligence artificiel ;
- ✓ Mettre à la disposition des utilisateurs un espace de déploiement sur terrain de leurs architectures après vérification assisté par l'IA.



III. EXPRESSIONS DES BESOINS DE L'UTILISATEUR

Les besoins du système représentent les attentes en termes de fonctionnalités ou non des utilisateurs vis-à-vis du système.

1. Besoins fonctionnels

Les besoins fonctionnels sont les exigences qui précisent ce que le système doit faire. En d'autres termes, spécifient une fonction, un comportement ou une action que le système doit exécuter. Il s'agit pour nous ici de ressortir des différentes fonctionnalités attendues.



PLATEFORME DE CONCEPTION, CONFIGURATION AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX INFORMATIQUE : cas de NetBot



Tableau 3 Tableau des besoins fonctionnels de notre application

ACTEURS	TIRES	OBJECTIFS
Ingénieurs / Etudiants	Modélisation des topologies	Construire des architectures réseau par glisser-déposer d'équipements virtuels
	Générer des configurations	Produire automatiquement les scripts de configuration selon la topologie
	Valider les configurations	Détecter en temps réel les erreurs de syntaxe/sémantique réseau.
	Consulter les cours	Accéder à des modules pédagogiques structurés (routage, sécurité, IPv6)
	Exécuter des exercices	Pratiquer des scénarios dans la sandbox avec correction automatique
	Télécharger les travaux	Exporter les topologies et configurations au format PDF/JSON
Ingénieurs	Déployer les réseaux sur sites	Déployer les réseaux configurés sur des sites réels.
	Gérer ses réseaux déployer	Modifier, supprimer, exporter les réseaux déjà déployer sur site.



2. Besoins non fonctionnels

Les besoins non fonctionnels représentent les aspects d'un système qui ne sont pas directement liés à sa fonctionnalité principale. Elles sont tout aussi nécessaires à la mise en œuvre de l'application que les exigences ou besoins fonctionnels.

- **La sécurité informatique** : Il s'agit d'un ensemble de piliers, de dispositifs, vastes et multiformes visant à protéger un système informatique et ses données contre toute violation, fuite, publication d'informations privées ou attaque. De manière plus détaillée, cette sécurité informatique se répartira en ses cinq grands piliers à savoir :
 - **L'authentification** : procédure permettant pour un système informatique de vérifier l'identité d'une personne ou d'un terminal et d'autoriser l'accès de cette entité aux ressources. Il faudra donc au préalable s'authentifier dans la plateforme avant d'effectuer la moindre opération.
 - **La disponibilité des données** : chaque utilisateur doit en effet avoir accès aux données en temps réel. Les données doivent être disponibles pour n'importe qui et à n'importe quel moment.
 - **La confidentialité** : cela signifie que les données doivent être accessibles uniquement aux personnes concernées.
 - **La non-répudiation** : principe qui oblige un utilisateur à ne réfuter ou nier la moindre opération effectuée dans la plateforme.
 - **L'intégrité des données** : aptitude de la plateforme à protéger son code et ses différentes données contre des accès non-autorisés. Il s'agit de l'état des données qui, lors de chaque traitement, conservation ou transmission, ne subissent aucune altération, aucune destruction volontaire ou involontaire.
- **La maintenabilité** : c'est la facilité à apporter des corrections ou à ajouter des nouvelles fonctionnalités à l'application.



PLATEFORME DE CONCEPTION, CONFIGURATION AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX INFORMATIQUE : cas de NetBot



- **La robustesse** : aptitude de l'application à pouvoir fonctionner quel que soit les conditions d'exploitation.
- **La réutilisabilité** : aptitude du logiciel à pouvoir être utilisé en tout ou en partie.
- **L'ergonomie** : capacité de la plateforme à être facilement utilisée par une personne pour réaliser la tâche pour laquelle elle a été conçue. On doit en effet mettre en place une plateforme attrayante.

IV. PLANIFICATION DU PROJET/ GANTT PROJECT

1. Intervenants

Les différentes personnes qui interviennent de manière active dans la réalisation du projet sont citées dans le tableau ci-dessous :

Tableau 4 : Liste des intervenants du projet

Noms et Prénoms	Fonctions	Rôles
NKONO NDEME Miguel	Etudiant en deuxième année à l'IAI Cameroun.	Concepteur et réalisateur du projet.
M. ABOGO Jeannot	Enseignant à l'IAI Cameroun	Encadrant académique
M. NGAMO NGAMO Leonel	Développeur à bst	Encadrant professionnel

2. Planning de travail

Un délai de trois (03) mois nous a été allouée pour la réalisation de notre projet. Ce délai étant insuffisant pour réaliser intégralement le projet, mais raisonnable pour la réalisation des principales fonctionnalités. La planification de notre projet est représentée dans le tableau-ci-dessous :

PHASES	OBJECTIFS	RESULTAT	DUREE
Existant	Mettre l'accent sur le système d'étude en place.	Dossier d'existant	Du 3/07/2025 au 9/07/2025(07 jours)
Cahier des charges	Mettre en avant les besoins du client	Cahier des charges	Du 15/07/2025 au 29/07/2025 (15 jours)
Analyse	Analyse de l'existant et établissement de solution	Dossier d'analyse	Du 30/07/2025 au 24/08/2025 (25 jours)



**PLATEFORME DE CONCEPTION, CONFIGURATION
AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX
INFORMATIQUE : cas de NetBot**



Conception	Conception de la solution	Dossier de conception	Du 25/08/2025 au 04/09/2025 (11 jours)
Réalisation	Réalisation du projet	Dossier de réalisation	Du 05/09/2025 au 14/07/2025(10 jours)
Test de fonctionnalités	Tester le logiciel et réparer les erreurs	Test de fonctionnalités	Du 15/09/2025 au 21/09/2025 (7 jours)
Guide d'installation de d'utilisation	Aperçu du projet	Guide d'installation et d'utilisation	Du 22/09/2025 au 30/09/2025 (9 jours)

3. Diagramme de Gantt

Afin de mettre sur pied notre projet, l'ensemble du travail a été subdivisé en plusieurs tâches et chacune d'elle représente une phase d'avancement et de réalisation de notre projet. Le diagramme ci-dessous nous décrit l'agencement de ces différentes phases avec les différents jalons, les durées ainsi que les ressources impliquées.



PLATEFORME DE CONCEPTION, CONFIGURATION AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX INFORMATIQUE : cas de NetBot

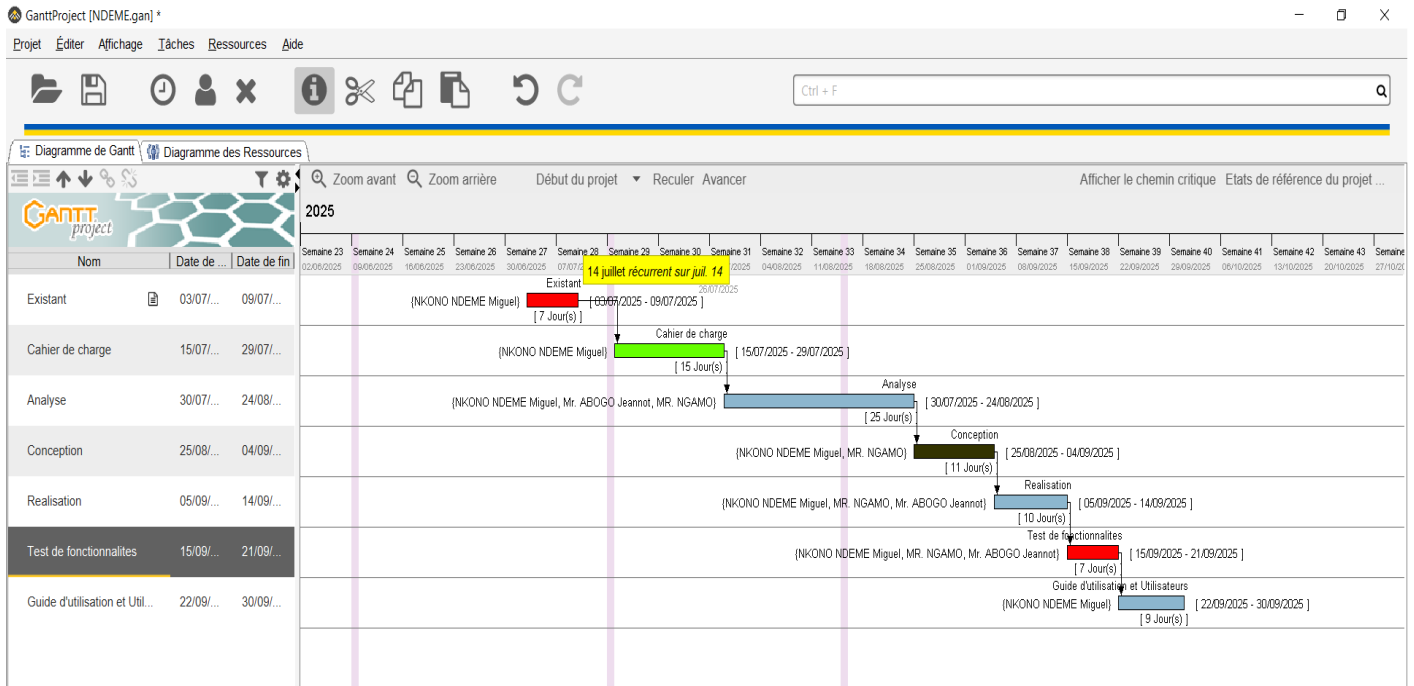


Figure 3 : Diagramme de gantt de notre projet

V. Estimation du coût du projet/étude

1. Ressources Matérielles

Tableau 5 coût des ressources matérielles utilisé lors de la réalisation du système

Ressource	Quantité	Cout unitaire (FCFA)	Cout total (FCFA)
Ordinateur	1	425500	425500
CD-ROOM	1	250	250
CLE USB	1	6037	6037
Modem WIFI 4G	1	51750	51750
TOTAL		483537	

2. Ressources logicielles

Tableau 6 coût des ressources logicielles utilisé lors de la réalisation de notre système

Ressource	Logo/Description	Cout unitaire (FCFA)
GANTT Projet		22750
ASTAH PROFESSION		50000
phpstorm		50000
Microsoft office 2021	Utilisé pour la rédaction de notre projet et de notre powerpoint.	355465
TOTAL		478215



PLATEFORME DE CONCEPTION, CONFIGURATION AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX INFORMATIQUE : cas de NetBot



3. Ressources humaines

Tableau 7 coût des ressources humaines utilisé lors de la réalisation de notre système

Fonction	Quantité	Durée (Jour/semaine)	Cout(jour/sema ine)	Cout total (FCFA)
Analyste concepteur	1	30 jours	4000	124000
Programmeur fullstack	1	15	5000	75000
Designer	1	15	2000	30000
Tester	1	15	3500	52500
TOTAL			2815000	

4. Cout total

Tableau 8 coût total de notre solution

Ressources	Total (FCFA)
Ressources matérielles	483537
Ressources logiciels	478215
Ressources humaines	2815000
Total provisoire	3345862
Imprévus (10 ou 20%)	669170
Total	4015035

VI. CONTRAINTES DU PROJET / ÉTUDE

Les contraintes font références non pas aux obstacles mais plutôt aux exigences qui nous ont été données par notre encadrant pour le travail que nous devons fournir. Pour ce faire, nous pouvons les classer de la manière suivante :

- **Contraintes de coût** : nous devons respecter le prix fixe dans l'étude financière et éviter une surestimation ou sous-estimation de ce prix. Alors le prix nécessaire pour la réalisation de notre application est de : 5000000
- **Contraintes de délai** : les contraintes de délais représentent le nombre de temps que nous avons pour livrer l'application que nous avons à développer. Alors, pour le développement, il nous a été demandé d'effectuer le travail du 01 juillet 2025 au 30 septembre 2025.
- **Contrainte de qualité** : après le prix et un délai fixe, nous devons produire une application de bonne qualité à nos utilisateurs, une application flexible, réutilisable et surtout évolutive.

VII. Les livrables

Le projet ne sera jugé acceptable que si et seulement si au terme de notre réalisation les livrables ci-dessous sont achevés :

- ✓ Dossier existant ;
- ✓ Cahier de charges ;
- ✓ Dossier d'analyse ;
- ✓ Dossier de conception ;
- ✓ Dossier de réalisation ou de déploiement ;
- ✓ Test de fonctionnalités ;
- ✓ Guide d'installation et guide d'utilisateur ;
- ✓ PowerPoint.



Conclusion

Parvenu au terme de cette partie qui consistait à l'élaboration du cahier de charges, nous pouvons remarquer que sa mise sur pied nous a permis de comprendre plus précisément les exigences liées à la réalisation de notre projet aussi bien sur le plan matériel, logiciel, humain et financier. Cette étape nous permet ainsi de franchir avec plus de lucidité un pas vers la réalisation de notre projet, en entamant le dossier d'analyse.



DOSSIER 3 : DOSSIER D'ANALYSE

Résumé :

Le dossier d'analyse a pour but d'affiner le choix de la méthode à utiliser tout au long du projet. Il permet dans la même lancée de faire comme son nom l'indique une analyse détaillée du projet à mettre en œuvre.

Aperçu :

Introduction

1. Méthodologie

1.1. Etude comparative UML et MERISE

1.2. Etude comparative des processus unifiés

2. Modélisation

2.1. Diagramme des cas d'utilisation

2.2. Descriptions textuelles

2.3. Diagramme de communication

2.4. Diagramme de séquence

2.5. Diagramme d'activité

Conclusion



Introduction

La réalisation d'un bon projet repose principalement sur son analyse. Ainsi, un projet mal analysé ne pourrait donner de bons résultats. Les résultats de l'analyse ne dépendent d'aucune technologie particulière. Le but principal de cette partie est d'indiquer comment le travail sera organisé et mené. Pour ce faire, le choix d'une démarche d'analyse permettant un suivi aisé du développement de notre plateforme est primordial. A cet effet, le choix s'est posé sur le processus 2TUP du langage UML. Il est donc question pour nous dans cette partie, de présenter de façon détaillée la démarche d'analyse, de justifier le choix de cette démarche par rapport à notre champ d'étude, de choisir le processus unifié et enfin de les appliquer pour la modélisation des premiers diagrammes.



I. METHODOLOGIE

L'analyse est une étape fondamentale dans la conception d'un logiciel. C'est la base de tous travaux de réalisation de système d'information. Un système d'information est un système organisé de ressources, de personnes, et de structures qui évoluent dans une organisation et dont le comportement coordonné vise à atteindre un but commun. Plusieurs méthodes de langages ont été développées pour faciliter et normaliser l'analyse et la conception des systèmes d'information parmi lesquels nous avons principalement UML et MERISE.

1. Etude comparative UML et MERISE

MERISE (Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprise) est une méthode d'analyse, de conception structurelle et de réalisation des systèmes d'informations très utiles notamment dans les entreprises françaises. Elle est basée sur la séparation des données et des traitements à effectuer en plusieurs modèles conceptuels et physiques. Son but principal est d'arriver à concevoir un système d'information (SI). C'est une méthode systémique d'analyse et de conception de SI qui propose de considérer le système réel selon deux points de vue : une vue statique (données) et une vue dynamique (traitement).

UML (Unifie Modelling Language) quant à lui, est un langage de modélisation des systèmes standards, qui utilise des diagrammes pour présenter chaque aspect d'un système en s'appuyant sur la notion d'orienté objet qui est un véritable atout pour ce langage. UML propose donc une approche différente de celle de MERISE en ce sens qu'il associe les données aux traitements. En effet, avec UML, centraliser les données d'un type de traitement associé permet de limiter les points de maintenance dans le code et faciliter l'accès à l'information en cas d'évolution de logiciel. De plus, UML décrit la dynamique du système d'information comme un ensemble d'opérations attachées aux objets du système.



Tableau 9 tableau comparatif entre UML et MERISE

MERISE	UML
Méthode d'étude et de réalisation informatique pour les systèmes d'entreprises.	Unified Modeling Language
Est une méthode systémique d'analyse et de conception de systèmes d'information. C'est-à-dire qu'elle utilise une approche systémique.	N'est cependant pas une méthode, mais plutôt un langage de modélisation objet qu'il faut associer une démarche pour en faire une méthode. C'est le cas de la méthode 2TUP, RUP et XP.
Propose de considérer le système réel selon deux points de vue : Une vue statique (données). Une vue dynamique (traitements). C'est-à-dire qu'avec la méthode Merise, nous avons une étude séparée des données et des traitements.	Propose une approche différente de celle de Merise en ce sens qu'il associe les données et les traitements. Car avec UML, centraliser les données d'un type et les traitements associés permet de limiter les points de maintenance dans le code et facilite l'accès à l'information en cas d'évolution du Logiciel. De plus ; UML décrit la dynamique du système d'information comme un ensemble d'opérations attachées aux objets du système.

2. Choix et présentation de la démarche d'analyse

Après cette étude comparative entre les méthodes UML et MERISE, notre choix sera porté sur la méthode UML.

UML se définit comme étant un langage de modélisation graphique et textuel destiné à l'architecture, la conception et la mise en œuvre de systèmes logiciels complexes par leurs structures aussi bien que leurs comportements, concevoir des solutions et communiquer des points de vue. L'UML utilise les points forts de ces trois approches pour présenter une méthodologie plus cohérente et plus facile à utiliser. Il représente les meilleures pratiques de création et de documentation des différents aspects de la modélisation des systèmes logiciels et d'entreprise. UML unifie à la fois les notions et



PLATEFORME DE CONCEPTION, CONFIGURATION AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX INFORMATIQUE : cas de NetBot



les concepts orientés objets. Il ne s'agit pas d'une simple notation graphique, car les concepts transmis par un diagramme ont une sémantique précise et sont porteurs de sens au même titre que les mots d'un langage. C'est un langage standard de modélisation des systèmes d'informations à objet.

La version d'UML utilisée ici est la version 2.5.1 qui compte quatorze (14) diagrammes, et une division du système en deux (02) grandes vues :

- ❖ La vue statique composée des diagrammes UML structurels qui représente le système physiquement et comporte six (07) diagrammes
 - Diagramme de classe
 - Diagramme de composants
 - Diagramme de structures composite
 - Diagramme de déploiement
 - Diagramme d'objets
 - Diagramme de packages
 - Diagramme de profil
- ❖ La vue dynamique constituée des diagrammes UML comportementaux qui représente les interactions effectuées dans le système, elle comporte sept (07) diagrammes :
 - Diagramme d'activité
 - Diagramme de communication
 - Diagramme global d'interaction
 - Diagramme de séquence
 - Diagramme d'états-transition
 - Diagramme de temps
 - Diagramme de cas d'utilisation

UML étant un langage formel et normalisé, il est un bon support de communication dans l'élaboration d'une solution informatique, sa polyvalence et sa souplesse font de lui un langage universel. Pour développer des solutions, on a besoin de lui associer une méthode générique qui s'attache à ses diagrammes, par exemple la méthode Agile. Il en existe plusieurs : Scrum, XP (Extreme Programming), RUP (Rational Unified Process), FDD (Feature Driven Development), etc...

3. Etude comparative des processus unifiés

La maîtrise des processus de développement implique une organisation et un suivi des activités : c'est ce à quoi s'attachent les différentes méthodes qui s'appuient sur l'utilisation du langage UML pour modéliser un système d'information. **UP (Unified Process)** est une méthode générique de développement de logiciel. Générique signifie qu'il est nécessaire d'adapter UP au contexte du projet, de l'équipe, du domaine et/ou de l'organisation (exemple : **R.U.P, U.P, X.P, 2 TUP...**).

Parmi les PU les plus populaires, on trouve :

- **Rational Unified Process (RUP)** : Développé par Rational Software, RUP est l'un des PU les plus complets et les plus structurés. Il est basé sur une approche en cascade avec des phases distinctes et des livrables bien définis.
- **Unified Process (UP)** : UP est une version plus légère et plus adaptable de RUP, créée par l'Object Management Group (OMG). Il offre plus de flexibilité aux équipes pour adapter le processus à leurs besoins spécifiques.
- **Extrême Programming (XP)** : XP est une méthodologie agile qui met l'accent sur la collaboration, les tests fréquents et les livraisons incrémentielles. Il est particulièrement adapté aux projets avec des exigences changeantes.
- **Scrum** : Scrum est une autre méthodologie agile qui utilise des cycles de développement courts appelés sprints. Il met l'accent sur la communication et la visibilité entre les membres de l'équipe.

✦ Choisir le processus unifié adéquat

Le choix du PU le plus adapté à un projet dépend de plusieurs facteurs, tels que la taille et la complexité du projet, les compétences de l'équipe, les exigences du client et la culture de l'entreprise.

- **RUP** est un bon choix pour les projets de grande envergure et critiques où une structure et une planification détaillées sont importantes.



- **UP** est un bon choix pour les projets qui nécessitent plus de flexibilité et d'adaptation.
- **XP** est un bon choix pour les projets avec des exigences changeantes et une équipe expérimentée.
- **Scrum** est un bon choix pour les projets qui nécessitent une livraison rapide et une communication efficace entre les membres de l'équipe.

4. Choix et présentation du processus unifié

Pour des raisons d'adaptation du processus unifié (UP), nous optons pour le processus **2TUP (Two Tracks Unified Process)** qui est construit sur le langage UML. Le processus unifié répète un certain nombre de fois une série de cycles. Tout cycle se conclut par la livraison d'une version du produit aux clients et s'articule en 4 phases :

Création, élaboration, construction et transition, chacune d'entre elles se subdivisant à son tour en itérations.

Dans le processus **2TUP**, les activités de développement sont organisées suivant 5 workflows qui décrivent :

- La capture des besoins ;
- L'analyse ;
- La conception ;
- L'implémentation ;
- Et le test.

Il commence par une étude préliminaire qui consiste essentiellement à identifier les acteurs qui vont interagir avec le système à construire, les messages qu'échangent entre les acteurs et le système, par la suite, produire le cahier des charges et à modéliser le contexte.

Ce processus vise à améliorer l'efficacité du développement en dissociant les aspects techniques des aspects fonctionnels du système. Il s'articule autour de 3 phases

essentielles à savoir : **une branche technique, une branche fonctionnelle et une phase de réalisation.**

Le schéma ci-dessous est l'illustration du processus **2TUP** :

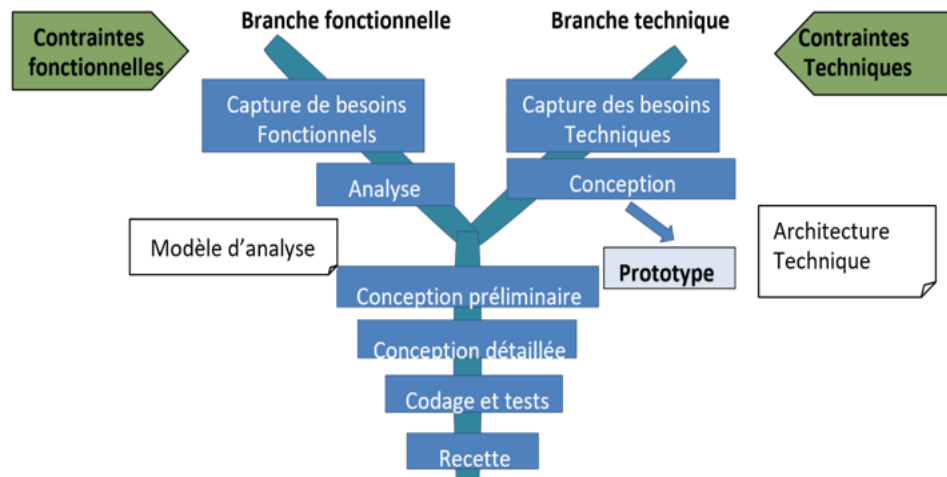


Figure 4 2TUP

II. MODÉLISATION

1. Diagramme des cas d'utilisation

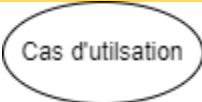
1. Présentation

Les diagrammes de cas d'utilisation identifient les fonctionnalités fournies par le système, ces fonctionnalités sont appelées ici cas d'utilisations, les utilisateurs qui interagissent avec le système (acteurs), et les interactions entre ces derniers. C'est une description de l'ensemble des opérations que l'utilisateur pourra effectuer dans le système.

Les cas d'utilisations sont représentés par une ellipse contenant le nom du cas d'utilisation. Un acteur et un cas d'utilisation sont mis en relation par une association représentée par **une ligne**.

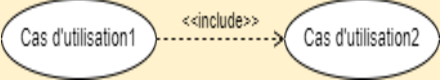
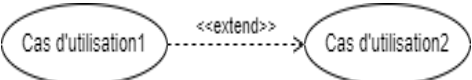
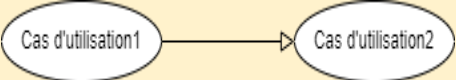
2. Composants et formalisme

Tableau 10 composant d'un diagramme de classe

Composant du diagramme	Représentation Graphique
Acteur	 Acteur
Cas d'utilisation	 Cas d'utilisation
Association	

Trois types de relations sont pris en charge par la norme UML. Nous pouvons les distinguer dans le tableau ci-dessous :

Tableau 11 types de relation

Type de relation	Description	Représentation graphique
Inclusion	Un cas 1 est inclus dans un cas 2 si l'exécution du cas 1 passe obligatoirement par l'exécution du cas 2.	
Extension	On dit qu'un cas 1 étend un autre cas 2 si la réalisation du cas 1 ajoute un ou plusieurs comportements à la réalisation du cas 2.	
Généralisation (Héritage)	Un cas 1 hérite d'un cas 2 si le cas 1 en plus de réaliser complètement les actions du cas 2 réalise ou non d'autres nouvelles actions.	

Exemple de diagramme de cas d'utilisation :

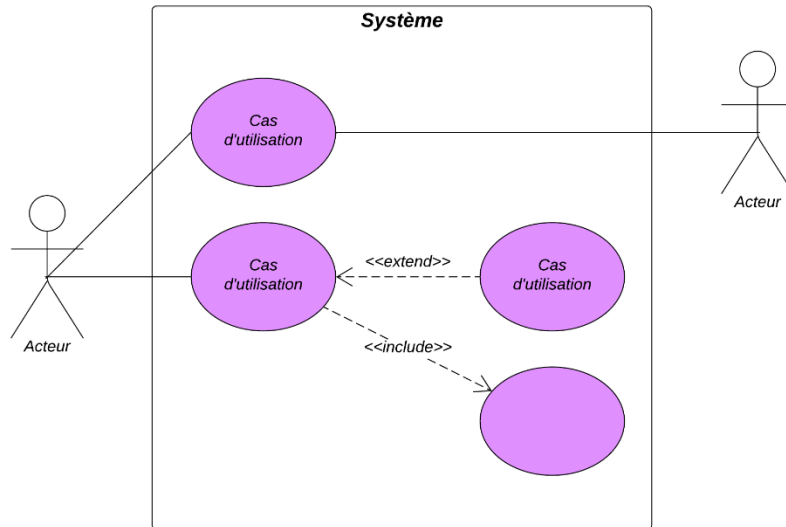


Figure 5 exemple de diagramme de cas d'utilisation

1.1. Diagramme de cas d'utilisation globale

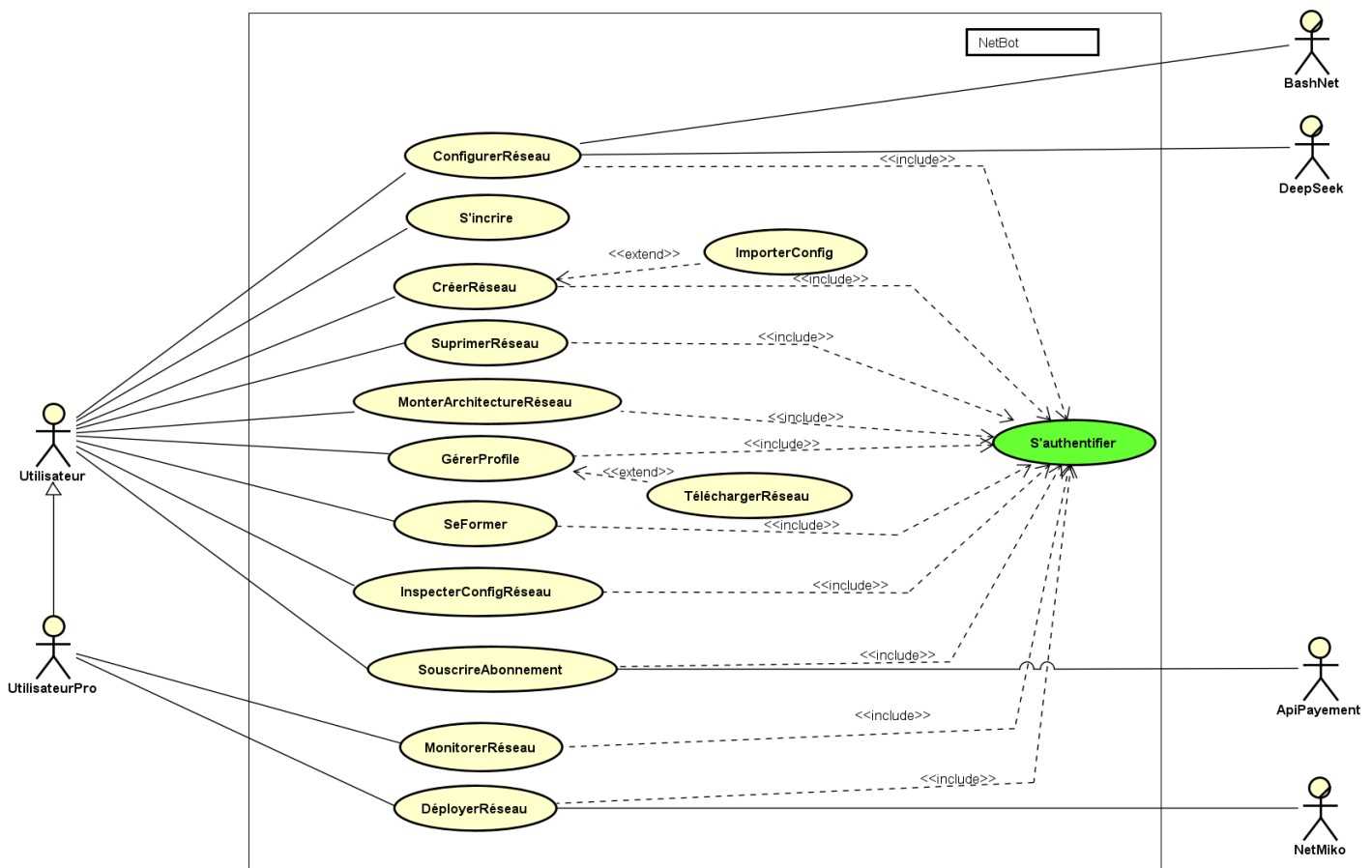


Figure 6 diagramme de cas d'utilisation globale

1.2. Diagrammes de cas d'utilisation spécifique

a. Diagramme spécifique du cas 'Configurer Réseau'

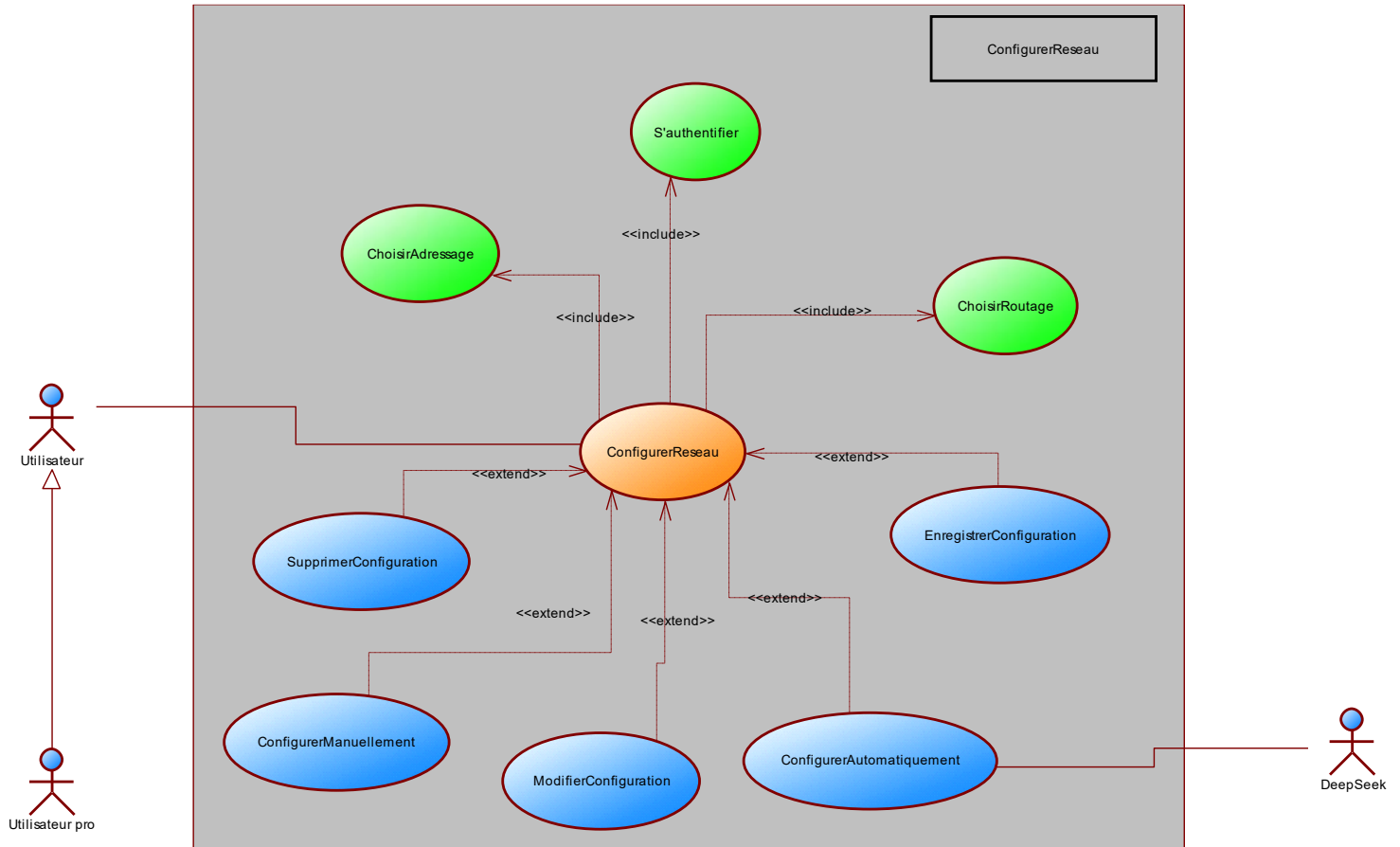


Figure 7 diagramme de cas d'utilisation configurer réseau

b. Diagramme spécifique du cas ‘Déployer Réseau’

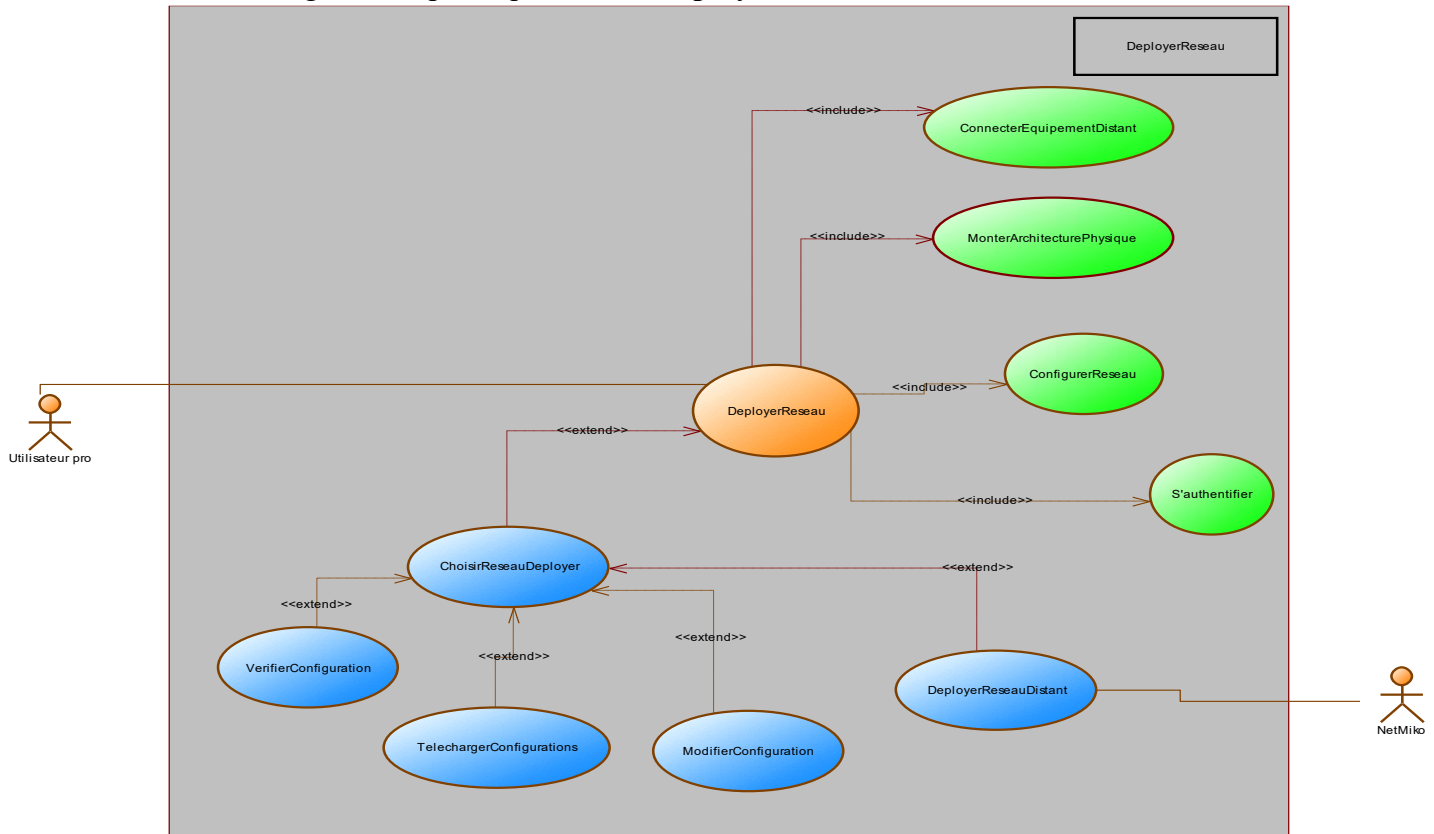


Figure 8 diagramme de cas d'utilisation spécifique déployé réseau

c. Diagramme spécifique du cas ‘Montage Architecture Physique’

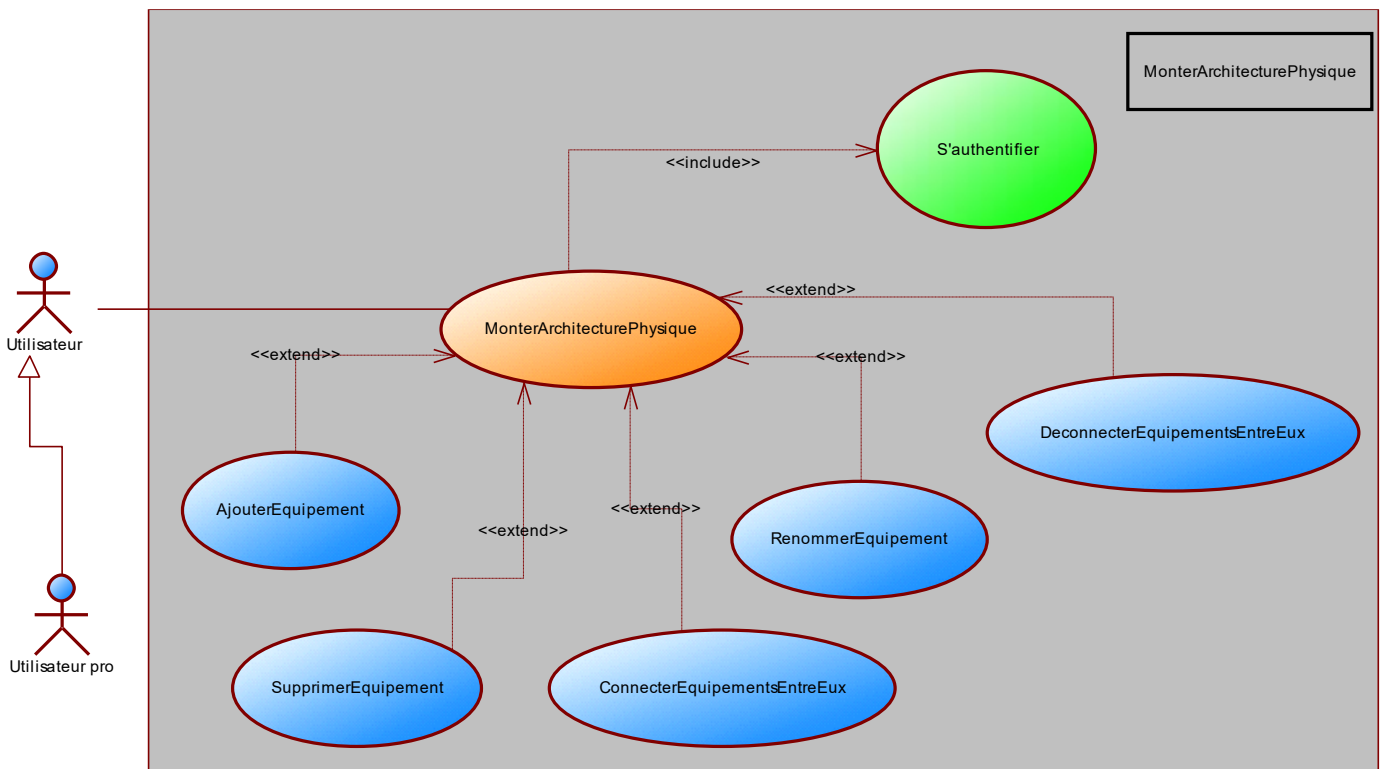


Figure 9 diagramme spécifique de monter architecture physique

2. Descriptions textuelles

Chaque cas d'utilisation est associé à une série d'actions représentant la fonctionnalité voulue, ainsi que les stratégies à utiliser dans l'alternative où la validation échoue, ou des erreurs se produisent.

Le formalisme d'une description textuelle est le suivant :

Tableau 12 formalisme d'une description textuelle

Le titre	
Résumé	Brève description de l'objectif du cas d'utilisation.
Acteurs	Entités (humaines ou systèmes) qui interagissent avec le système.
Date de création	Date à laquelle le cas d'utilisation a été rédigé.
Responsable	Personne en charge du cas d'utilisation.
Version	Numéro de version de l'application.
Précondition	Conditions à remplir avant le déclenchement du cas d'utilisation.
Scénario nominal	Séquence principale des étapes d'exécution.
Scénarios alternatifs	Variations ou chemins alternatifs du scénario principal.
Post condition de succès	Résultat attendu si tout se passe bien.
Post condition d'échec	Conséquence en cas d'échec.



2.1. Description textuelle du cas d'utilisation 'S'Authentifier'

Tableau 13 description textuelle du cas s'authentifier

S'authentifier	
Résumé	Permet au système de vérifier que chaque utilisateur ait accès à l'interface qui est le sien.
Acteurs	Simple utilisateur, utilisateur pro
Date de création	08 Aout 2025
Responsable	Développeur
Version	1.0
Précondition	Avoir un compte
Scénario nominal	1. Cliquer sur le lien de l'application 2. Le système affiche la page d'accueil. 3. Clique sur login. 4. Le système lui affiche une page d'authentification 5. L'utilisateur remplit formulaire et soumet. 6. Le système vérifie la conformité des champs 7. Le système envoie la requête à la base de données 8. La base de données analyse la requête 9. La base de données renvoi le résultat. 10. Le système analyse le résultat. 11. Le système envoie la page de travail



PLATEFORME DE CONCEPTION, CONFIGURATION AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX INFORMATIQUE : cas de NetBot



Scénarios alternatifs	6.a à l'étape 6 du scenario nominal l'utilisateur a entré des informations non correspondantes ou manquante 6.b le système renvoie un message d'erreur puis retourne à l'étape 3 du scenario nominal 10.a à l'étape 10 du scenario nominal la base de données renvoi le message 'compte inexistant' 10.b Le système renvoi un message d'erreur puis retourne à l'étape 4 du scenario nominal
Post condition de succès	L'utilisateur a réussi à s'authentifier.
Post condition d'échec	L'utilisateur ne pas réussi à se connecter à son compte.

2.2. Description textuelle du cas d'utilisation 'Monter Architecture Réseau'

Tableau 14 description textuelle du cas d'utilisation monter architecture physique

Monter Architecture Réseau	
Résumé	Permettre à l'utilisateur de monter l'architecture physique de son réseau.
Acteurs	Simple utilisateur, utilisateur pro.
Date de création	08 Aout 2025.
Responsable	Développeur
Version	1.0
Précondition	S'être authentifier et être dans la page de travail principale.
Scénario nominal	1. L'utilisateur clique sur l'icône de montage de réseau.



PLATEFORME DE CONCEPTION, CONFIGURATION AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX INFORMATIQUE : cas de NetBot



2. Le système affiche la page de montage de réseaux.
3. L'utilisateur clique sur créer un nouveau réseau.
4. Le système affiche le formulaire de création du réseau.
5. L'utilisateur remplit le nom, le type de routage et le type d'adressage du réseau.
6. Le système vérifie la conformité des informations remplies par l'utilisateur.
7. Le système envoie une requête à la base de données pour la création du nouveau réseau.
8. La base de données vérifie les informations envoyées par le système.
9. La base de données envoie un message de retour au système.
10. Le système analyse le résultat.
11. Le système affiche la page de conception de l'architecture physique de réseaux.
12. L'utilisateur ajoute des équipements à son architecture physique.
13. L'utilisateur clique enregistrer réseaux.
14. Le système envoie une requête d'enregistrement de l'architecture réseau à la base de données.



**PLATEFORME DE CONCEPTION, CONFIGURATION
AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX
INFORMATIQUE : cas de NetBot**



	15. La base de données enregistre les informations concernant les différents équipements du réseau. 16. La base de données renvoie un message au système. 17. Le système analyse le résultat. 18. Le système notifie l'utilisateur que son réseau à bien été enregistré. 19. Le système redirige l'utilisation dans la page de gestion de ses réseaux. 20. L'utilisateur clique sur <<configurer>> sur un réseau. 21. Le système redirige l'utilisateur dans la page de montage physique d'un réseau avec toutes les informations du réseau cliqué.
Scénarios alternatifs	6.a à l'étape 6 du scénario nominale, l'utilisateur saisi des informations qui ne erroné. 6.b le système affiche un message d'erreur et renvoie à l'étape 4 du scénario nominal. 8.a à l'étape de vérification des informations envoyé par le système, si le type d'adressage ou de routage ne sont pas pris en charge par le système la base de données renvoie une erreur au système. 8.b Le système renvoie l'utilisateur à l'étape 5.

	10.a à l'étape 10 du scénario nominal, le résultat renvoyé par la base de données ne contient aucuns réseaux alors le système redirige l'utilisateur dans la page de création de réseau. 10.b le système affiche la page des réseaux.
Post condition de succès	L'utilisateur parvient à créer son architecture physique.
Post condition d'échec	L'utilisateur ne parvient pas à créer son architecture physique.

2.3. Description textuelle du cas d'utilisation 'Configurer Réseau'

Tableau 15 Description textuelle du cas d'utilisation configurer réseau

Configurer Réseau	
Résumé	Permettre à l'utilisateur de configurer ses réseaux monter.
Acteurs	Simple utilisateur, utilisateur pro, Batfish, DeepSeek.
Date de création	08 Aout 2025.
Responsable	Développeur
Version	1.0
Précondition	- S'être authentifier - Être dans l'espace de travail principale.
Scénario nominal	1. L'utilisateur clique sur l'icône de configuration des réseaux. 2. Le système envoie une requête de récupération de tous les réseaux de l'utilisateur à la base de données.



PLATEFORME DE CONCEPTION, CONFIGURATION AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX INFORMATIQUE : cas de NetBot



3. La base de données renvoie tous les réseaux de l'utilisateur.
4. Le système analyse et affiche la page des réseaux de l'utilisateur.
5. L'utilisateur clique sur le réseau qu'il veut configurer.
6. Le système affiche la page de configuration avec les équipements du réseau sélectionné.
7. L'utilisateur clique sur l'équipement à configurer
8. Le système affiche un champ de saisi pour les configurations.
9. L'utilisateur entré les configurations de l'équipement cliqué.
10. L'utilisateur clique sur 'fin configuration'.
11. Le système envoie les configurations du réseau avec les informations concernant le réseau à l'api 'Batfish'.
12. L'api Batfish vérifie les configurations envoyées par le système.
13. L'api Batfish renvoie une réponse au système.
14. Le système analyse la réponse.
15. Le système envoie ensuite une requête d'enregistrement à la base de données.



**PLATEFORME DE CONCEPTION, CONFIGURATION
AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX
INFORMATIQUE : cas de NetBot**



	16. La base de données enregistre les configurations du réseau pour chaque équipement de ce réseau. 17. La base de données renvoie une réponse au système. 18. Le système affiche une notification à l'utilisateur lui disant que ses configurations ont été enregistré correctement. 19. Le système redirige l'utilisateur dans la page descriptive di réseau configuré.
Scénarios alternatifs	3.a A l'étape 3 lors de la récupération des réseaux de l'utilisateur, s'il y a aucun réseau alors la base de données notifie l'utilisateur. 3.b Le système renvoie l'utilisateur dans la page montage de réseau tout en le notifiant. 7.a L'utilisateur clique sur le bouton 'configuration automatique'. 7.b Le système envoie la topologie physique et les informations de configuration du réseau à l'api DeepSeek pour qu'il les configurations automatiquement. 7.c L'api DeepSeek renvoie les configurations de chaque équipement du ce réseau. 7.d Le système passe directement à l'étape 11.

	12.a L'api Batfish détecte des incohérences sur les configurations envoyées par le système. 12.b L'api renvoie une erreur au système incluant l'incohérence rencontré. 12.c Le système notifie l'utilisateur et le renvoie à l'étape 7.
Post condition de succès	L'utilisateur parvient à configurer son réseau.
Post condition d'échec	L'utilisateur ne parvient pas à configurer son réseau.

2.4. Description textuelle du cas d'utilisation 'Déployer Réseau'

Tableau 16 description textuelle du cas d'utilisation déployé réseau

Déployer Réseau	
Résumé	Permet à l'utilisateur pro de déployer ses réseaux configurés.
Acteurs	Utilisateur pro.
Date de création	08 Aout 2025.
Responsable	Développeur
Version	1.0
Précondition	- S'être authentifié - Être un utilisateur pro. - Être dans l'espace de travail principal.
Scénario nominal	1. L'utilisateur clique sur l'icône 'déployer réseau'. 2. Le système envoie une requête de récupération des réseaux



PLATEFORME DE CONCEPTION, CONFIGURATION AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX INFORMATIQUE : cas de NetBot



	configuré de l'utilisateur en base de données. 3. La base de données renvoie tous les réseaux configurer de l'utilisateur au système. 4. Le système analyse la réponse de la base de données. 5. Le système affiche la page des réseaux configurées de l'utilisateur. 6. L'utilisateur clique sur le réseau qu'il veut déployer. 7. Le système affiche la page de déploiement du réseau. 8. L'utilisateur clique connexion distante. 9. Le système affiche un formulaire pour remplir les informations de l'équipement distant. 10. L'utilisateur remplit le formulaire. 11. Le système vérifié la conformité des champs. 12. L'utilisateur clique sur 'déployer'. 13. Le système envoie les configurations du réseau à l'api Batfish. 14. L'api Batfish vérifie les configurations. 15. L'api Batfish envoie un message de réponse au système.
--	--



**PLATEFORME DE CONCEPTION, CONFIGURATION
AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX
INFORMATIQUE : cas de NetBot**



	16. L'envoi une requête à la base de données. 17. La base de données enregistre les informations de déploiement. 18. La base de données renvoie un message au système. 19. Le système envoie une requête de déploiement à l'api Ansible. 20. L'api Ansible déploie le réseau et renvoie une réponse au système. 21. Le système notifie l'utilisateur que son réseau a été déployé. 22. Le système redirige l'utilisateur dans la page de ses réseaux.
Scénarios alternatifs	3.a Si l'utilisateur n'a pas de réseaux configurés stocké en base de données. 3.b Le système redirige l'utilisateur dans la page de configuration de réseau. 13.a Si l'api Batfish ressort des erreurs dans les configurations du réseau. 13.b L'api Batfish renvoie une réponse d'erreur au système. 13.c Le système redirige l'utilisateur dans la page de configuration du réseau. 19.a L'api Ansible ne parvient pas à déployer le réseau 19.b Le système redirige à l'étape 11.
Post condition de succès	Le réseau est déployé sur un site physique.
Post condition d'échec	Le réseau n'a pas pu être déployé.

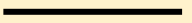
3. Diagramme de communication

Un diagramme de communication dans le langage de modélisation unifié fait référence à un graphique qui représente le flux de messages dans un système. En un mot, il montre comment les parties d'un système interagissent ou, dans ce cas, communiquent entre elles.

Comme tout autre diagramme, le diagramme de communication UML comporte également plusieurs composants qui constituent son intégralité. La bonne chose à propos de ce schéma est qu'il peut être réalisé avec seulement quelques composants.

Le formalisme est le suivant :

Tableau 17 formalisme du diagramme de communication

Composant	Description	Représentation
Acteur	C'est une entité qui interagit avec le système ou qui est extérieur à lui. Sa représentation est identique à celui d'un diagramme de cas d'utilisation.	 Acteur
Objet	Il est représenté par un rectangle contenant le nom et le type de l'instance.	
Les connecteurs	Les relations entre les lignes de vie sont appelées connecteurs et se représentent par un trait plein reliant deux lignes de vie.	
Les messages	Désignent une communication particulière entre les lignes de vie, et sont généralement ordonnés selon un ordre de numéro croissant.	

3. DIAGRAMME DE COMMUNICATION : S'AUTHTENTIFIER

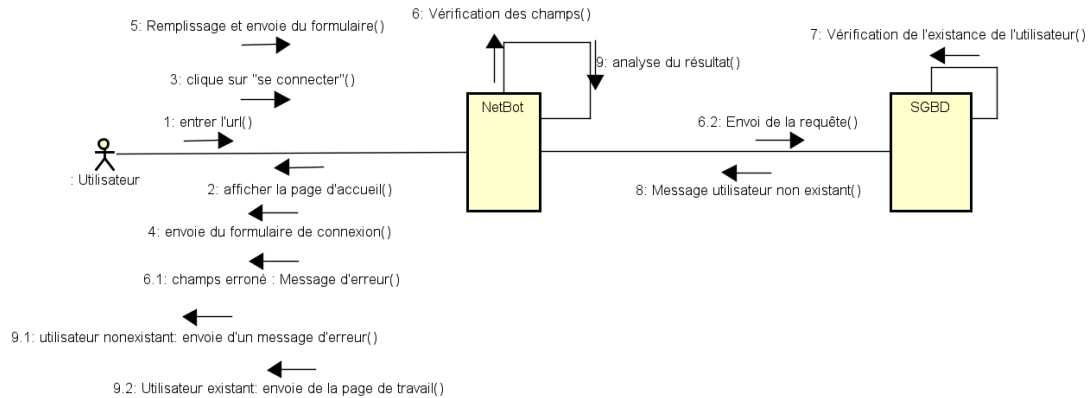


Figure 10 diagramme de communication du cas s'authentifier

4. DIAGRAMME DE COMMUNICATION : MONTER ARCHITECTURE PHYSIQUE

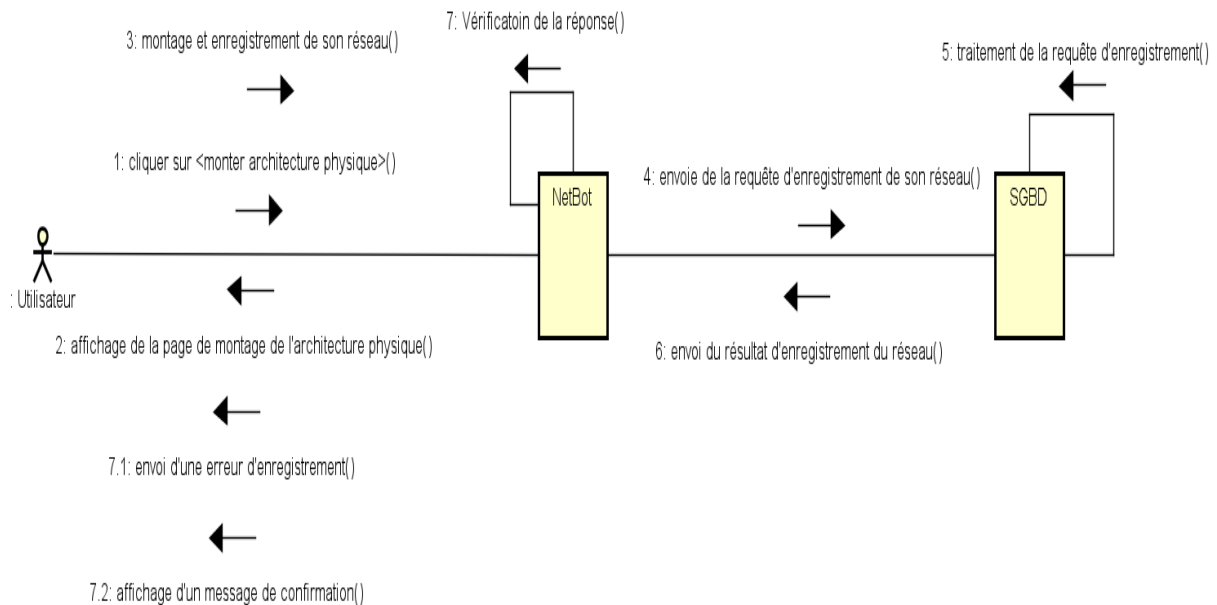


Figure 11 diagramme de communication du cas monter architecture physique

5. DIAGRAMME DE COMMUNICATION : CONFIGURER RESEAU

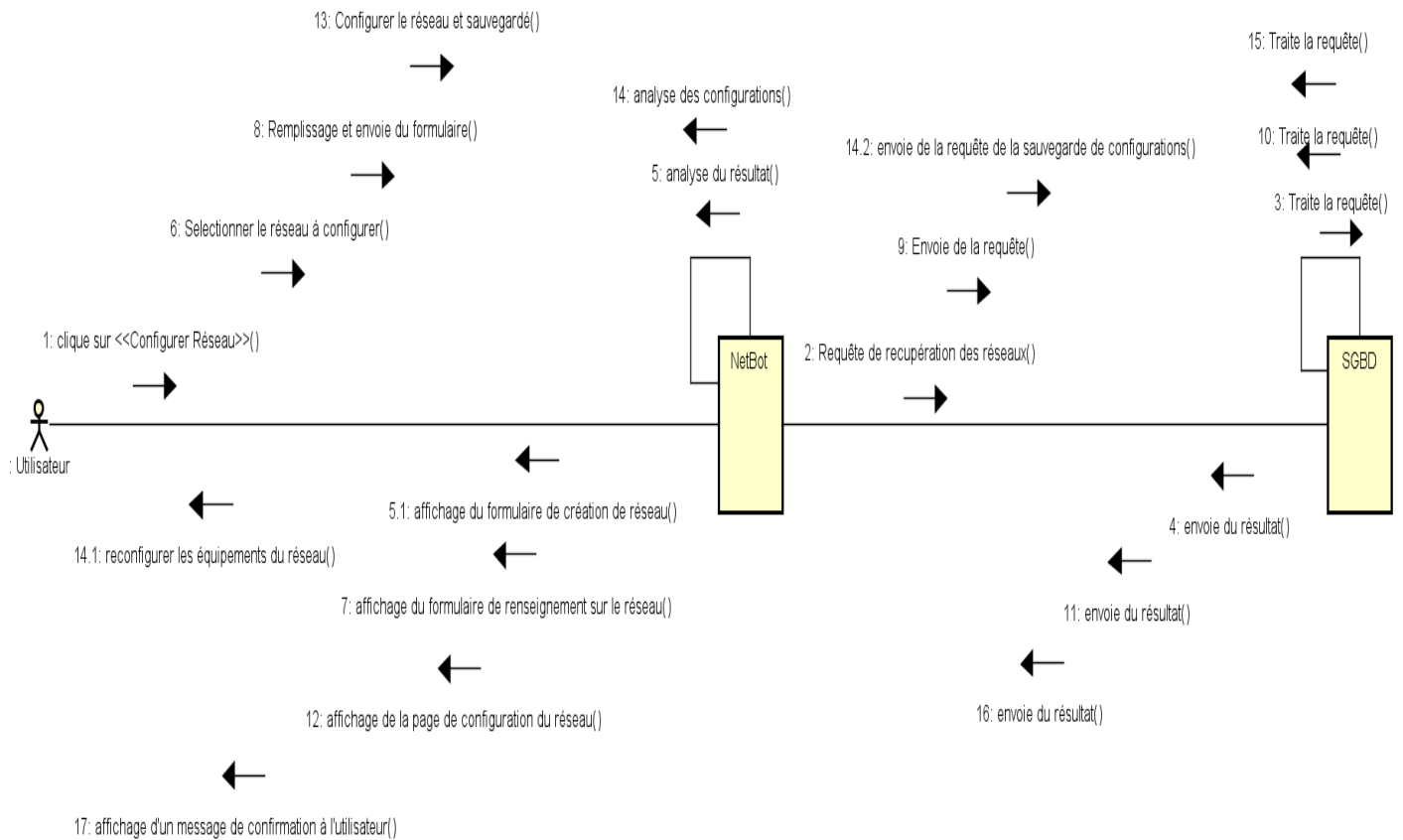


Figure 12 diagramme de communication du cas configurer réseau

6. DIAGRAMME DE COMMUNICATION : DEPLOYER RESEAU

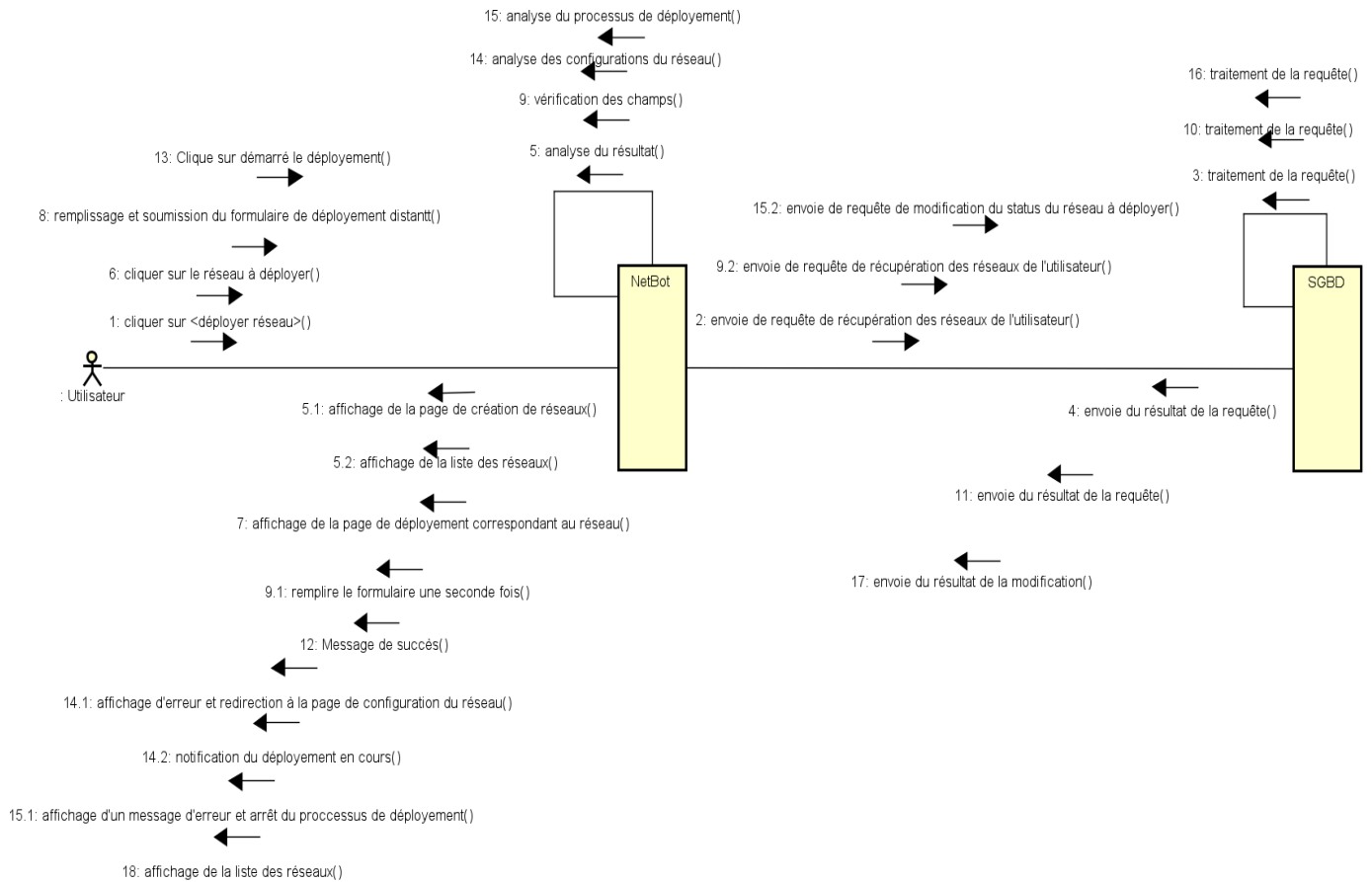


Figure 13 déployer réseau

4. Diagramme de séquence

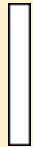
1. Présentation

Les diagrammes de séquences sont des représentations graphiques des interactions entre les acteurs et le système selon un ordre chronologique dans la formulation UML. Le diagramme de séquences permet de décrire les interactions entre les objets au sein d'un diagramme des cas d'utilisation.

Le diagramme de séquence énumère les objets horizontalement et le temps verticalement. Il modélise l'exécution des différents modèles en fonction du temps. Dans ce diagramme, les objets et les acteurs sont énumérés en colonnes avec leur ligne de vie verticale indiquant la durée de vie de l'objet.

2. Composant et Formalisme

Tableau 18 formalisme du diagramme de séquence

Composant	Description	Représentation
Objet	Se décrit comme étant l'instance d'une classe	
Acteur	Il s'agit d'une personne qui interagit et communique avec le système.	
Ligne de vie	La ligne de vie, représente le déroulement temporel d'un processus.	
Activation	Il s'agit ici du temps nécessaire pour qu'un objet ou un acteur accomplisse une tâche, elle indique quand l'objet effectue une action.	
Message synchrone	On utilise ce symbole lorsqu'un expéditeur doit attendre une réponse à un message avant de continuer. Le diagramme doit montrer à la fois l'appel et la réponse	
Messages de retour asynchrones	Les messages asynchrones ne nécessitent pas de réponse avant que l'expéditeur ne continue. Seul l'appel doit être inclus dans le diagramme.	
Message de retour	Ces messages sont des réponses aux appels.	

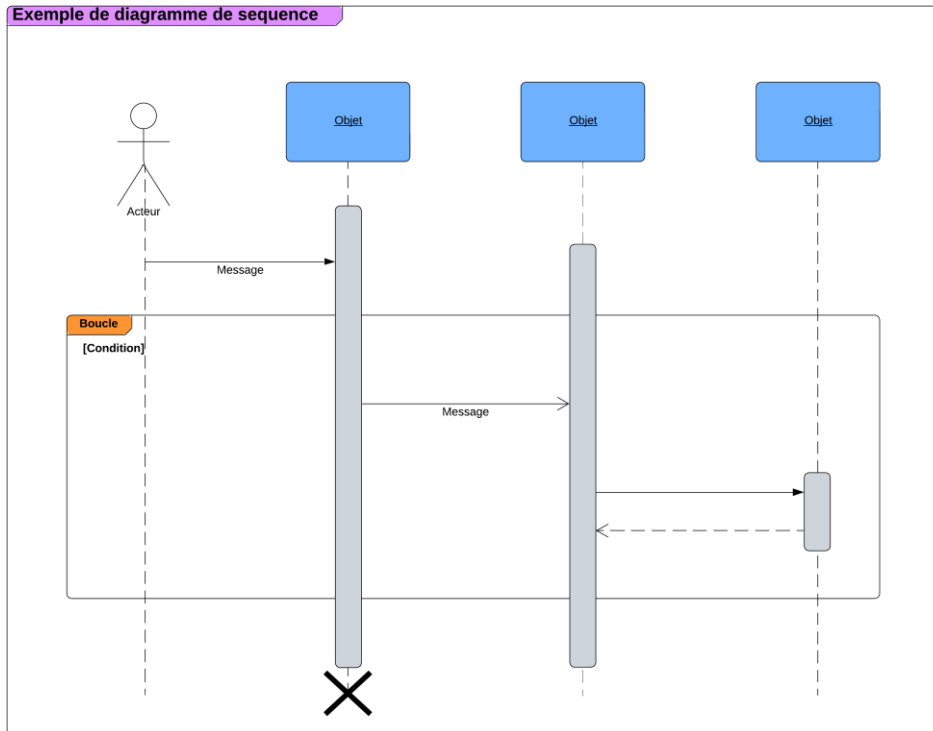


Figure 14 exemple de diagramme de séquence

3. DIAGRAMME DE SEQUENCE : S'Authentifier

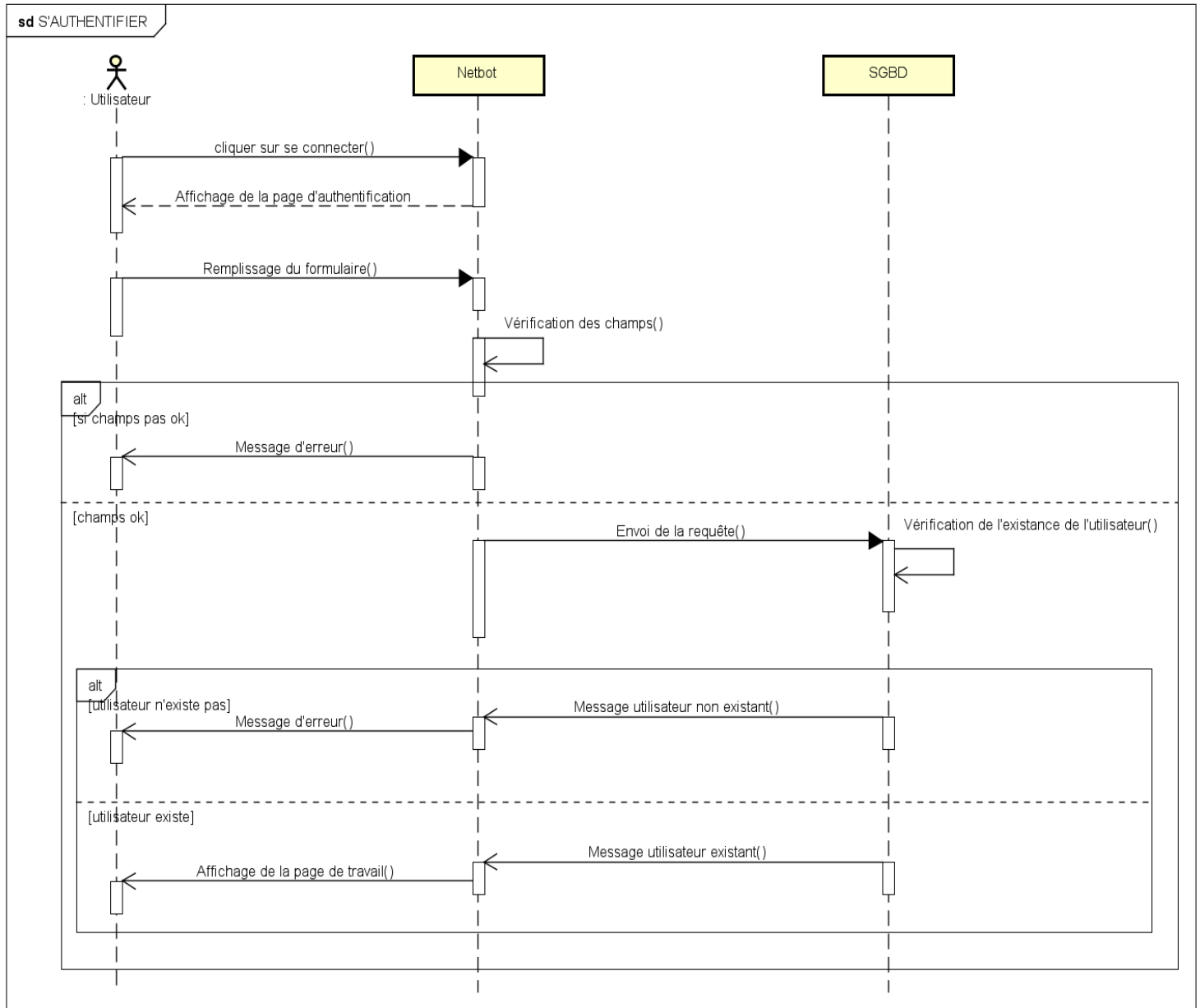


Figure 15 diagramme de séquence du cas s'authentifier

4. DIAGRAMME DE SEQUENCE : Monter Architecture Physique

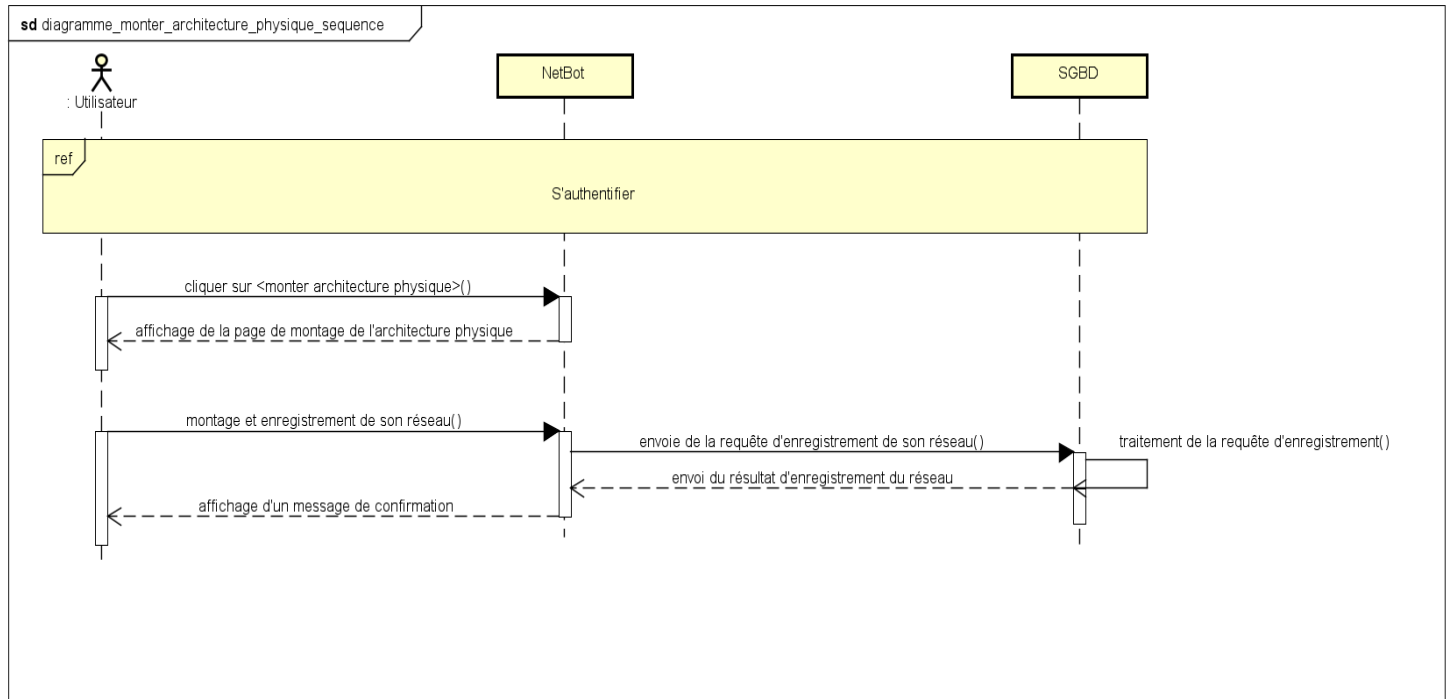


Figure 16 diagramme de séquence du cas d'utilisation monter architecture physique

5. DIAGRAMME DE SEQUENCE : Configurer Réseau

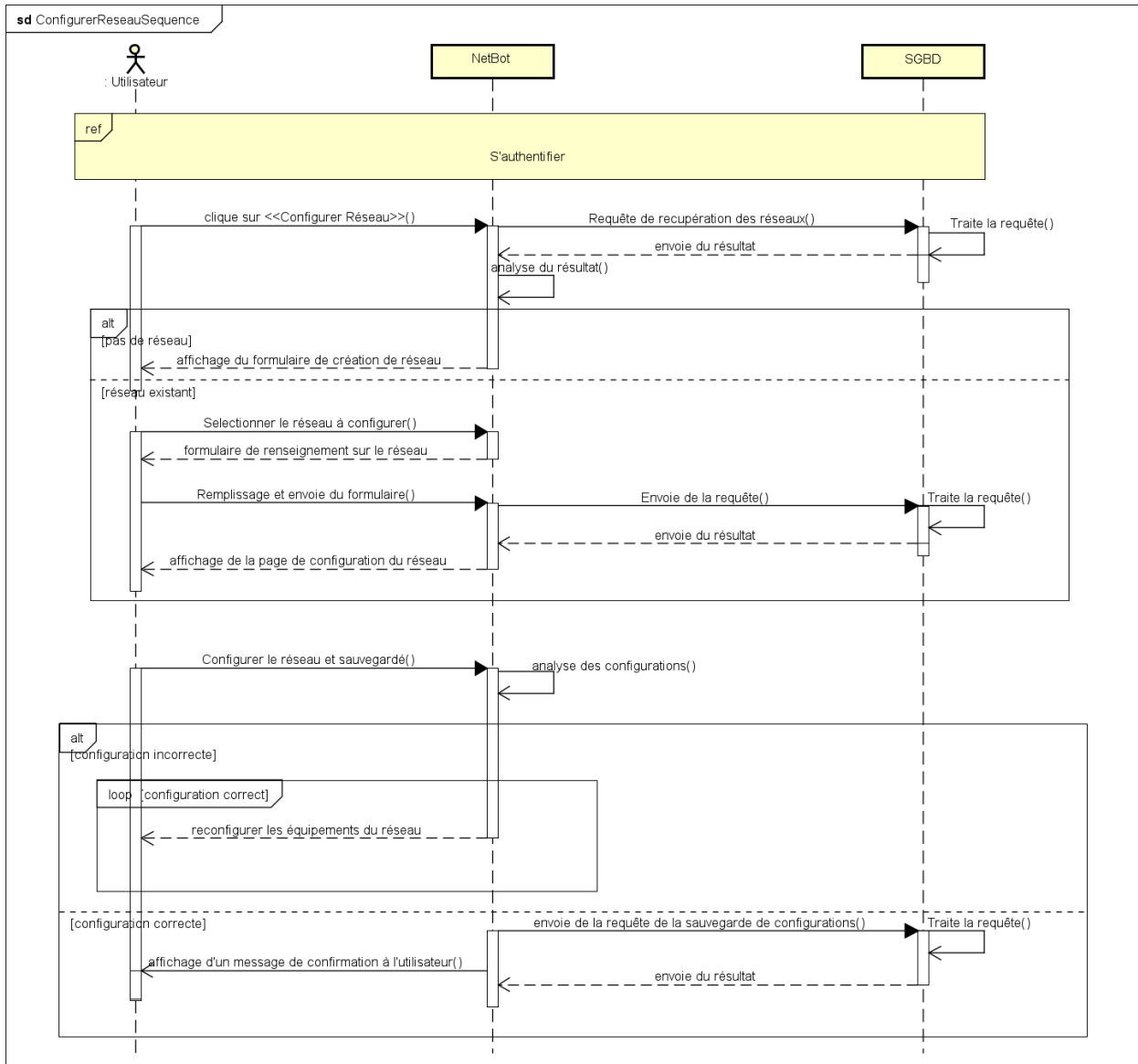


Figure 17 diagramme de séquence du cas configurer réseau

6. DIAGRAMME DE SEQUENCE : Déployer Réseau

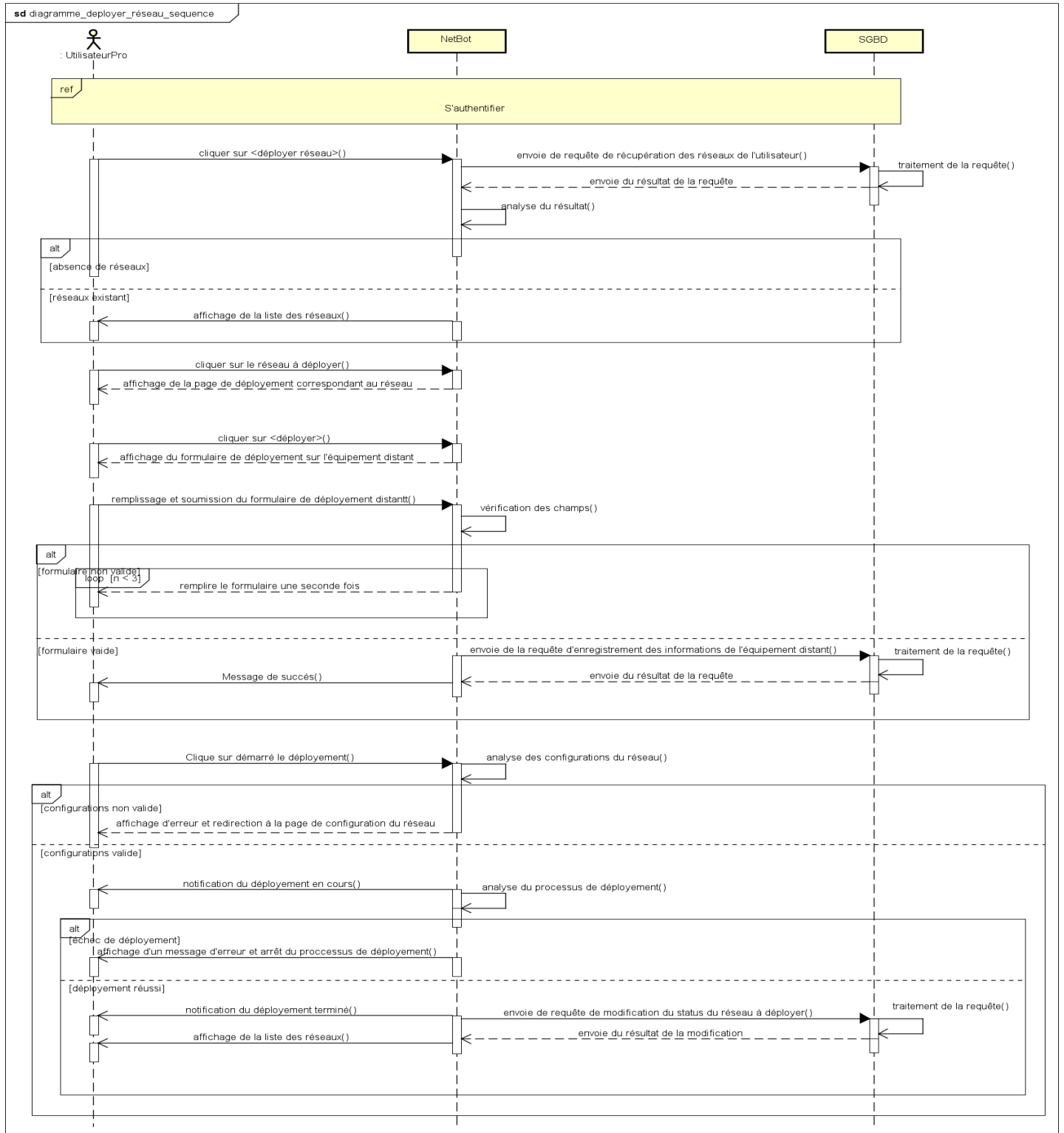


Figure 18 diagramme de séquence du cas déployé réseau

5. Diagramme d'activité

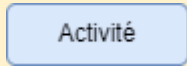
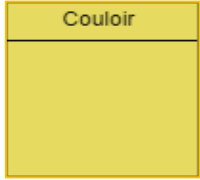
1. Présentation

Les diagrammes d'activités sont particulièrement adaptés à la description des cas d'utilisation. Plus précisément, ils viennent illustrer et consolider la description textuelle des cas d'utilisation.

L'usage général du diagramme d'activité permet de mettre l'accent sur les traitements et de formaliser graphiquement la séquence d'actions réalisées dans un cas d'utilisation.

2. Composant et Formalisme

Tableau 19 formalisme du diagramme d'activité

Composant	Description	Représentation
Une action	Rectangle aux coins arrondis qui représente une opération élémentaire et instantanée	
Une transition	Ligne qui représente le passage d'une activité à une autre	
Nœud initial	Cercle plein à partir duquel un flot débute dans un diagramme d'activité	
Nœud final	Un nœud de fin d'activité : un petit cercle plein marquant l'arrêt l'exécution de l'activité enveloppante s'arrête	
	Un nœud de fin de flot : cercle vide barré d'une croix marquant l'arrêt d'un flot à l'intérieur du diagramme	
Couloir	Les diagrammes d'activités font intervenir les acteurs de chaque activité. Chaque activité sera placée dans une colonne (couloir) qui correspond à l'acteur qui l'effectue.	
Barre de synchronisation	Barre horizontale sur laquelle convergent les activités qui sont exécutées en parallèle.	

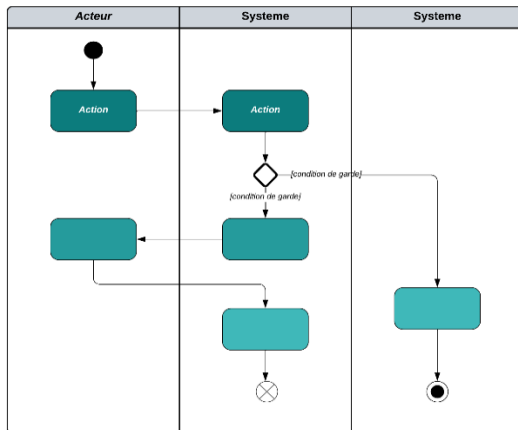


Figure 19 exemple de diagramme d'activité

3. DIAGRAMME D'ACTIVITE : S'Authentifier

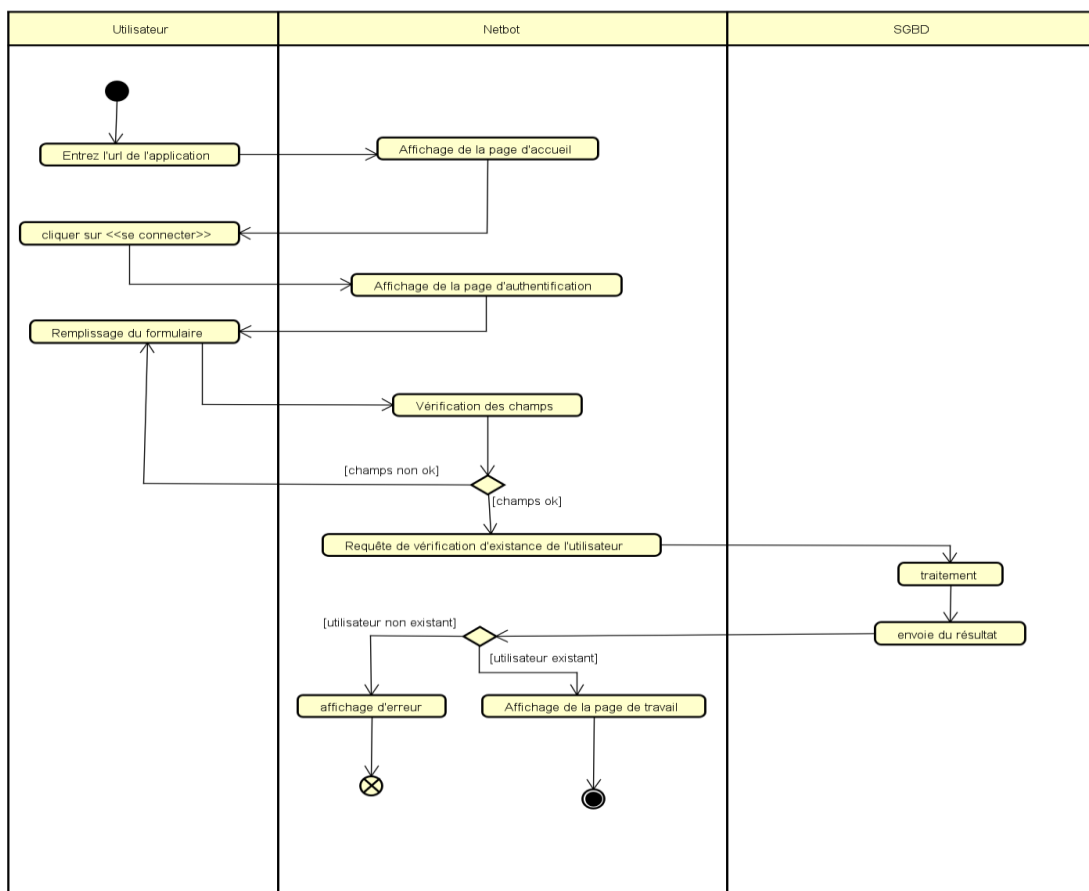


Figure 20 diagramme d'activité du cas s'authentifier

4. DIAGRAMME D'ACTIVITE : Configurer Réseau

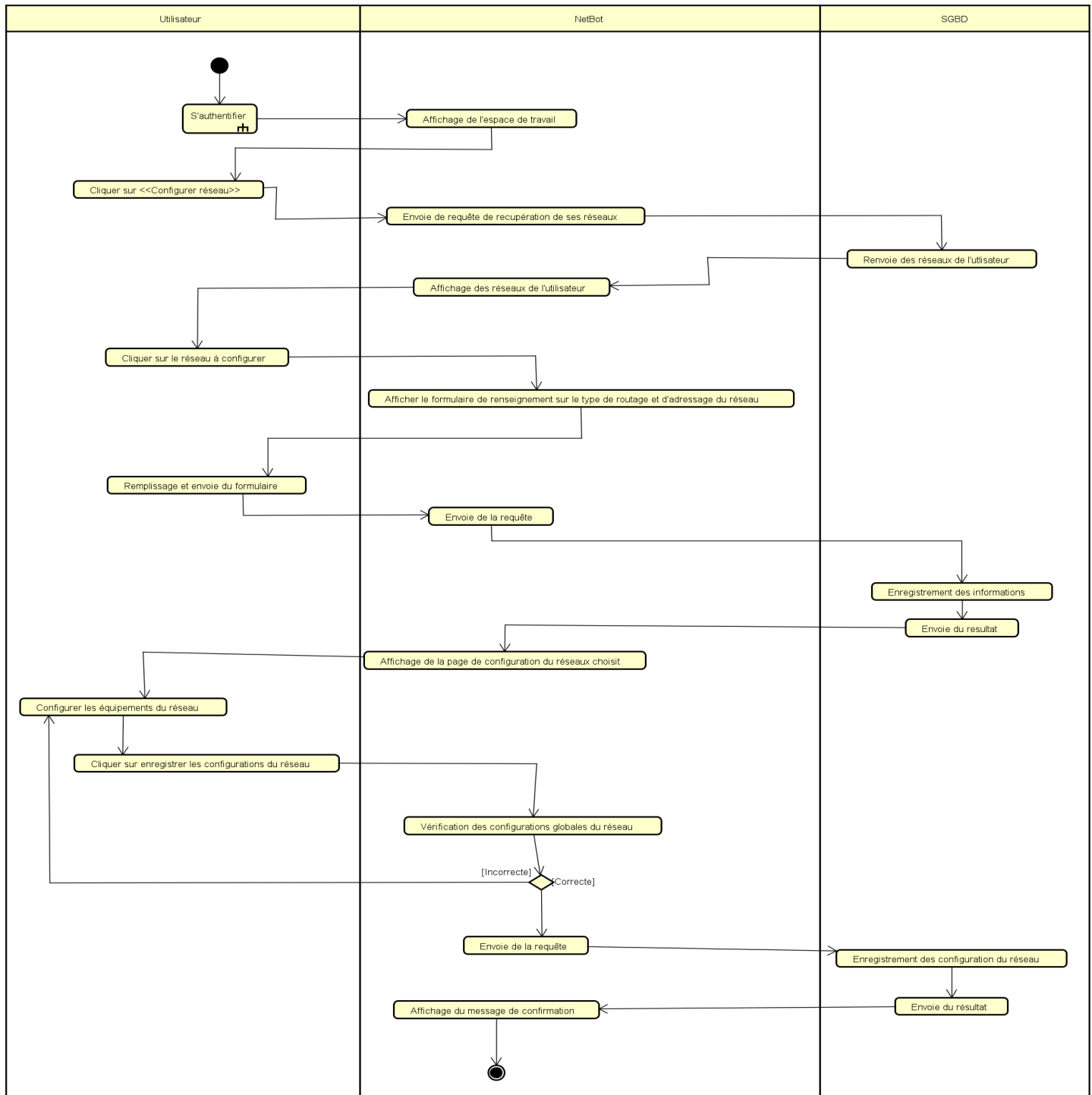


Figure 21 diagramme d'activité du cas configurer réseau

5. DIAGRAMME D'ACTIVITE : Monter Architecture Physique

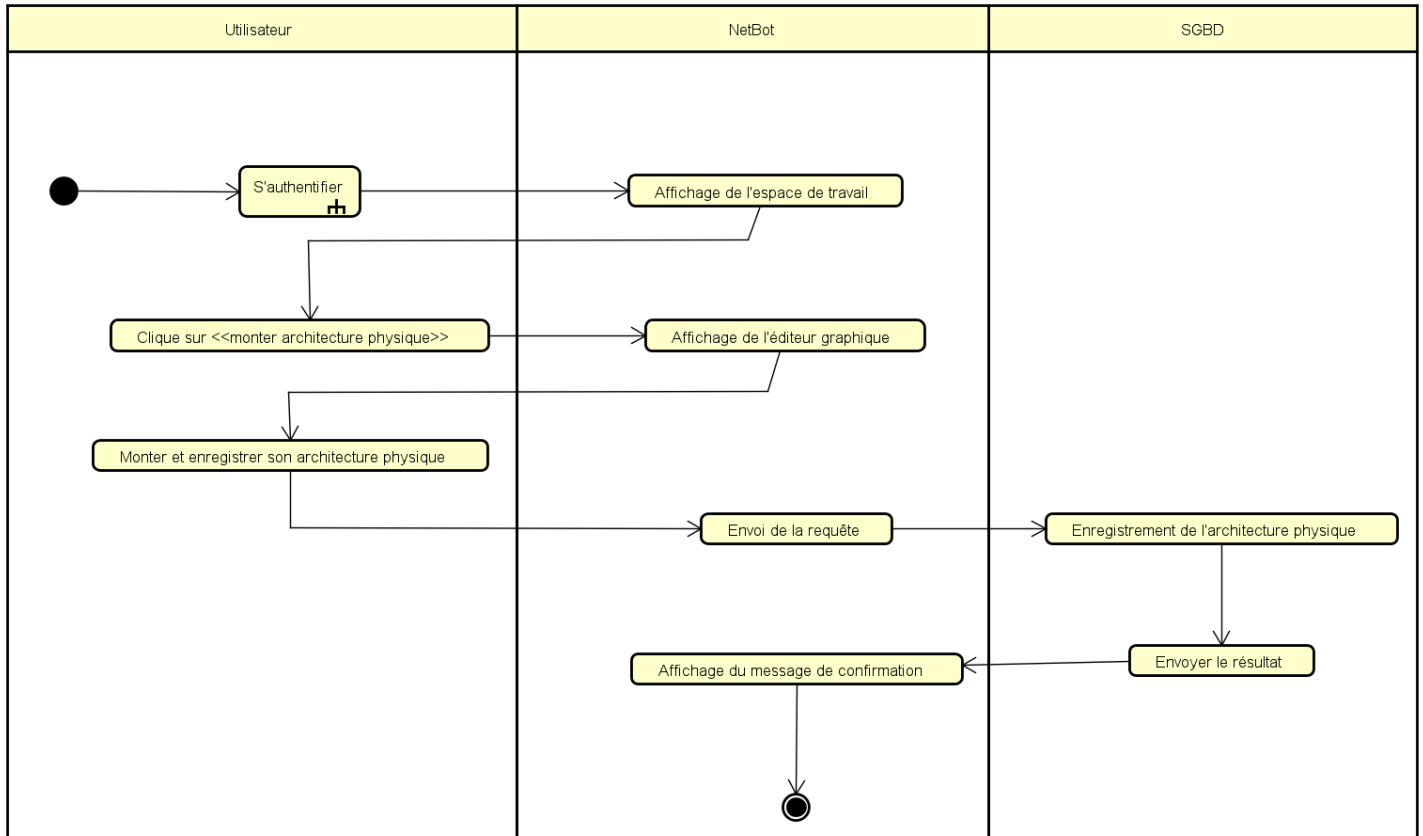


Figure 22 diagramme d'activité du cas monter architecture physique

6. DIAGRAMME D'ACTIVITE : Déployer Réseau

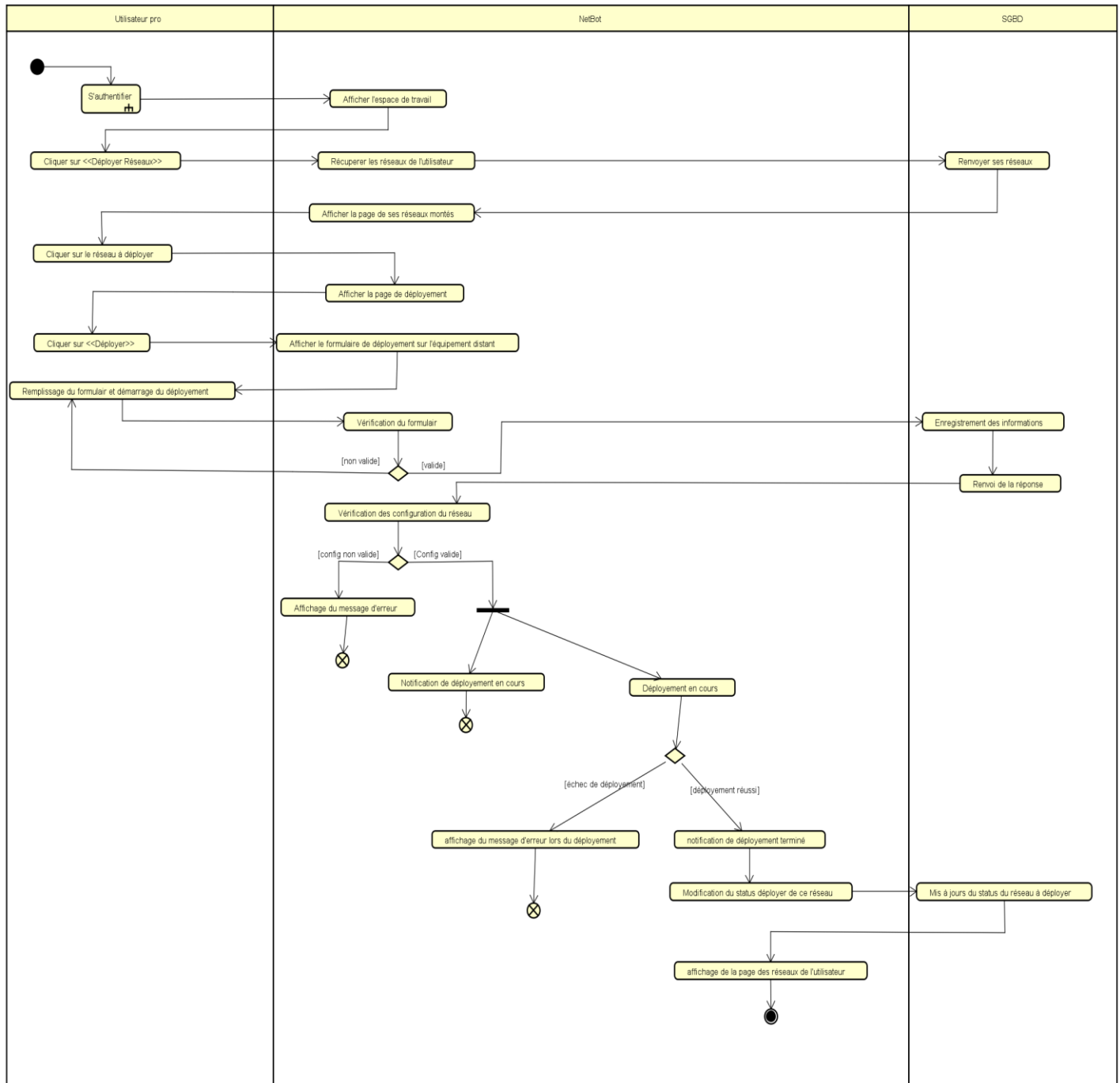


Figure 23 diagramme d'activité du cas déployé réseau



Conclusion

Dans ce dossier, on a présenté une étude du système existant, les lacunes qu'il comprend ainsi que les solutions que nous proposons pour pallier ces problèmes. Ce dossier d'analyse nous a permis d'avoir un aperçu détaillé du système à mettre sur pied. Toujours en exploitant le langage UML et le processus associé 2TUP, nous poursuivrons avec le dossier de conception dans lequel nous représenterons les diagrammes de la branche technique.

DOSSIER 4 : DOSSIER DE CONCEPTION

Résumé :

Le dossier de conception permet de modéliser dans son ensemble la solution proposée et de recueillir les informations nécessaires à la mise sur pied d'une base de données complexes et efficaces. De ce fait il prévoit le système futur.

Aperçu :

Introduction

1. Diagramme de classe
2. Diagramme d'état transition
3. Diagramme de paquetage

Conclusion



Introduction

Une bonne méthodologie de la réalisation d'un logiciel suppose une bonne maîtrise d'analyse et conception. Cette dernière nous offre tous les modèles destinés à assurer le fonctionnement du logiciel. Dans le dossier de conception, il est question de ressortir à travers les diagrammes UML, l'architecture statique de notre futur logiciel. De manière globale, le dossier de conception offre une vue panoramique sur l'ensemble des éléments et les interactions prise en compte dans le dossier d'analyse.

II. Diagramme de classe

1. Présentation

Le diagramme de classe met en évidence d'éventuelles relations entre ces classes. Le diagramme de classes comporte 6 concepts : **classe, attribut, identifiant, relation, opération, généralisation/spécialisation**. Ce diagramme est constitué des éléments suivants :

- **Classe** : il s'agit de la description abstraite à un ensemble d'objet : elle définit leurs structures, leurs comportements et leurs relations.
- **L'attribut** : Il représente la modélisation d'une information élémentaire représentée par son nom et format.

UML définit 3(trois) niveaux de visibilité pour les attributs :

- **Public (+)** : l'élément est visible par tous les clients de la classe.
- **Protégé (#)** : l'élément est visible que par les sous-classes ou les classes filles.
- **Privé (-)** : l'élément n'est visible que par les objets de la classe auxquels il appartient.
- **L'identifiant** : c'est un attribut particulier, qui permet de repérer de façon unique chaque objet, instance de la classe.
- **Multiplicité** : elle définit le nombre d'instance de la classe, elle est définie par un nombre entier ou un intervalle de valeur (1...1, 0...1, 1...*, 0...*).
- **Associations** : Une association est une relation entre deux classes (association binaire) ou plus (association n-aire), qui décrit les connexions structurelles entre leurs instances.

Il existe plusieurs types de relations dont les plus connues sont les suivants :

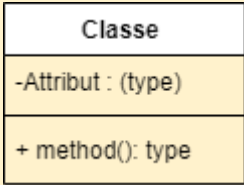
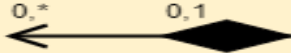
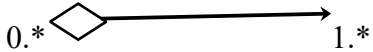
- **Généralisation** : Relation d'héritage, dans laquelle les objets de l'élément spécialisé (Classe enfant) peuvent remplacer les objets de l'élément général (Classe parent)

- **Agrégation** : utilisé pour les relations de types « contenant / contenue »
- **La composition** : La composition est un cas particulier de l'agrégation dans laquelle la vie des composants est liée à celle des agrégats.

Le tableau suivant nous donne une liste assez détaillée du formalisme du diagramme de classe en un tableau :

2. Composants et Formalisme

Tableau 20 formalisme du diagramme de classe

Elément	Représentation
Classe	
Association	
Composition	
Agrégation	
Généralisation	
Multiplicité	0.. *, 0...1, 1...1, 1...*
Modificateur d'accès	+, -, #, ~, /

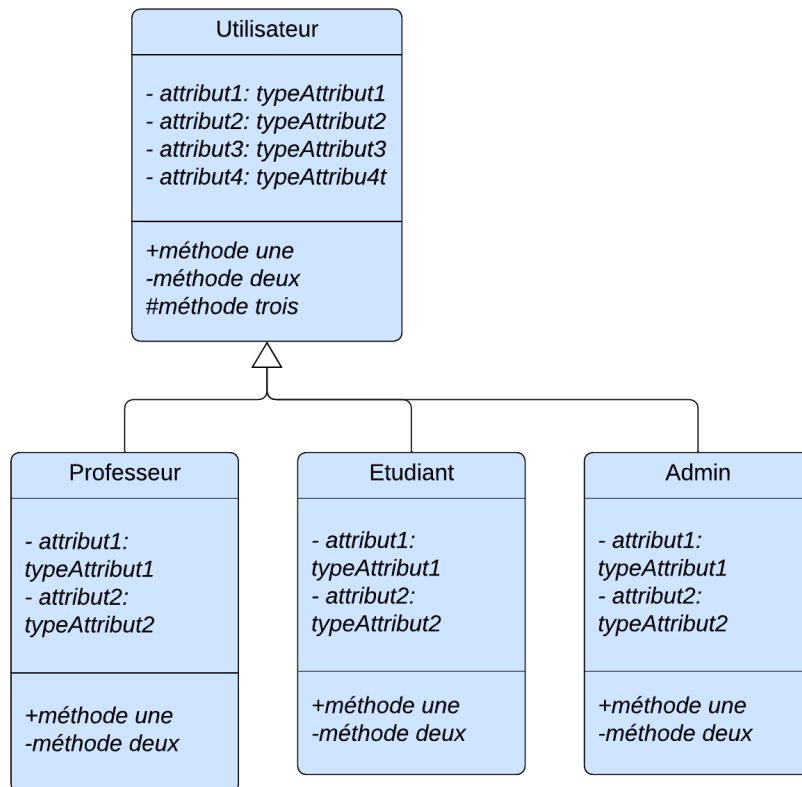


Figure 24 formalisme d'un diagramme de classe

Le diagramme de classe de notre projet est le suivant :

1. Diagramme de classe de notre système

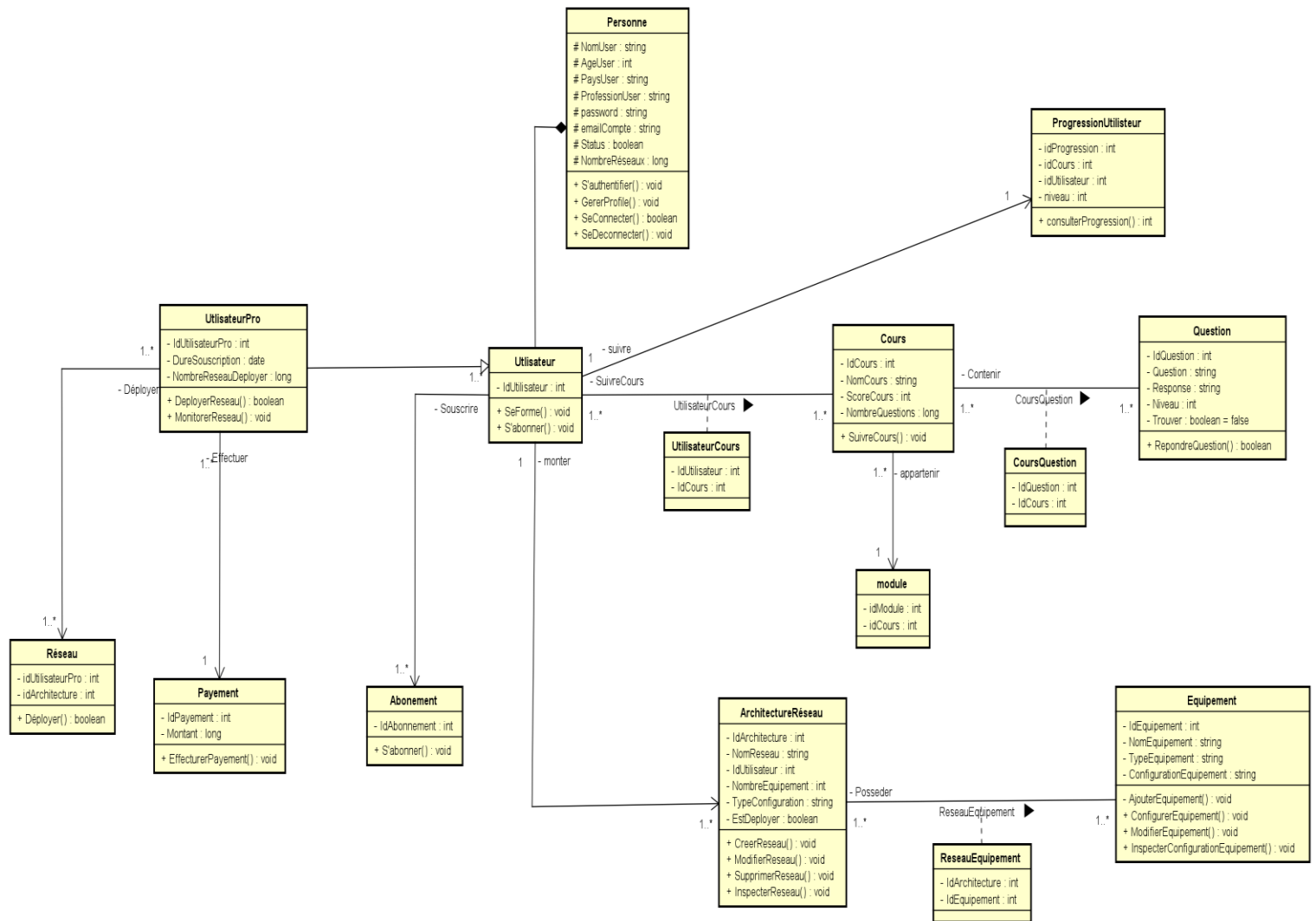


Figure 25 diagramme de classe de notre système

2. Règles métiers de notre diagramme de classe :

Dans le cadre de programmation Orienté Objet, chaque classe doit être responsable de tâches précises, laissant à d'autres classes le soin de prendre en charge d'autres tâches. Les relations entre classes qui ressortent de notre analyse sont les suivantes :

Tableau 21 règles métiers de notre système

Relation	Description
R1	Un Utilisateur peut gérer son compte et un compte peut être géré par un et seule utilisateur.
R2	Un utilisateur pro peut déployer un ou plusieurs réseaux et un réseau peut être déployé par un et un seul utilisateur pro.
R3	Un utilisateur peut monter un ou plusieurs réseaux et un réseau peut être monté par un et un seul utilisateur.
R4	Un utilisateur peut effectuer un ou plusieurs paiements et un paiement peut être effectué par un et un utilisateur.
R5	Un utilisateur pro peut souscrire à un ou plusieurs abonnements et un abonnement peut recevoir un abonnement d'un et d'un seul utilisateur.
R6	Un utilisateur peut administrer un réseau et un réseau peut être administré par un utilisateur.
R7	Un réseau peut posséder un ou plusieurs équipements et un équipement peut être possédé par un ou plusieurs réseau
R8	Un utilisateur peut suivre un ou plusieurs cours et un cours peut être suivi par un ou plusieurs utilisateur
R9	Un cours peut contenir un ou plusieurs questions et une question peut être contenue par un ou plusieurs cours
R10	Un utilisateur pro peut déployer un ou plusieurs réseau
R11	Un utilisateur peut suivre sa progression et une progression peut être suivie par un et un seul utilisateur.

III. Diagramme d'état transition

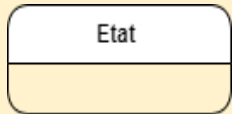
1. Présentation

Le diagramme d'états-transition est comme son nom l'indique décrit les transitions entre les états et les actions que le système ou ses parties réalisent en réponse à un événement. Les diagrammes d'états-transitions permettent justement de spécifier les réactions d'une partie du système à des événements discrets. Un événement se produit à un instant précis et est dépourvu de durée.

Le diagramme d'états-transition décrit l'état ou le comportement d'un objet à un événement précis du système. A cet effet il est composé des éléments suivants :

2. Composants et Formalisme

Tableau 22 formalisme du diagramme d'état transition

Elément	Description	Représentation
Etat initial	Il s'agit de l'état dans lequel il se trouve lors de sa création. Est représenté par un point noir.	
Etat final	Correspond à une étape où l'objet n'est plus nécessaire dans le système et où il est détruit. Il se représente par un point entouré d'un cercle.	
Etat intermédiaire	Est une situation stable qui possède une certaine durée pendant laquelle un objet exécute une activité ou attend un Événement. Il est représenté par un rectangle arrondi contenant son nom.	
Transition	Une transition définit la réponse d'un objet à l'arrivée d'un événement. Elle est représentée par une flèche.	

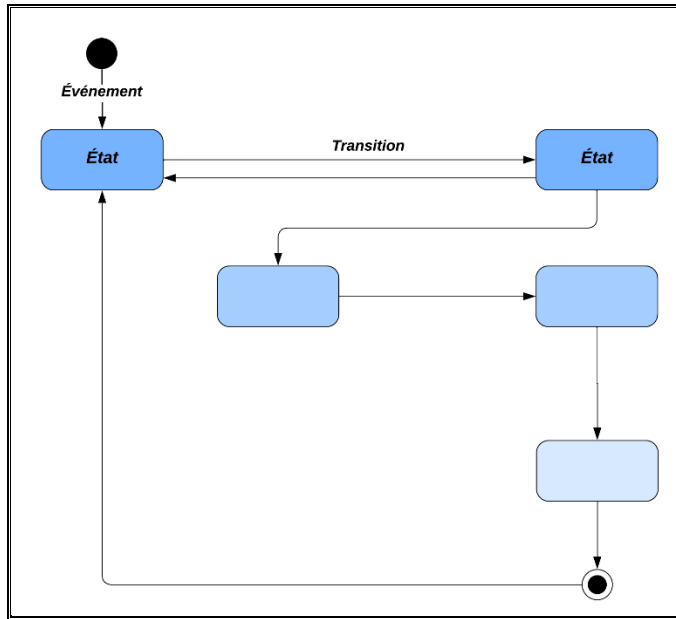


Figure 26 formalisme du diagramme d'état transition

Quelques diagrammes d'état-transition de notre projet ci-dessous :

- Cas de la connexion :

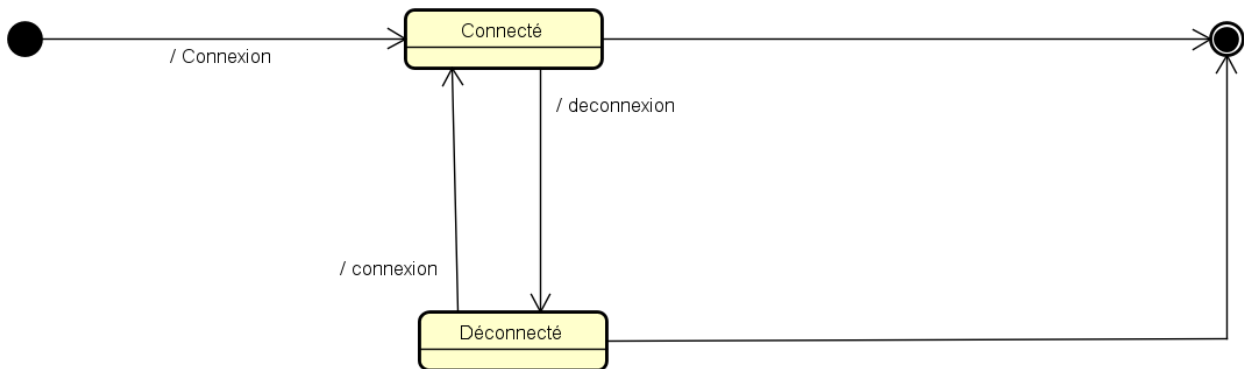


Figure 27 diagramme d'état transition du cas de connexion

- Cas d'un utilisateur :

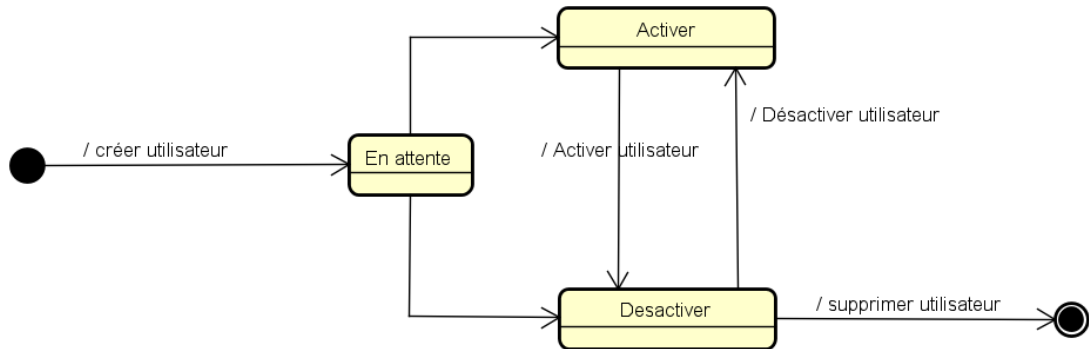


Figure 28 Diagramme d'état transition d'un utilisateur

IV. Diagramme de paquetage

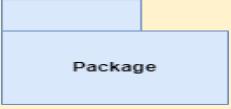
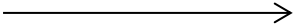
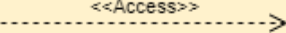
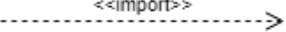
1. Présentation

Les diagrammes de package (ou diagramme de paquetages) permettent de décomposer le système en catégories ou parties plus facilement observable s'appelés « packages ». L'utilisation la plus courante pour les diagrammes de paquetages est d'organiser des diagrammes de Cas d'Utilisation et des diagrammes de Classes. Bien que l'utilisation des Diagrammes de Paquetages ne se limite pas à ces éléments UML.

2. Composants et Formalisme

Le formalisme est le suivant :

Tableau 23 formalisme du diagramme de paquetage

Composant	Description	Représentation
Package	Regroupement des cas d'utilisation en catégorie	
Merge	Représente la fusion de deux packages. Le contenu du package cible est intégré au package source.	
Access	Indique que les éléments d'un package ont accès aux éléments publics d'un autre package.	
Import	Permet à un package d'importer les éléments publics d'un autre package.	

Le diagramme paquetage de notre projet est le suivant :

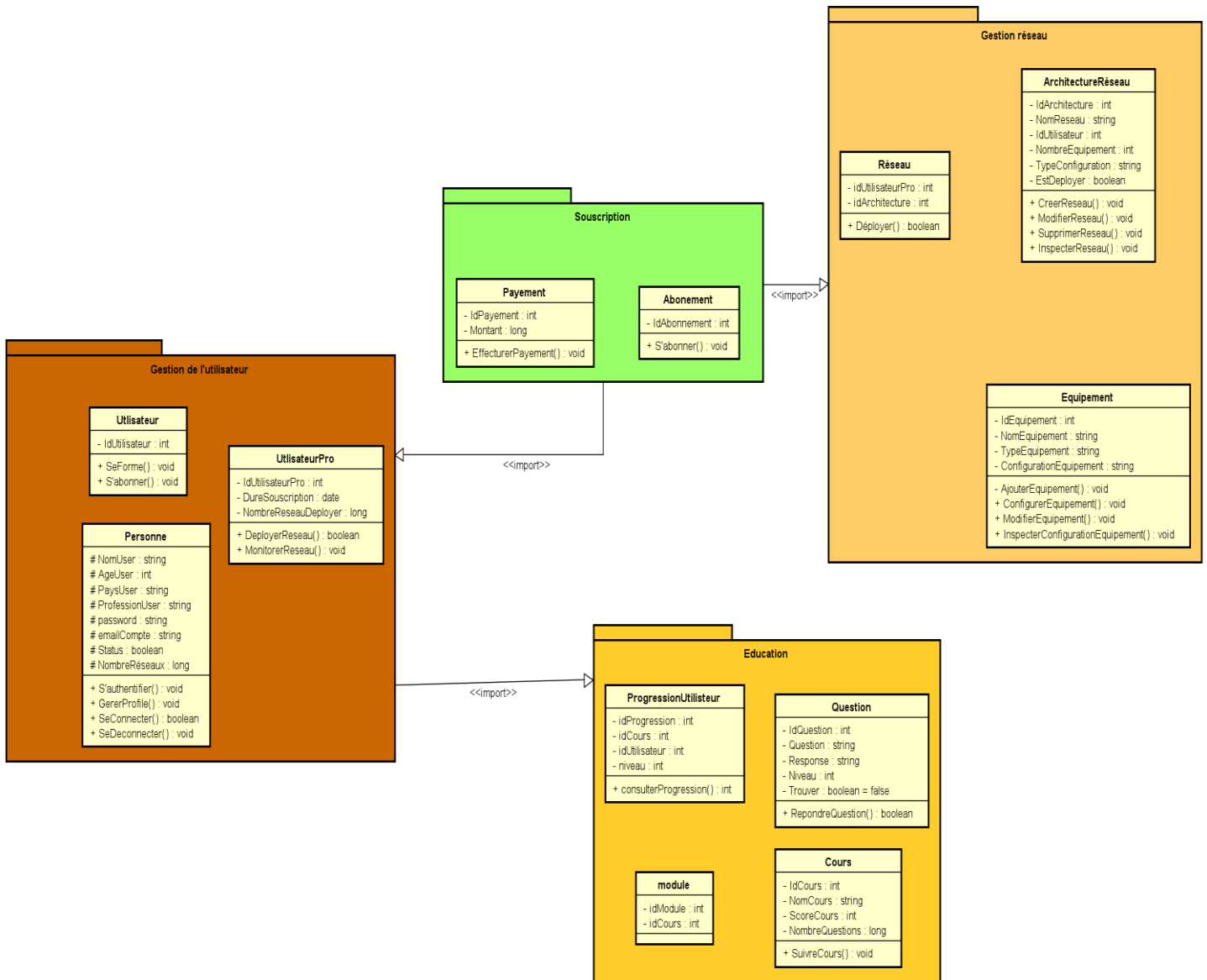


Figure 29 diagramme de paquetage de notre système



Conclusion

En sommes, ce dossier analyse nous a permis de bien délimiter les besoins fonctionnels de notre application afin d'avoir une étude existentielle, sa critique et d'avoir un aperçu détaillé du nouveau système à mettre en place à travers la méthode d'analyse UML associé à son processus 2TUP.



DOSSIER 5 : DOSSIER DE RÉALISATION

Résumé :

La réalisation consiste à la mise en œuvre de projets dans un langage de programmation conformément aux spécifications définies dans le dossier précédent. Elle renferme en son sein les diagrammes de composant et de déploiement. A la sortie de cette partie, il sera produit une documentation de programmation expliquant l'architecture de la base de données et l'architecture de notre code.

Aperçu :

Introduction

1. Choix des technologies
2. Architecture physique et logique du système
3. Diagramme de déploiement
4. Diagramme de composant

Conclusion



Introduction

Fortement influencé par la phase de conception mais aussi d'analyse, la réalisation s'attaque au domaine physique et concret du projet. Tout au long de cette phase, la maîtrise d'œuvre met en place les différents codes et architecture utilisée dans la mise sur pied de la solution. Il sera donc question pour nous de présenter dans un premier temps le langage de programmation utilise, puis les outils de développement, enfin l'architecture de notre application.

I. Choix des technologies

1. Environnement logiciel

Différents logiciels utilisés lors de la réalisation de notre solution :

Tableau 24 différents logiciels utilisés pour la réalisation de notre solution

NOM DU LOGICIEL	DESCRIPTION
	Google Chrome : navigateur web pour tester et déployé des applications web.
	Git : Système de contrôle de version pour gérer les modifications du code et faciliter la collaboration entre développeurs.
	Gantt Project : Outil de gestion de projet pour créer des diagrammes de Gantt, planifier des tâches et suivre l'avancement.
	Astar : Outil de modélisation pour concevoir des bases de données et des systèmes d'information (modèles UML, MCD, MLD).
	PhpStorm : Environnement de développement intégré (IDE) léger pour coder, déboguer, et gérer des projets en Php.
	PyCharm : Environnement de développement intégré (IDE) léger pour coder, déboguer et gérer des projets en langage Python.
	Gns3 : est un logiciel libre et gratuit qui permet aux professionnels et aux étudiants de créer et de simuler des architectures réseau complexes en utilisant des appareils virtuels réels ou émuls

2. Technologies Web utilisée

Technologies utilisées en frontend pour notre solution :

Tableau 25 différentes technologies utilisées en frontend pour réaliser notre solution.

TECHNOLOGIES FRONT-END	
	HTML : Langage de balisage standard pour structurer le contenu des pages web.
	CSS : Feuille de style en cascade pour styliser et agencer visuellement les pages web.
	TailwindCSS : Framework CSS pour concevoir des interfaces web réactives et attractives rapidement avec des composants préconstruits.
	JavaScript : Langage de programmation pour ajouter des interactions dynamiques et des fonctionnalités aux pages web.
	VueJS : Framework JavaScript pour la réalisation d'applications web dynamique et interactive plus rapidement.

3. Technologies Web backend

Technologies utilisées en backend pour la réalisation de notre solution :

Tableau 26 différentes technologies utilisées en backend pour la réalisation de notre application

TECHNOLOGIES BACKEND	
	Laravel : Framework web Php pour développer des application web sécurisé et évolutives rapidement.
	Php : Langage de programmation qui permet de développer des solutions web dynamiques, modulable, évolutive et sécurisé.
	Python : Langage de programmation polyvalent utilisé pour le développement web, l'automatisation, la data science, et l'intelligence artificielle.
	Flask : Framework Python qui permet de concevoir des server et api web sécurisé.

II. Architecture physique et logique du système

1. Architecture physique

Notre application utilise une architecture N-tiers. L'architecture N-tier (anglais tier : étage, niveau), ou encore appelée multi-tier, est une architecture client-serveur dans laquelle une application est exécutée par plusieurs composants logiciels distincts. Dans cette architecture, le lien entre les niveaux est défini et limité à des interfaces et interfaces assurent la modularité et l'indépendance technologique et topologique de chaque niveau.

Les différentes couches d'une architecture 4-tier sont :

🌈 **La couche de présentation** contient les différents types de clients, léger (ASP, JSP) ou lourd (Applet) ;

🌈 **La couche applicative** contient les traitements représentant les règles métier ;

Rédigé par : **NKONO NDEME Miguel**, étudiant en 2^e année Système et Réseaux
IAI Cameroun, Centre d'Excellence Technologique PAUL BIYA
Année académique 2024-2025

- ✚ **La couche d'objets métier** est représentée par les objets du domaine, c'est à dire l'ensemble des entités persistantes de l'application ;
- ✚ **La couche d'accès aux données** contient les usines d'objets métier, c'est à dire les classes chargées de créer des objets métier de manière totalement transparente, indépendamment de leur mode de stockage (SGBDR, Objet, Fichiers, ...).

Cette séparation par couches de responsabilités sert à découpler au maximum une couche de l'autre afin d'éviter l'impact d'évolutions futures de l'application. Par exemple : si l'on est amené à devoir changer de base de données relationnelle, seule la couche d'accès aux données sera impactée, la couche de service et la couche de présentation ne seront pas concernées car elles auront été découplées des autres.

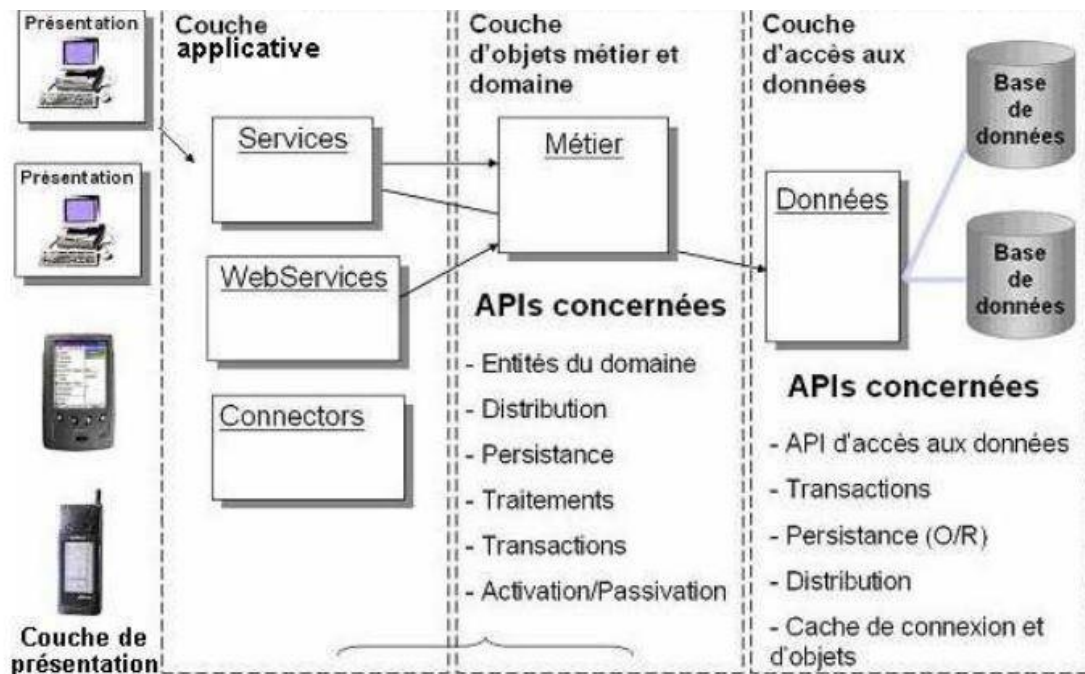


Figure 30 architecture physique n-tiers



2. Architecture logique

Nous allons baser notre logique applicative sur le Modèle Vue Controller (MVC), qui est couramment utilisé dans le développement d'application web avec les framework tel que Laravel en Php, Symfony, NextJs en Javascript.

Le Modèle Vue Controller (MVC) est un patron d'architecture logicielle fréquemment utilisé pour la mise en œuvre d'interfaces utilisateur. Généralement, il divise l'application en trois parties distinctes, favorisant ainsi la modularité et simplifiant la collaboration et la réutilisation de code. Cette structure rend également les applications plus flexibles et mieux adaptées aux cycles d'itération. Le MVC se compose de trois types de modules, chacun étant responsable de tâches spécifiques :

- 🎨 **Modèle (Model) :** C'est la représentation des données et des règles métier de l'application. Le modèle peut être une base de données, un fichier XML, un service web ou toute autre source de données. Le modèle est indépendant de la vue et du contrôleur, ce qui permet de modifier facilement l'un des composants sans affecter les autres.
- 🎨 **Vue (View) :** st responsable de l'affichage et de la mise à jour des données du modèle. Elle peut être une page web ou une fenêtre de bureau. La vue ne contient pas de code de logique métier, elle n'affiche que les données qui lui sont fournies par le contrôleur.
- 🎨 **Controller :** st responsable de récupérer les données du modèle et de les envoyer à la vue, ainsi que de recevoir les entrées de l'utilisateur depuis la vue et de les envoyer au modèle pour effectuer des opérations métier.

III. Diagramme de déploiement

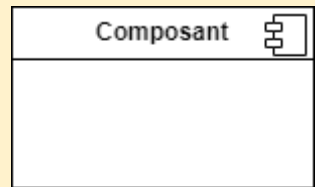
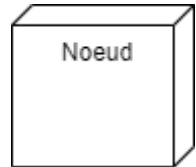
1. Présentation

Le diagramme de déploiement est une vue statique qui sert à représenter l'utilisation de l'infrastructure physique par le système. Et la manière dont les composants du système sont répartis ainsi que les relations entre eux.

Le formalisme est le suivant :

2. Composants et Formalisme

Tableau 27 formalisme du diagramme de déploiement

Elément	Description	Représentation
Les composants	Il représente une entité logicielle du système (Fichier de code source, programmes, documents, fichiers de ressource, etc.). Un composant est représenté par une boîte rectangulaire avec deux rectangles dépassant du côté gauche portant le nom du composant.	
Les nœuds	Un nœud représente un ensemble d'éléments matériels du système. Cette entité est représentée par un cube tridimensionnel.	
Les Associations	Une association, représentée par une ligne pleine entre deux nœuds, indique une ligne de communication entre les éléments matériels.	
Les dépendances	Une dépendance est utilisée pour mobiliser la relation entre deux composants. La notation utilisée pour cette relation de dépendance est une flèche de pointillés.	

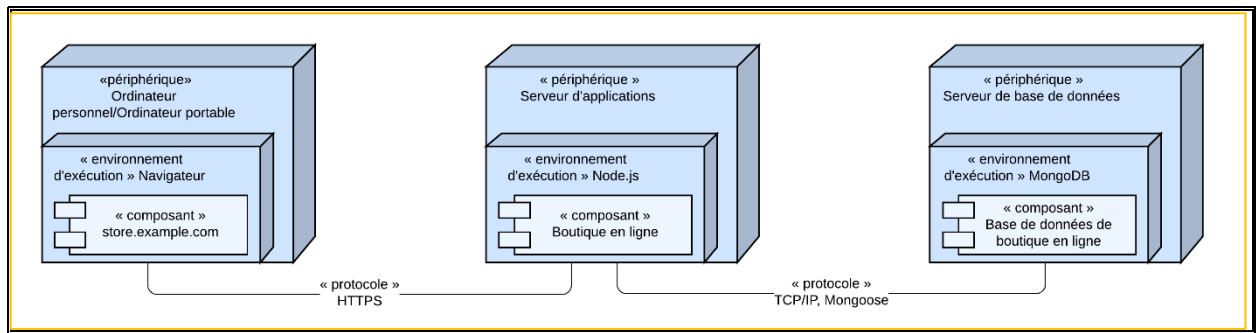


Figure 31 formalisme du diagramme de déploiement

Le diagramme suivant est une architecture utilisée pour le déploiement de notre application :

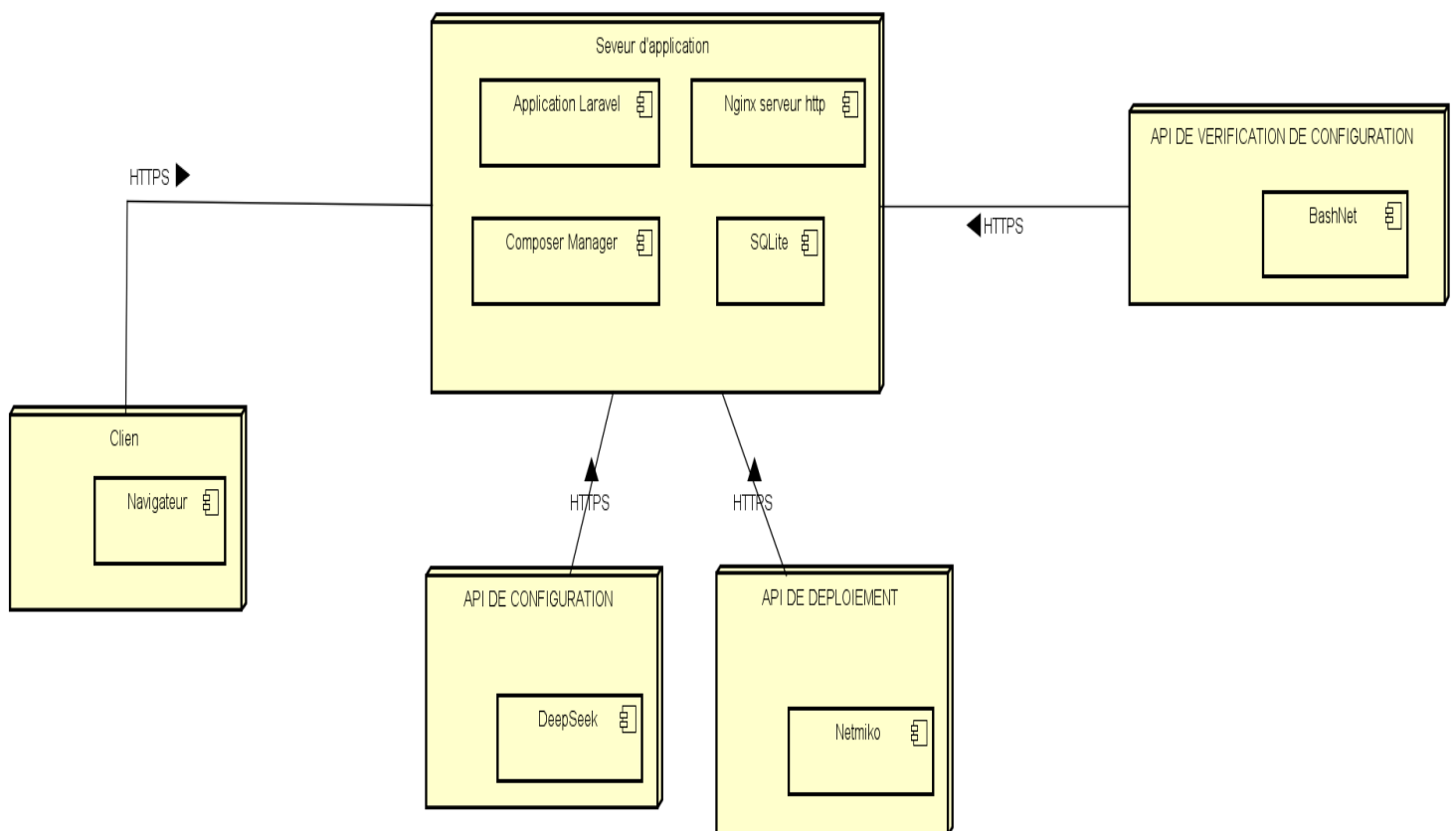


Figure 32 diagramme de déploiement de la solution

IV. Diagramme de composant

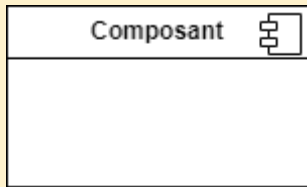
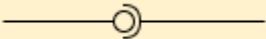
1. Présentation

Le diagramme de composants est principalement employé pour décrire les dépendances entre divers composants (les dépendances entre fichiers exécutables et les fichiers sources). Le diagramme de composants modélise les composants logiciels utilisés pour implémenter un système et l'association entre ces composants

Le diagramme de composant décrit les composants et leurs dépendances dans leurs environnements de réalisation. A cet effet il est composé des éléments suivants :

2. Composants et Formalisme

Tableau 28 formalisme du diagramme de composant

Elément	Description	Représentation
Les Composants	Un composant représente une entité logicielle d'un système. Un composant est représenté par une boîte rectangulaire, avec deux rectangles dépassant du côté gauche.	
Dépendances	Elles sont utilisées pour modéliser la relation entre deux composants. La notation utilisée pour cette relation de dépendance est une flèche pointillée, se dirigeant d'un composant donné au composant dont il dépend.	
Interface (Nœud)	Lien entre deux composants logiciels. Il en existe 2 : Interface fournie Interface requise	

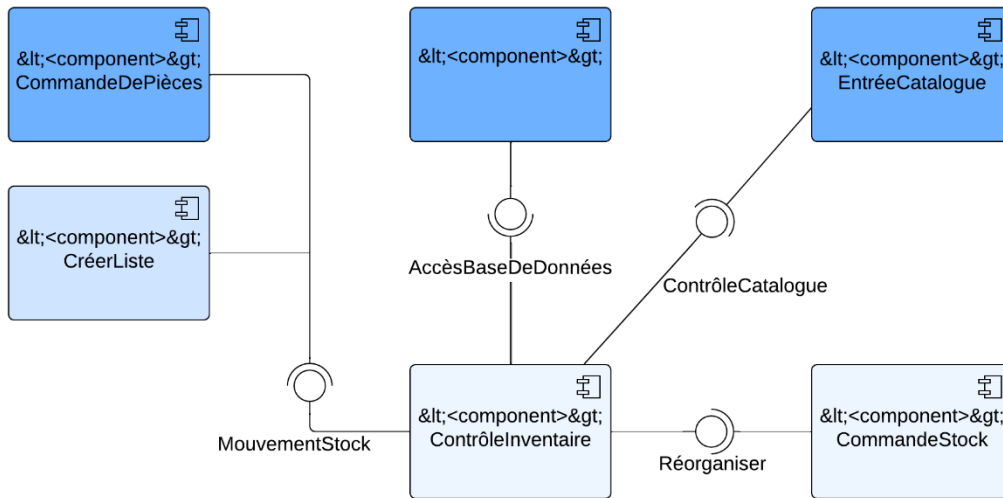


Figure 33 formalisme du diagramme de composant

Le diagramme suivant montre la structure des composants logiciels du système :

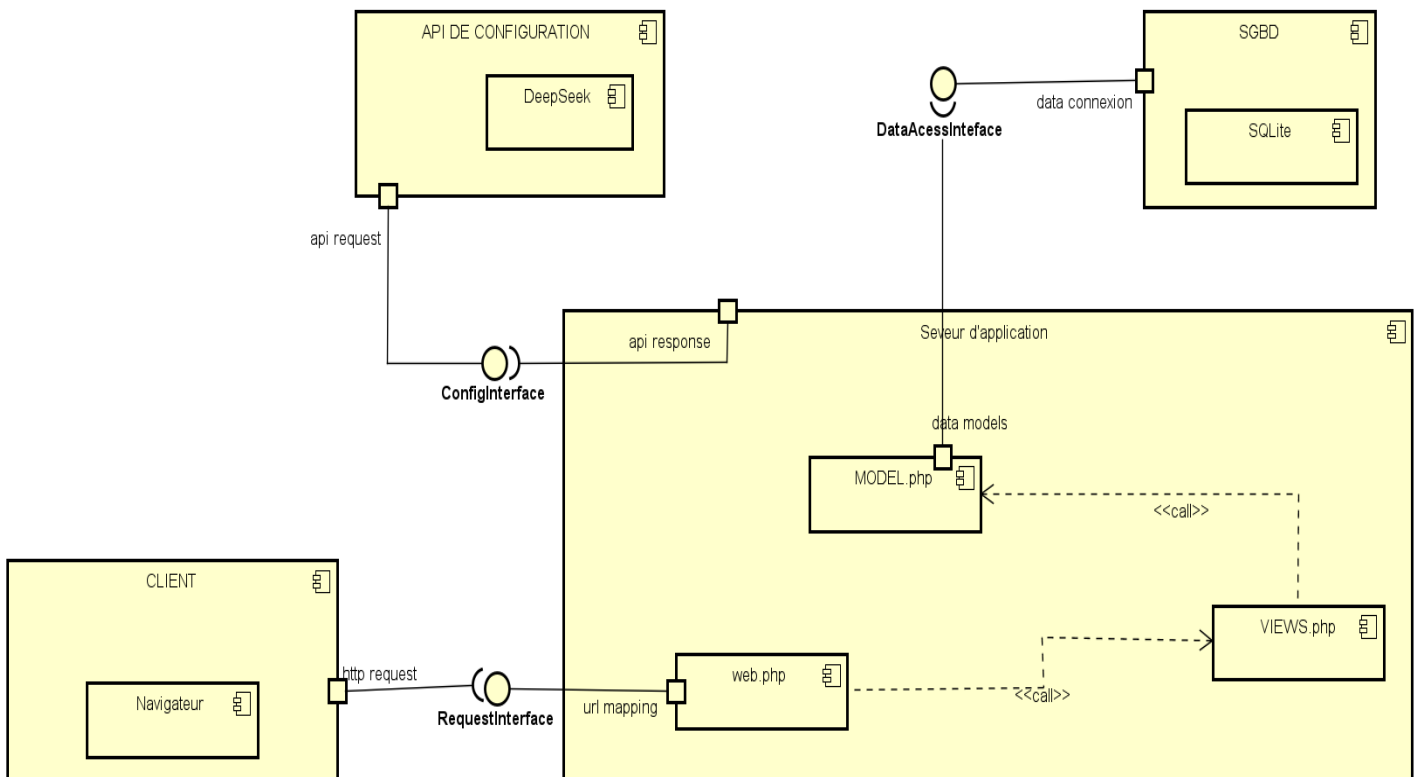


Figure 34 diagramme de composant de notre solution



Conclusion

Le dossier de réalisation nous a permis de présenter de façon générale les outils utilisés pour la réalisation de notre application. Cependant, connaître toutes les fonctionnalités du système est d'une importance capitale. La prochaine partie sera donc dédiée au test de fonctionnalités.



DOSSIER 6 : TEST DE FONCTIONNALITÉS

Résumé :

Un test désigne une procédure de vérification partielle d'un système. Son objectif principal est d'identifier un nombre maximal de comportements problématiques du logiciel. Il permet ainsi, dès lors que les problèmes identifiés seront corrigés, d'en augmenter la qualité.

Aperçu :

Introduction

- 1. Fonctionnalités**
- 2. Tests**

Conclusion



Introduction

Les tests de fonctionnalités sont une étape cruciale dans le cycle de vie du développement logiciel. Ils consistent à vérifier que les différentes fonctionnalités d'une application ou d'un système fonctionnent comme prévu. Ces tests se concentrent sur l'interaction entre l'utilisateur et le système et visent à valider que les fonctionnalités répondent aux exigences spécifiées. Contrairement aux tests unitaires, qui examinent chaque composant individuellement, les tests fonctionnels couvrent un ensemble de composants ou de modules intégrés pour s'assurer que l'application fonctionne correctement dans son ensemble. Ces tests aident à détecter les défauts qui peuvent avoir été introduits pendant le processus de développement, garantissent la conformité aux besoins des utilisateurs et identifient les éventuelles déviations par rapport aux spécifications. Dans un contexte où la qualité du logiciel est primordiale.



I. Fonctionnalités

Les fonctionnalités de notre application web sont définies en prenant en comptes les besoins et les attentes des utilisateurs. Elles désignent les différentes actions du système qui permettent à l'utilisateur d'effectuer des tâches spécifiques. Ainsi les fonctionnalités de notre application web sont les suivantes :

- ❖ **Montage d'une architecture physique personnalisée** : l'utilisateur construire son architecture physique à l'aide d'un éditeur graphique de type glissé-déposer.
- ❖ **Configuration d'un réseau informatique** : le réseau de l'utilisateur est configuré automatiquement par l'intelligence artificielle.
- ❖ **Déploiement d'un réseau informatique** : l'utilisateur pro, celui qui a payé un abonnement, a la possibilité de déployer son réseau sur un site physique.
- ❖ **Suivre des cours** : les utilisateur on la possibilité de suivre des cours dans la plateforme question d'augmenter leurs compétences dans le domaine du réseau.

II. Tests

1. Les types de tests fonctionnels

- ✚ **Tests unitaires** : Les tests unitaires vérifient le bon fonctionnement de composants individuels ou de petites unités de code, généralement des fonctions ou des méthodes. L'objectif est de s'assurer que chaque unité de code fonctionne comme prévu, indépendamment des autres parties du système. Ces tests sont généralement écrits et exécutés par les développeurs et permettent de détecter des erreurs dès le début du cycle de développement, ce qui facilite le débogage et améliore la qualité du code ;
- ✚ **Tests d'acceptation** : Ces tests vérifient si le système répond aux exigences et aux attentes des utilisateurs finaux. Ils sont souvent réalisés par les utilisateurs eux-mêmes pour s'assurer que le produit est conforme à leurs besoins ;
- ✚ **Tests d'intégration** : Ces tests se concentrent sur l'interaction entre différents modules ou composants d'une application. L'objectif est de s'assurer que les

modules fonctionnent ensemble comme prévu et que les données circulent correctement entre eux ;

- ✚ **Tests de système** : Les tests de système évaluent l'application dans son ensemble, en vérifiant que toutes les fonctionnalités fonctionnent comme spécifier dans les exigences. Cela inclut des tests sur l'interface utilisateur, la performance et la sécurité ;
- ✚ **Tests de régression** : Ces tests sont effectués après des modifications du code (comme des corrections de bugs ou des mises à jour) pour s'assurer que les nouvelles modifications n'ont pas introduit de nouveaux défauts dans les fonctionnalités existantes.
- ✚ **Tests de performance** : Ces tests mesurent la réactivité, la rapidité et la stabilité d'une application sous diverses charges. Ils visent à s'assurer que le système peut gérer le volume prévu d'utilisateurs et de transactions ;
- ✚ **Tests de compatibilité** : Ces tests vérifient que l'application fonctionne correctement sur différentes plateformes, navigateurs, systèmes d'exploitation et appareils. Cela garantit une expérience utilisateur cohérente quel que soit le contexte d'utilisation ;
- ✚ **Tests de sécurité** : Ces tests évaluent la sécurité de l'application, en vérifiant les vulnérabilités et les failles potentielles qui pourraient être exploitées par des attaquants. Ils incluent des tests d'authentification, d'autorisation et de protection des données ;
- ✚ **Tests d'usabilité** : Ces tests examinent l'expérience utilisateur et la facilité d'utilisation de l'application. Ils sont souvent réalisés avec de vrais utilisateurs qui interagissent avec le système, permettant d'identifier des problèmes d'ergonomie ou de navigation ;
- ✚ **Tests de localisation** : Ces tests s'assurent que l'application est adaptée à différents marchés et cultures, en vérifiant les traductions, les formats de date, les symboles monétaires, et d'autres éléments spécifiques à chaque région ;
- ✚ **Tests de montée en charge** : Ces tests vérifient comment l'application se comporte sous une charge croissante. Ils aident à identifier les points de rupture et à s'assurer que le système peut évoluer avec la demande des utilisateurs.

2. Procédure pour effectuer des tests en Laravel :

Arrivé à l'étape cruciale des tests dans le développement de notre application NetBot, nous avons mis en place une stratégie de test complète utilisant l'écosystème de test de Laravel combiné à notre architecture Vue.js. Cette procédure s'est articulée autour de plusieurs principes fondamentaux :

- **Création de classes de test héritant de TestCase** : Nous avons développé des classes de test qui étendent la classe de base TestCase de Laravel, permettant ainsi de bénéficier de toutes les fonctionnalités intégrées du framework, notamment la gestion automatique des bases de données de test et les helpers d'assertion spécifiques à Laravel.
- **Utilisation de setUp() pour initialiser l'environnement de test** : La méthode setUp() a été systématiquement employée pour préparer l'environnement de test avant l'exécution de chaque méthode de test. Cette approche nous a permis de créer des objets factices, de peupler la base de données avec des données de test, et d'authentifier des utilisateurs virtuels, garantissant ainsi que chaque test démarre avec un contexte propre et reproductible.
- **Implémentation de méthodes de test spécifiques** : Chaque méthode de test a été nommée avec le préfixe test_ et conçue pour vérifier des fonctionnalités précises de notre application - que ce soit les modèles Eloquent pour la persistance des données, les contrôleurs pour la logique métier, les formulaires de validation, ou l'intégration avec les composants Vue.js. Ces méthodes utilisent des assertions Laravel spécialisées comme assertStatus(), assertJsonStructure(), et assertDatabaseHas() pour valider le comportement attendu.
- **Exécution des tests via php artisan test** : La commande php artisan test a été utilisée pour lancer la suite de tests complète. Laravel exécute alors tous les tests, gère automatiquement l'environnement de test, et génère un rapport détaillé indiquant les tests réussis et ceux ayant échoué, avec des messages d'erreur explicites pour faciliter le débogage.

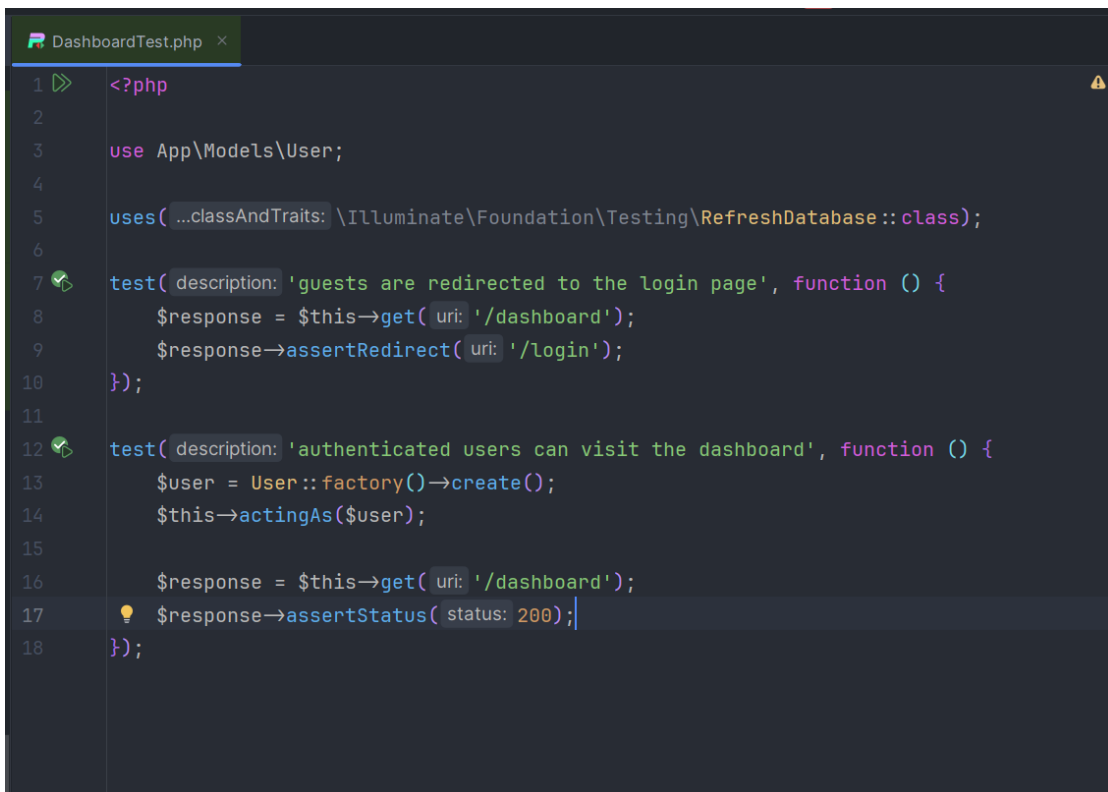
Pour les composants Vue.js spécifiques, nous avons complété cette approche avec Laravel Dusk pour simuler les interactions utilisateur réelles et valider le rendu des

composants frontend. L'ensemble de cette procédure nous a permis de garantir la robustesse de notre application NetBot tout en facilitant le développement continu grâce à la détection précoce des régressions.

3. Différents tests effectués

1. Tests unitaires (tests de vue)

Ces tests vérifient le bon fonctionnement des vue Laravel, en particulier les vues d'informations de l'utilisateur, de la liste des réseaux de l'utilisateur. Les tests incluent également la vérification du statut de réponse HTTP et l'utilisation du bon modèle.



```
1 <?php
2
3 use App\Models\User;
4
5 uses( ...classAndTraits: \Illuminate\Foundation\Testing\RefreshDatabase::class);
6
7 test( description: 'guests are redirected to the login page', function () {
8     $response = $this->get( uri: '/dashboard');
9     $response->assertRedirect( uri: '/login');
10 });
11
12 test( description: 'authenticated users can visit the dashboard', function () {
13     $user = User::factory()->create();
14     $this->actingAs($user);
15
16     $response = $this->get( uri: '/dashboard');
17     $response->assertStatus( status: 200);
18 });
```

Figure 35 dashboard_test

```
DashboardTest.php PasswordUpdateTest.php LearningControllerTest.php networkModelTest.php x
7
8 uses( ...classAndTraits: RefreshDatabase::class);
9
10 describe( description: 'Network Model', function () {
11     test( description: 'has fillable attributes', function () {
12         $fillable = [
13             'user_id',
14             'name',
15             'routing_type',
16             'addressing_type',
17             'network_configuration',
18             'is_deployed',
19         ];
20
21         $network = new Network();
22         expect($network→getFillable())→toEqual($fillable);
23     });
24
25     test( description: 'belongs to user', function () {
26         $user = User::factory()→create();
27         $network = Network::factory()→for($user)→create();
28
29         expect($network→user)→toBeInstanceOf( class: User::class);
30         expect($network→user→id)→toBe($user→id);
31     });
32 }
```

Figure 36 network vue test

```
97 test( description: 'scope for deployed networks', function () {
98     Network::factory()→create(['is_deployed' ⇒ true]);
99     Network::factory()→create(['is_deployed' ⇒ false]);
100
101     $deployedNetworks = Network::where('is_deployed', true)→get();
102
103     expect($deployedNetworks)→toHaveCount( count: 1);
104     expect($deployedNetworks→first()→is_deployed)→toBeTrue();
105 });
106
107 test( description: 'scope for user networks', function () {
108     $user1 = User::factory()→create();
109     $user2 = User::factory()→create();
110
111     Network::factory()→for($user1)→create();
112     Network::factory()→for($user2)→create();
113
114     $user1Networks = Network::where('user_id', $user1→id)→get();
115
116     expect($user1Networks)→toHaveCount( count: 1);
117     expect($user1Networks→first()→user_id)→toBe($user1→id);
118 });
```

Figure 37 réseau déployé test

```
1 <?php
2
3 uses( ...classAndTraits: \Illuminate\Foundation\Testing\RefreshDatabase::class);
4
5 test( description: 'registration screen can be rendered', function () {
6     $response = $this->get( uri: '/register');
7
8     $response->assertStatus( status: 200);
9 });
10
11 test( description: 'new users can register', function () {
12     $response = $this->post( uri: '/register', [
13         'name' => 'Test User',
14         'email' => 'test@example.com',
15         'password' => 'password',
16         'password_confirmation' => 'password',
17     ]);
18
19     $this->assertAuthenticated();
20     $response->assertRedirect( route( name: 'dashboard', absolute: false));
21 });
```

Figure 38 registration page test

2. Tests d'intégration

Ces tests vérifient la création correcte des objets dans la base de données, ici les utilisateurs ainsi que les réseaux des utilisateurs.

```
describe( description: 'Network Model - Factory', function () {  
  test( description: 'creates network with valid data', function () {  
    $network = Network::factory()→create();  
  
    expect($network)→toBeInstanceOf( class: Network::class)  
      →and($network→name)→toBeString()  
      →and($network→user_id)→not→toBeNull();  
  });  
  
  test( description: 'can create network with specific attributes', function () {  
    $user = User::factory()→create();  
    $network = Network::factory()→create([  
      'name' ⇒ 'Custom Network',  
      'user_id' ⇒ $user→id,  
      'is_deployed' ⇒ true  
    ]);  
  
    expect($network→name)→toBe( expected: 'Custom Network')  
      →and($network→user_id)→toBe($user→id)  
      →and($network→is_deployed)→toBeTrue();  
  });  
});
```

Figure 39 création d'un réseau test

```
test( description: 'can create network with configuration', function () {  
  $config = '{"topology": "star", "devices": 5}';  
  $network = Network::factory()→create([  
    'network_configuration' ⇒ $config  
  ]);  
  
  expect($network→network_configuration)→toBe($config);  
});
```

Figure 40 création de réseau avec configuration

3. Résultats des tests

```
✓ email verification screen can be rendered 2.05s
✓ email can be verified 0.03s
✓ email is not verified with invalid hash 0.03s

PASS Tests\Feature\Auth\PasswordConfirmationTest
✓ confirm password screen can be rendered 2.05s
✓ password can be confirmed 0.03s
✓ password is not confirmed with invalid password 0.23s

PASS Tests\Feature\Auth\PasswordResetTest
✓ reset password link screen can be rendered 2.06s
✓ reset password link can be requested 0.25s
✓ reset password screen can be rendered 2.30s
✓ password can be reset with valid token 0.26s

PASS Tests\Feature\Auth\RegistrationTest
✓ registration screen can be rendered 2.06s

tbot > tests > Feature > Auth > AuthenticationTest.php PHP: 8.2 161.26 LF UTF-8 4 spaces
```

Figure 41 résultat des tests

```
✓ Test Results 3sec 350ms ✓ 1 test passed 1 test total, 3 sec 350 ms
"C:\Users\Nkono Ndeme Miguel\.config\herd\bin\php84.bat" "C:\Users\Nkono Ndeme Miguel\Herd\netbot\vendor\pestphp\pe
Tests: 1 passed (3 assertions)
Duration: 4.04s
Process finished with exit code 0
```

Figure 42 suite résultat des tests

Nous avons exécuté la commande ***php artisan test*** dans le terminale intégré de phpstorm pour exécuter tout les tests. Les tests se trouve dans le dossier *tests* à la racine du projet.



Conclusion

Les tests de fonctionnalités sont essentiels pour garantir la qualité globale du logiciel. Ils valident non seulement que les fonctionnalités clés fonctionnent comme prévu, mais aussi que l'expérience utilisateur répond aux attentes. En identifiant les défauts et les lacunes, ils contribuent à améliorer la robustesse, la fiabilité et les performances du logiciel avant sa publication. Les tests fonctionnels contribuent ainsi à la satisfaction des utilisateurs et sont un élément central du processus de développement logiciel.



DOSSIER 7 : GUIDE D'INSTALLATION ET GUIDE D'UTILISATEUR

Résumé :

Le manuel d'installation et d'utilisation est le document établi après réalisation d'une application, d'un logiciel ou d'une plateforme. Il renseigne sur la question « comment installer et utiliser le logiciel, l'application ou la plateforme que l'on a devant soi ? »

Aperçu :

Introduction

- 1. Guide d'installation**
- 2. Guide utilisateur**

Conclusion



Introduction

Ce manuel propose des directives exhaustives et opérationnelles pour vous accompagner à travers les étapes d'installation, de configuration et de lancement de l'application. Son but est d'assurer une expérience sans accroc pour les utilisateurs à venir et de faciliter la prise en main de l'application, en expliquant de manière concise et accessible ses fonctionnalités, son utilisation, et les étapes à suivre pour tirer le meilleur parti de ses capacités. Dans cette section, nous vous offrirons une vue d'ensemble des principales phases requises pour installer l'application, en mettant en évidence les éléments essentiels qui simplifieront la marche à suivre du système ensuite nous présenterons un aperçu des principales caractéristiques de l'application et des actions que les utilisateurs peuvent entreprendre, leur permettant ainsi de profiter pleinement de cet outil de recherche de logement convivial et efficace.

I. Guide d'installation

Afin de réaliser une installation adéquate de l'application en local, nous nécessitons des outils suivants en dur dans notre machine : un navigateur, un serveur, une navigateur web, web, nodejs et php, etc....

Nous mettons à disposition le fichier exécutable du projet. Ci-dessous vous trouverez les instructions à suivre :

- ✓ Installation de nodejs (version 22) ;
- ✓ Rendez vous sur cette page pour télécharger la version de nodejs adéquate : <https://nodejs.org/en/download>.

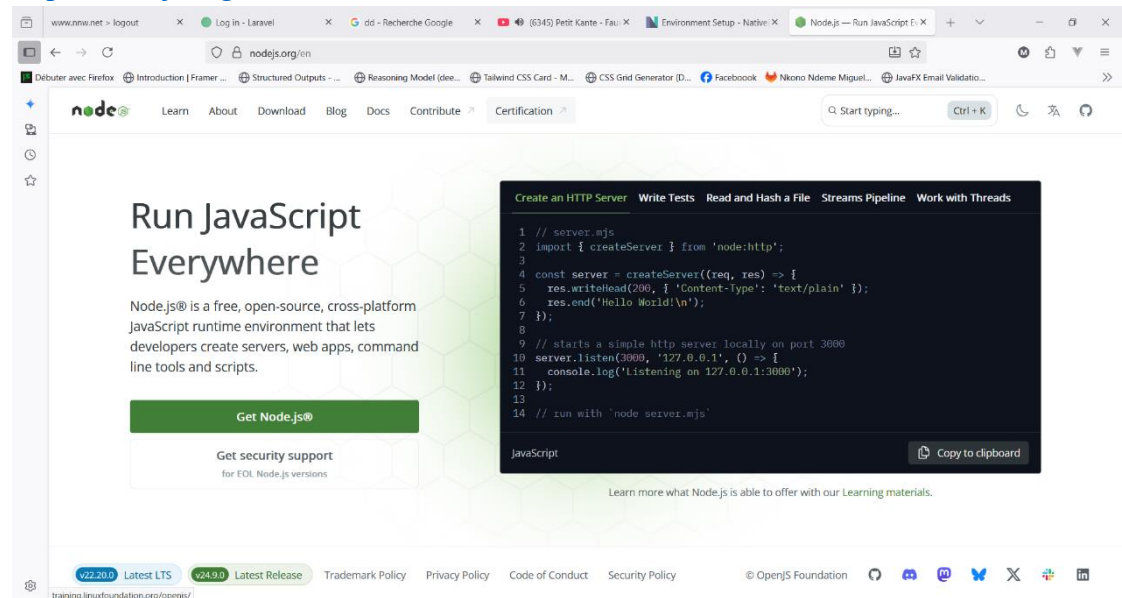


Figure 43 page d'installation de nodejs

✓ Cela vous amènera sur cette page :

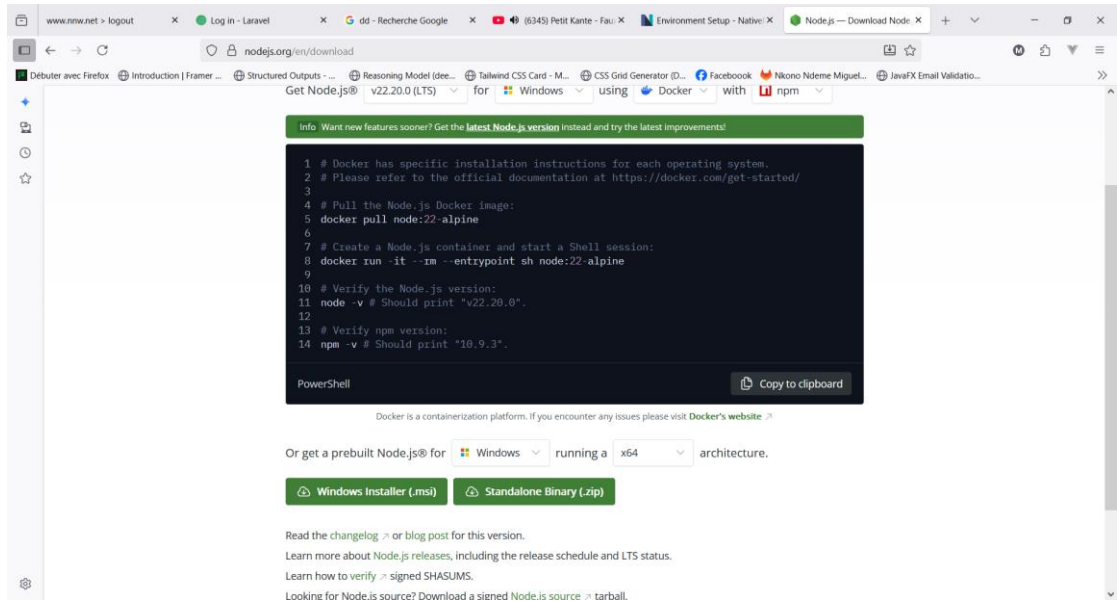


Figure 44 page d'installation de nodejs pour windows

- ✓ Puis cliquez sur *Windows Installer (.msi)* pour télécharger l'exécutable d'installation et vous suivez simplement les étapes d'installation.
- ✓ Puis vous vous rendez sur ce site pour télécharger php, laravel et composer : <https://php.new/> ;

✓ Vous verrez cette page :

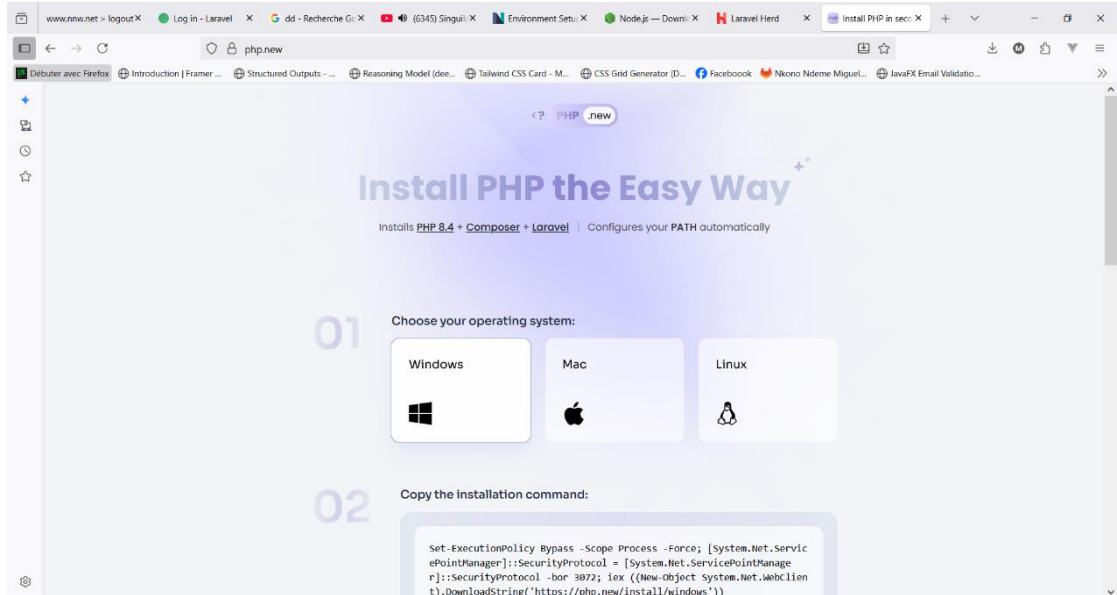


Figure 45 page d'installation de php, laravel et composer

- ✓ Vous suivez les étapes d'installation comme indiqué dans le site.
- ✓ Et en suite vous vous rendez sur ce site pour télécharger le serveur d'application : <https://herd.laravel.com/download/latest/windows> ;

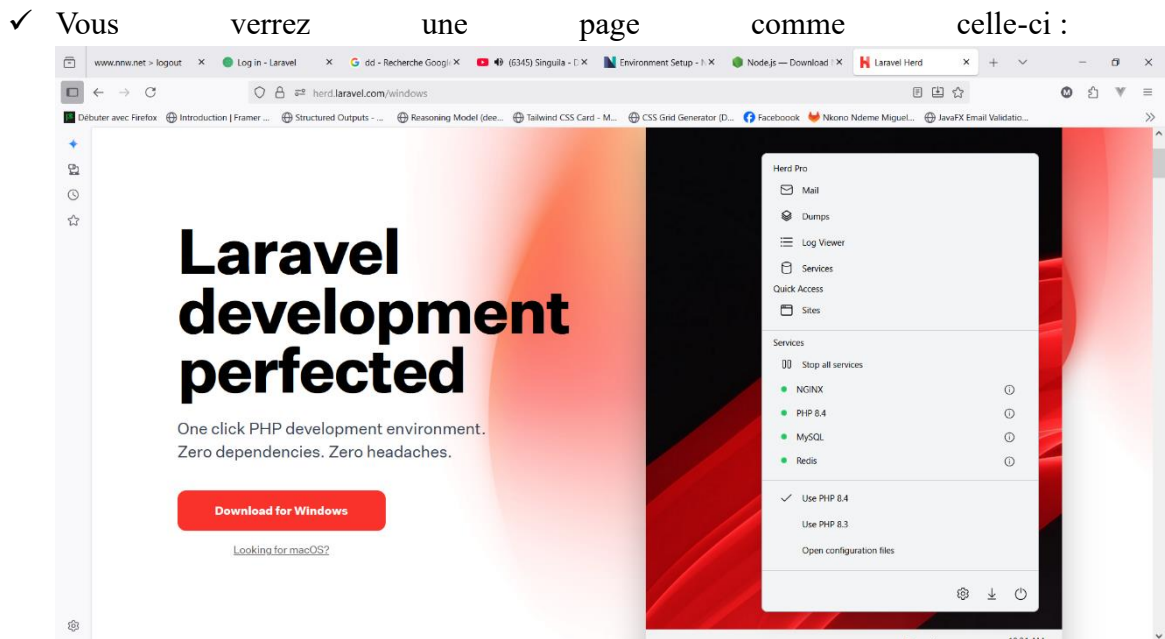


Figure 46 page d'installation de herd

- ✓ Cliquez sur *Download for Windows* pour télécharger *Herd* pour windows;
- ✓ Suivez les instructions d'installations de herd ;
- ✓ Enfin pour vérifier que tous les outils on proprement été installé dans la machine on clique tape les commandes suivant dans votre terminal ou votre session powershell :
 - Pour nodejs
 - **Node –version**
 - **Npm –version**
 - Pour php, laravel et compser :
 - **Php –version**
 - **Laravel –version**
 - **Composer –version**
- ✓ Si tous c'est bien passé, votre machine serrez déjà bien configuré pour exécuter l'application en local dans votre machine.

II. Guide utilisateur

1. La page d'accueil

Pour accéder à l'application et commencé à l'utiliser suivez les différentes étapes ci-dessous :

- ✓ Ouvrez votre terminal depuis la barre de recherche de Windows puis accéder au répertoire de l'application en utilisant la command **cd** comme suite : **cd repertoire_contenant_le_projet/** ;
- ✓ Etant dans le répertoire du projet exécuter la commande suivante : **composer run dev** pour lancer le serveur d'application et l'application ;
- ✓ Ouvrez votre navigateur et insérez dans la barre de recherche <http://127.0.0.1:8000/> ou <http://netbot.test/> et cela vous renvoyer sur la page d'accueil suivante :

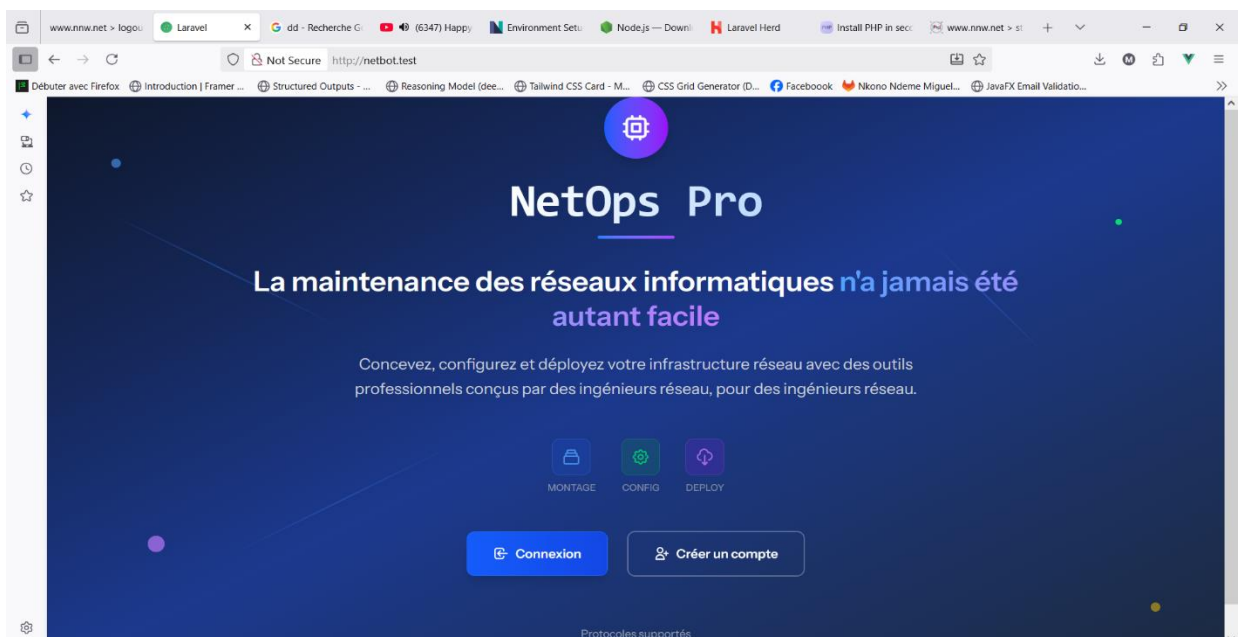


Figure 47 page d'accueil de l'application



PLATEFORME DE CONCEPTION, CONFIGURATION AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX INFORMATIQUE : cas de NetBot



✓ Page de connexion :

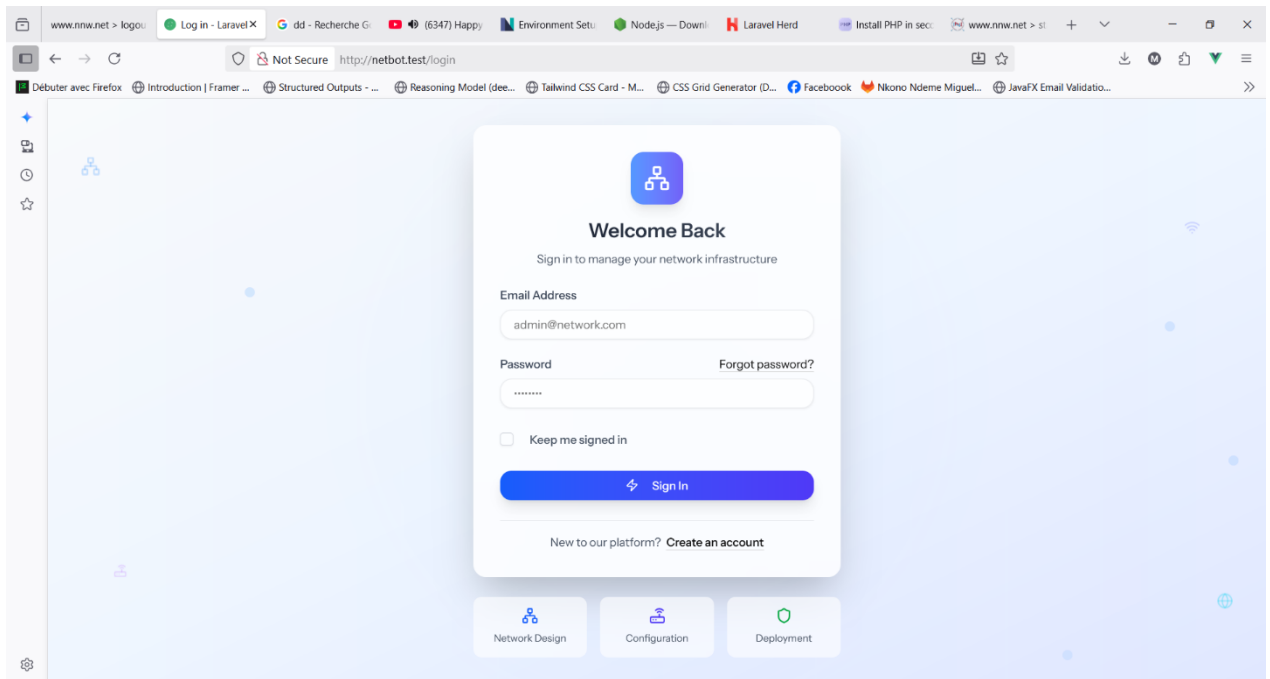


Figure 48 page de connexion

2. Montage de réseau physique

L'utilisateur a la possibilité monter son réseau, le configurer manuellement ou automatiquement et le déployer etc ...

✓ Dashboard de l'utilisateur

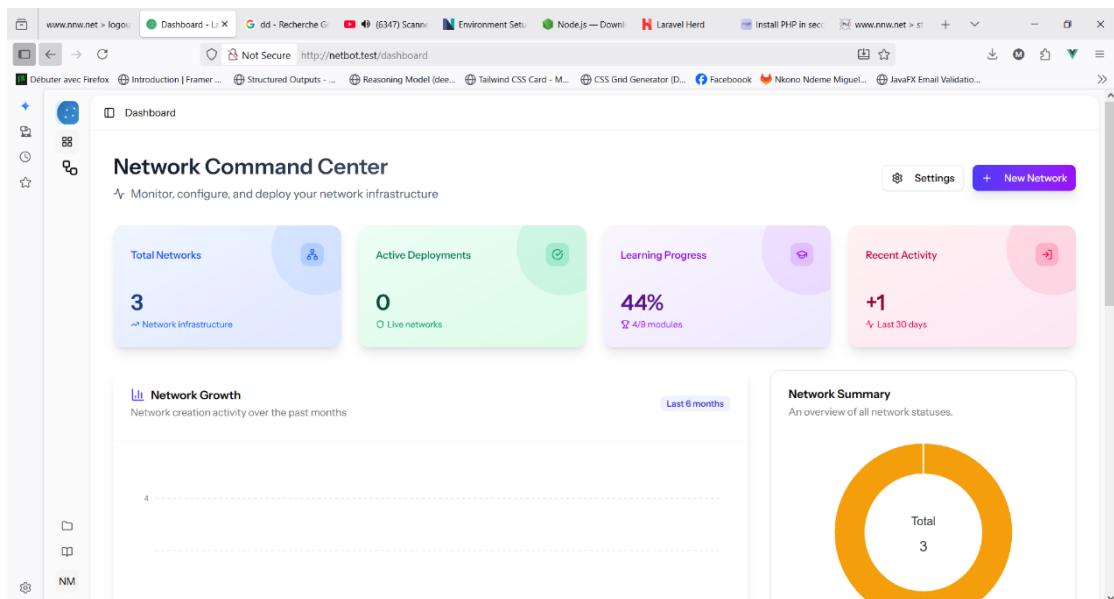


Figure 49 dashboard

✓ Page de création d'un réseau

Figure 50 page de création de réseau

✓ Page de gestion des réseaux de l'utilisateur

Figure 51 page de gestion de reseau de l'utilisateur

✓ Page de montage de réseau

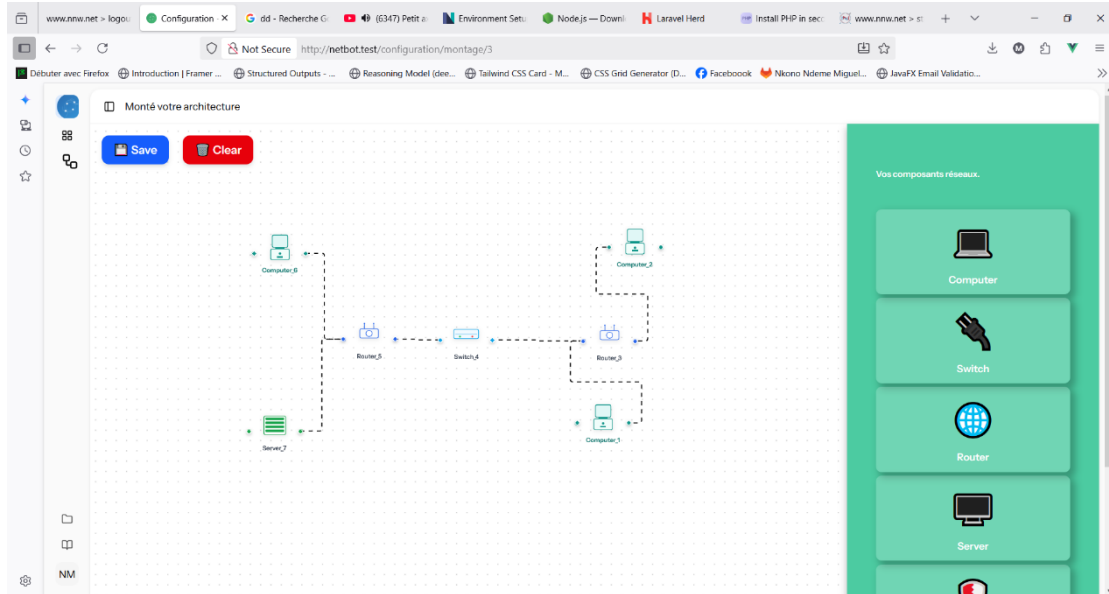


Figure 52 page de montage reseau physique

✓ Page de configuration d'un réseau :

Nom de l'équipement	Type	Adresse IP
Computer_1	Computer	192.168.64.54
Computer_2	Computer	192.168.70.20
Router_3	Router	192.168.45.10

Figure 53 page de description du reseau



PLATEFORME DE CONCEPTION, CONFIGURATION AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX INFORMATIQUE : cas de NetBot



www.netbot.net > login

Modifier réseau: X

did - Recherche G

(6347) Elang

Environnement Setu

Node.js - Downl

Laravel / lend

Install PHP in sec

www.netbot.net > s

Not Secure http://netbot.test/network/3/edit

Debiter avec Firefox

Introduction | Framers ...

Structured Outputs - ...

Reasoning Model (dee...

Tailwind CSS Card - M...

CSS Grid Generator (D...

Facebook

Nkono Ndeme Miguel...

JavaFX Email Validatio...

Retour aux détails

Modifier réseau configurer automatiquement

Annuler Enregistrer

Modifiez les paramètres et équipements de votre réseau.

Informations principales

Configuration de base de votre réseau

Nom du réseau

réseau configurer automatiquement

Type de routage

OSPF

Type d'adressage

IPv4

Équipements (7)

Configurez les équipements de votre réseau

+ Ajouter

Équipement 1

Nom de l'équipement

Computer_1

Type d'équipement

Ordinateur

Adresse IP

192.168.64.54

Node ID

1

Configuration (optionnel)

Commandes de configuration pour cet équipement...

Figure 54 page de configuration manuelle du réseau



Conclusion

Malgré la simplicité de notre application nous avons tenu à établir ce manuel utilisateur afin de présenter de façon claire et précise le processus d'installation puis d'utilisateur. Ce guide contient toutes les informations permettant aux futurs utilisateurs de connaître l'environnement et les fichiers nécessaires au bon fonctionnement du logiciel.



CONCLUSION GÉNÉRALE

Arrivé au terme de ce projet réalisé dans le cadre de l'obtention du diplôme d'Ingénieur en Informatique, nous avons eu l'opportunité de concevoir et développer une solution innovante dans le domaine de l'administration réseau, visant à résoudre des problématiques concrètes rencontrées par les professionnels et étudiants en réseaux informatiques. Notre projet de plateforme SaaS d'automatisation réseau intelligente NetBot s'est articulé autour de plusieurs phases essentielles : l'étude de l'existant qui nous a permis d'analyser les limites des solutions actuelles comme Cisco Packet Tracer et les outils d'automatisation professionnelle ; l'analyse fonctionnelle et technique qui a défini précisément les besoins et objectifs du système ; la conception architecturale qui a établi les fondations d'une solution intégrée et modulaire ; et enfin la réalisation effective de l'application grâce à l'utilisation combinée de technologies modernes telles que Laravel, Vue.js, Python Flask, TailwindCSS, ainsi que divers outils spécialisés comme GNS3 pour la simulation réseau, Git pour le contrôle de version, et les environnements de développement PhpStorm et PyCharm.

Cette expérience nous a permis de mettre en pratique nos connaissances théoriques et de développer nos compétences techniques dans un contexte professionnel, tout en nous confrontant aux défis réels de la gestion de projet informatique et du développement d'une application complexe. Nous avons ainsi pu concevoir une plateforme web innovante qui permet aux administrateurs réseau et étudiants de bénéficier d'un environnement unifié pour la modélisation visuelle de topologies, la génération automatique de configurations multi-vendeurs, la validation intelligente par IA, et l'apprentissage adaptatif - le tout accessible via une interface intuitive et responsive.

De ce fait, comment garantir que l'automatisation intelligente des configurations réseau s'adapte efficacement aux besoins spécifiques de chaque organisation tout en restant accessible aux utilisateurs de différents niveaux techniques, et comment assurer l'évolution continue de la plateforme pour intégrer les nouvelles technologies réseau et les avancées en intelligence artificielle ?



BIBLIOGRAPHIE

- Fowler, M. (2018). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional. → Référence essentielle pour l'architecture des applications d'entreprise, particulièrement pertinente pour notre stack Laravel/Vue.js.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. → Fondamental pour la conception de l'architecture logicielle et l'implémentation des patterns dans notre plateforme.
- Stalling, W. (2021). *Foundations of Modern Networking*. Pearson Education. → Couvre les principes réseaux essentiels implémentés dans notre module d'automatisation.
- Laravel Documentation Team (2024). *Laravel Documentation* [En ligne]. Disponible à : <https://laravel.com/docs> → Documentation officielle complète pour le développement backend avec Laravel.
- Vue.js Core Team (2024). *Vue.js Guide* [En ligne]. Disponible à : <https://vuejs.org/guide/> → Référence officielle pour le développement frontend avec Vue.js et l'intégration avec Laravel.
- Freeman, A. (2020). *Pro Vue.js 3*. Apress. → Guide avancé pour la création d'applications interactives avec Vue.js 3.
- Doyle, J. & Carroll, J. (2021). **Routing TCP/IP, Volume 1**. Cisco Press. → Référence approfondie sur les protocoles de routage implémentés dans NetBot.
- Odom, W. (2020). **CCNA 200-301 Official Cert Guide**. Cisco Press. → Standards industriels pour les configurations réseau supportées par notre plateforme.
- Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media. → Principes architecturaux pour notre approche microservices avec Laravel et Flask.
- Netmiko Documentation (2024). *Netmiko Multi-vendor Library* [En ligne]. Disponible à : <https://github.com/ktbyers/netmiko> → Bibliothèque Python pour l'automatisation réseau multi-vendeurs.
- Batfish Project (2024). *Batfish Analysis Tool* [En ligne]. Disponible à : <https://github.com/batfish/batfish> → Outil d'analyse sémantique des configurations réseau.



WEBOGRAPHIE

- <https://www.ansi.org/resource-center/annual-report-2023> Rapport annuel de l'ANSI détaillant les priorités en matière de normalisation technique, notamment dans les domaines de la cybersécurité et des infrastructures critiques. Il souligne l'importance des standards pour l'innovation technologique, alignée sur les besoins de sécurisation des réseaux (visité le 05/08/2025 à 13 : 03).
- <https://patnuc.cm/2025/07/11/numerique-pour-tous-le-cameroun-valide-sa-strategie-nationale-de-service-universel/> Document officiel validant la stratégie camerounaise d'inclusion numérique (2025-2030). Il cadre avec la vision "Cameroon Digital 2030" et les objectifs de couverture réseau nationale, notamment le déploiement d'infrastructures hybrides (fibre optique, satellite) et la formation des populations vulnérables (visité le 05/08/2025 à 13 : 03).
- <https://www.bmz-digital.global/en/initiatives/digital-transformation-center-cameroon/> Présentation des initiatives soutenues par la coopération allemande pour moderniser l'écosystème numérique camerounais. Inclut des données clés sur l'accessibilité internet (44% d'utilisateurs) et les projets de parcs technologiques à Douala et Yaoundé (visité le 05/08/2025 à 13 : 03).
- <https://laravel.com/docs/11.x> Documentation officielle de Laravel 11.x couvrant l'architecture MVC, l'éloquent ORM, et le système de tests PHPUnit, essentiels pour le développement backend de NetBot (visité le 05/08/2025 à 13:15).
- <https://vuejs.org/guide/> Guide complet de Vue.js 3 détaillant la Composition API, les composables et l'intégration avec Laravel, fondamentaux pour l'interface utilisateur interactive de NetBot (visité le 05/08/2025 à 13:18).
- <https://flask.palletsprojects.com/> Documentation officielle de Flask Python, framework utilisé pour les microservices d'automatisation réseau et l'intégration IA dans NetBot (visité le 05/08/2025 à 13:20).
- <https://github.com/ktbyers/netmiko> Bibliothèque Python multi-vendeurs pour l'automatisation réseau, utilisée dans NetBot pour la génération et le déploiement des configurations (visité le 05/08/2025 à 13:22).



**PLATEFORME DE CONCEPTION, CONFIGURATION
AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX
INFORMATIQUE : cas de NetBot**



Annexes



TABLE DES MATIÈRES

DÉDICACE	III
REMERCIEMENTS.....	IV
SOMMAIRE	V
LISTE DES TABLEAUX.....	VI
LISTE DES FIGURES	VII
GLOSSAIRE.....	VIII
RESUME	IX
ABSTRACT.....	X
INTRODUCTION GÉNÉRALE	2
PARTIE I : PHASE D'INSERTION	3
I. ACCUEIL ET INTÉGRATION EN ENTREPRISE.....	5
1. Accueil en entreprise.....	5
2. Intégration en entreprise	5
II. PRESENTATION DE BATCH SMART TECHONOLOGIES	6
a. Historique.....	6
b. Situation géographique	6
c. Organigramme	7
d. Missions	8
e. Activités et partenaires.....	9
f. Ressources de l'entreprise.....	10
PARTIE II : PHASE TECHNIQUE	12
INTRODUCTION	13
DOSSIER 1 : L'EXISTANT	14
I. PRÉSENTATION DU THÈME/PROJET.....	16
II. ETUDE DE L'EXISTANT	17
III. CRITIQUE DE L'EXISTANT	18
IV. PROBLÉMATIQUE	19
V. PROPOSITION DE SOLUTION	20
DOSSIER 2 : CAHIER DES CHARGES.....	22
I. CONTEXTE ET JUSTIFICATION DE L'ÉTUDE /DU PROJET	24
1. Contexte	24



**PLATEFORME DE CONCEPTION, CONFIGURATION
AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX
INFORMATIQUE : cas de NetBot**



2.	Justification	25
II.	LES OBJECTIFS DE L'ÉTUDE/PROJET	26
1.	Objectif général.....	26
2.	Objectifs spécifiques	26
III.	EXPRESSIONS DES BESOINS DE L'UTILISATEUR.....	27
1.	Besoins fonctionnels	27
2.	Besoins non fonctionnels	29
IV.	PLANIFICATION DU PROJET/ GANTT PROJECT	31
1.	Intervenants.....	31
2.	Planning de travail	31
3.	Diagramme de Gantt	32
V.	Estimation du coût du projet/étude	34
1.	Ressources Matérielles.....	34
2.	Ressources logicielles	34
3.	Ressources humaines	35
4.	Cout total.....	35
VI.	CONTRAINTES DU PROJET / ÉTUDE.....	36
VII.	Les livrables	36
DOSSIER 3 : DOSSIER D'ANALYSE.....		38
I.	METHODOLOGIE.....	40
1.	Etude comparative UML et MERISE	40
2.	Choix et présentation de la démarche d'analyse.....	41
3.	Etude comparative des processus unifiés.....	43
4.	Choix et présentation du processus unifié	44
II.	MODÉLISATION	45
1.	Diagramme des cas d'utilisation.....	45
2.	Descriptions textuelles	50
3.	Diagramme de communication	61
4.	Diagramme de séquence	64
5.	Diagramme d'activité.....	71
DOSSIER 4 : DOSSIER DE CONCEPTION		77
II.	Diagramme de classe	79
1.	Diagramme de classe de notre système	82



**PLATEFORME DE CONCEPTION, CONFIGURATION
AUTOMATIQUE ET DEPLOIEMENT DES RESEAUX
INFORMATIQUE : cas de NetBot**



2. Règles métiers de notre diagramme de classe :	83
III. Diagramme d'état transition	84
IV. Diagramme de paquetage	87
DOSSIER 5 : DOSSIER DE RÉALISATION	90
I. Choix des technologies	92
II. Architecture physique et logique du système	94
1. Architecture physique	94
2. Architecture logique	96
III. Diagramme de déploiement	97
IV. Diagramme de composant	99
DOSSIER 6 : TEST DE FONCTIONNALITÉS	102
I. Fonctionnalités	104
II. Tests	104
1. Les types de tests fonctionnels	104
2. Procédure pour effectuer des tests en Laravel :	106
3. Différents tests effectués	107
DOSSIER 7 : GUIDE D'INSTALLATION ET GUIDE D'UTILISATEUR	113
I. Guide d'installation	115
II. Guide utilisateur	118
1. La page d'accueil	118
2. Montage de réseau physique	119
CONCLUSION GÉNÉRALE	124
Perspectives	
BIBLIOGRAPHIE	IX
WEBOGRAPHIE	X
TABLE DES MATIÈRES	XII